

On the Statistical vs. Computational Security of the DKLs23 Multiparty ECDSA Protocol

Gil Segev*

Hebrew University and Coinbase
segev@cs.huji.ac.il

Abstract. The DKLs23 protocol (Doerner, Kondi, Lee and shelat, IEEE S&P '24) is a state-of-the-art multiparty ECDSA signing protocol. Due to its exceptional combination of simplicity, efficiency, and statistical UC security within an elegant hybrid model providing access to standard ideal functionalities, it is rapidly seeing widespread adoption.

We provide a comprehensive security analysis of the DKLs23 protocol, showing that although it is not statistically secure as originally claimed, it is nevertheless computationally secure, and can be made statistically secure via a lightweight refinement. Our contributions are as follows:

- **Statistical insecurity.** Within the hybrid model utilized for the original analysis, we construct a computationally-unbounded “split-view” adversary that successfully attacks the protocol by causing two or more honest parties to output different valid signatures on the same message (specifically, signatures with independent nonces). Considering any ideal ECDSA signing functionality that outputs a single signature per session, such an attack cannot be simulated in the ideal model, and thus the protocol is not statistically secure.
- **Computational security.** We prove that the protocol is nevertheless computationally secure based on the assumption that ECDSA is strongly unforgeable up to sign (i.e., up to the trivial $(r, \pm s)$ malleability), as defined by Groth and Shoup (EUROCRYPT '22). Specifically, complementing our split-view attack, we show that any adversary for which the protocol’s execution is distinguishable from its ideal-model simulation can be *efficiently* transformed into an algorithm that breaks the strong unforgeability up to sign of ECDSA.
- **A refined statistically-secure protocol.** Identifying the root cause of our split-view attack, we present a refined protocol that is statistically secure. Our refinement incorporates a lightweight consistency check, where each party sends a single group element as part of the protocol’s third-round message. Our refinement additionally introduces a relaxed zero-sharing functionality that serves as a drop-in replacement for the original protocol’s zero-sharing functionality. We show that this relaxed functionality can be unconditionally realized by a simple one-round protocol (executed in parallel with the first round of signing) without a dedicated setup or pre-shared seeds. Consequently, when paired with a compatible (e.g., Paillier-based) VOLE instantiation, this eliminates the need for long-term secure pairwise storage across the entire signing protocol.

* This work was carried out while at Coinbase.

1 Introduction

Multiparty ECDSA protocols, which distribute signing authority among a quorum of parties, have become a cornerstone of digital asset management. However, unlike multiparty protocols for Schnorr signatures [Sch90] (see, for example, [MOR01, NKDM03, BN06, KG20, Mak22, Lin24, BDLR25a, BDLR25b], and the many references therein), constructing multiparty protocols for ECDSA signatures is notoriously challenging due to the non-linear structure of ECDSA. This difficulty has sparked more than two decades of exciting research, evolving from early solutions reliant on heavy, often inefficient cryptographic machinery to modern, lightweight protocols designed to meet the rigorous low-latency and high-throughput demands of blockchain applications (e.g., [Lan95, GJKR96, Lin17, DKLs18, GG18, LN18, DKLs19, CCL⁺20, CGG⁺20, ANO⁺22, GS22a, GS22b, ABC⁺24, DKLs24, KGG24]).

In this landscape, the recent protocol by Doerner, Kondi, Lee, and Shalat [DKLs23, DKLs24] (hereafter referred to as the DKLs23 protocol) stands out as a particularly attractive construction. Notable for achieving an exceptional combination of simplicity, efficiency, and statistical UC security within an elegant hybrid model providing access to standard ideal functionalities, the protocol employs a clever rewriting of the signing equation alongside lightweight consistency checks that avoid zero-knowledge proofs. This minimalist design results in a three-round signing protocol that incurs very low computational and bandwidth costs. Due to these appealing properties, the protocol is rapidly seeing widespread adoption.¹ Consequently, ensuring the rigorous security of the DKLs23 protocol is of both foundational and practical importance to the broader cryptographic community.

Our contributions. In this work we provide a comprehensive security analysis of the DKLs23 protocol, showing that although it is not statistically secure as originally claimed, it is nevertheless computationally secure, and can be made statistically secure via a lightweight refinement. Specifically, our contributions are as follows:

- **Statistical insecurity.** We show that the DKLs23 protocol does not achieve statistical UC security as originally claimed. Specifically, within the hybrid model utilized for its original analysis, we construct a computationally-unbounded “split-view” adversary that successfully attacks the protocol by causing two or more honest parties to output different valid signatures on the same message (specifically, signatures (r, s) and (r', s') with $r \neq r'$). Considering any ideal ECDSA signing functionality that outputs a single signature per session, such an attack cannot be simulated in the ideal model. Our attack relies solely on the adversary’s ability to compute discrete logarithms (all other operations performed by the adversary are polynomial time). Therefore, any security analysis establishing that the protocol is *computationally* secure within the hybrid model utilized for its original analysis

¹ See <https://dkls.info/> for an actively maintained list of implementations and deployments.

must rely at least on the hardness of the discrete logarithm problem (which is essential for the security of ECDSA).

More broadly, beyond the specific context of the DKLS23 protocol, our split-view attack highlights a subtle potential pitfall in the design of multiparty signing protocols: when the protocol does not enforce a globally consistent view of the signing nonce, a computationally unbounded adversary may exploit per-party differences to cause honest parties to output distinct valid signatures. Our work demonstrates that the absence of such a mechanism cannot generally be compensated by other consistency checks in the protocol. We view this as a cautionary design principle for future protocols that aim for statistical security in similar hybrid models.

- **Computational security.** We prove that, despite its statistical insecurity, the DKLS23 protocol is nevertheless computationally secure based on the assumption that the ECDSA signature scheme is *strongly unforgeable up to sign* (i.e., up to the trivial $(r, \pm s)$ -malleability of ECDSA signatures), as defined by Groth and Shoup [GS22b]. This assumption guarantees, in particular, that given a uniformly distributed ECDSA public key $Q = x \cdot G$ and oracle access to a signing oracle, it is computationally infeasible to produce an additional valid signature $\sigma' = (r', s')$ on a previously signed message m such that r' is distinct from the nonce of the signature provided by the oracle. Complementing our split-view attack, we show that any adversary for which the protocol’s execution is distinguishable from its ideal-model simulation can be efficiently transformed into an algorithm that breaks the strong unforgeability up to sign of ECDSA.
- **A refined statistically-secure protocol.** We identify the root cause of our split-view attack: The lack of a mechanism to enforce a globally-consistent view of the nonce used to generate the signature. To address this, we refine the protocol to incorporate a lightweight consistency check, hereafter referred to as the *nonce-consistency check*. This check requires each party to send a single additional group element as part of their third-round message, and then verify that they receive the identical element from all other parties. This simple modification ensures that all honest parties hold consistent views of the nonce used to generate the signature, enabling us to prove the statistical security of the protocol.

Our refined protocol and analysis introduce two further contributions to enhance the efficiency and modularity of the original protocol:

- **Setup-free zero-sharing.** We introduce a relaxed zero-sharing functionality, $\mathcal{F}_{\text{rZero}}$, which serves as a drop-in replacement for the zero-sharing functionality $\mathcal{F}_{\text{Zero}}$ used in the original protocol. We show that our relaxed functionality $\mathcal{F}_{\text{rZero}}$ can be unconditionally realized by a simple one-round protocol that eliminates the need for a dedicated setup or pre-shared seeds (unlike the realization suggested in [DKLS23, DKLS24]). This one-round protocol can be run in parallel with the first round of signing, preserving the overall round complexity.

The practical motivation for this relaxation is particularly strong when considering different underlying instantiations of the VOLE functional-

ity. While OT-based instantiations require the maintenance of pairwise state (e.g., for establishing and updating base OTs), this is not true for all implementations. If the VOLE functionality is instead instantiated using Paillier-based techniques (e.g., [Lin17, LN18, GG18, CGG⁺20, LS26]), there is generally no need to establish or maintain pairwise states. In such deployments, our relaxed zero-sharing functionality provides a substantial architectural advantage by eliminating the need for long-term secure pairwise storage across the entire signing protocol.

- **Decoupled security analysis.** Following Lindell [Lin24], we decouple the security analysis of the refined signing protocol from the key generation phase. This modular approach proves that the signing protocol maintains the security guarantees of a local signing operation, regardless of whether the keys were generated by a trusted dealer, a distributed protocol, or a hierarchical deterministic method (e.g., BIP-32).

While this modular abstraction captures most key-generation protocols, it abstracts away the specific “relaxed” key-generation functionality utilized by Doerner et al. [DKLs23, DKLs24], which intentionally avoids the zero-knowledge proofs typically required to extract corrupted parties’ secret key shares upfront. Therefore, to demonstrate that our refinement is fully compatible with their native architecture, we additionally prove the statistical UC security of the refined protocol when directly considering their specific key-generation and signing functionalities.

For concreteness and simplicity, we present our analysis in the full threshold setting (i.e., n -out-of- n). We note that all our results, including the attack on the original protocol and the security of our refined protocol, extend to the general threshold setting (i.e., t -out-of- n).

Paper organization. In Section 2, we present the notation and ideal functionalities used throughout the paper. In Section 3, we provide an overview of the DKLs23 protocol and then describe the refined protocol, explicitly highlighting the modifications made to the original protocol. In Section 4, we present our split-view attack on the original protocol, demonstrating its statistical insecurity in the absence of the nonce-consistency check. In Section 5, we prove that the refined protocol is statistically secure, and in Section 6 we prove that the original protocol is computationally secure.

2 Preliminaries

Notation. Let $\mathcal{G} = (\mathbb{G}, G, q)$ be a description of a cyclic group $\mathbb{G}$ of prime order q that is generated by $G \in \mathbb{G}$. Let κ_c and κ_s denote the computational and statistical security parameters, respectively. We denote scalars via lower-case letters (e.g., $a \in \mathbb{Z}_q$), vectors of scalars via boldface lower-case letters (e.g., $\mathbf{a} = (a_1, \dots, a_n) \in \mathbb{Z}_q^n$), group elements via upper-case letters (e.g., $G \in \mathbb{G}$), and vectors of group elements via boldface upper-case letters (e.g.,

$\mathbf{G} = (G_1, \dots, G_n) \in \mathbb{G}^n$). For a scalar $a \in \mathbb{Z}_q$ and a vector of scalars $\mathbf{b} = (b_1, \dots, b_n) \in \mathbb{Z}_q^n$ we let $a \cdot \mathbf{b} = (a \cdot b_1, \dots, a \cdot b_n) \in \mathbb{Z}_q^n$.

Ideal functionalities. We follow the UC framework’s standard *delayed output* mechanism [Can00], which gives the adversary control over message delivery. This mechanism, implicitly assumed for all functionalities defined below, works as follows: (1) When a functionality is ready to send an output to a party, it first sends a “ready” notification to the adversary containing a unique identifier for that specific output, (2) If the output is public then its value is included in this notification and otherwise its value is withheld to maintain confidentiality, and (3) The functionality then pauses and delivers the actual output to the party only after receiving a corresponding “continue” signal from the adversary. For the sake of brevity, the delayed output mechanism will not be explicitly included in each functionality’s steps.

- $\mathcal{F}_{\text{Com}}$. The commitment functionality is defined as follows:
 - Upon receiving $(\text{commit}, \text{sid}, i, j, x)$ from party P_i , where $i, j \in [n]$ and $\text{sid} = \text{sid}' || \text{pid}_i || \text{pid}_j$ for the respective party identifiers pid_i and pid_j of P_i and P_j , if a tuple $(\text{sid}, i, j, \cdot)$ has not been stored, then store (sid, i, j, x) and send $(\text{receipt}, \text{sid}, i, j)$ to party P_j .
 - Upon receiving $(\text{decommit}, \text{sid}, i, j, x)$ from party P_i , where $i, j \in [n]$ and $\text{sid} = \text{sid}' || \text{pid}_i || \text{pid}_j$ for the respective party identifiers pid_i and pid_j of P_i and P_j , if a tuple (sid, i, j, x) has been stored then send $(\text{decommit}, \text{sid}, i, j, x)$ to party P_j .
- $\mathcal{F}_{\text{Zero}[q, n]}$. The zero-sharing functionality, parameterized by a prime $q \in \mathbb{N}$ and an integer $n \in \mathbb{N}$, is defined as follows:
 - Upon receiving $(\text{sample}, \text{sid})$ from party P_i , where $i \in [n]$ and $\text{sid} = \text{sid}' || \text{pid}_1 || \dots || \text{pid}_n$ for the respective party identifiers $\text{pid}_1, \dots, \text{pid}_n$ of $P_1, \dots, P_n$, if a tuple $(\text{sid}, \cdot)$ has not been stored then for every $j \in [n]$ uniformly sample $\zeta_j \in_R \mathbb{Z}_q$ subject to $\sum_{j \in [n]} \zeta_j = 0 \bmod q$, store $(\text{sid}, (\zeta_1, \dots, \zeta_n))$, and send $(\text{mask}, \text{sid}, \zeta_i)$ to party P_i . If a tuple $(\text{sid}, (\zeta_1, \dots, \zeta_n))$ has been stored then send $(\text{mask}, \text{sid}, \zeta_i)$ to party P_i .
- $\mathcal{F}_{\text{RVOLE}[q, \ell]}$. The Random Vector Oblivious Linear Evaluation (RVOLE) functionality, parameterized by a prime $q \in \mathbb{N}$ and an integer $\ell \in \mathbb{N}$, is defined as follows for any two parties P_i and P_j playing the roles of Alice and Bob, respectively:
 - Upon receiving $(\text{Alice-sample}, \text{sid}, i, j)$ from party P_i , where $i, j \in [n]$ and $\text{sid} = \text{sid}' || \text{pid}_i || \text{pid}_j$ for the respective party identifiers pid_i and pid_j of P_i and P_j , if a tuple $(\text{ready}, \text{sid}, i, j, \cdot)$ has not been stored then sample $a_{\text{in}} \in_R \mathbb{Z}_q$ if P_i is honest or receive $(\text{Alice-sample}, \text{sid}, i, j, a_{\text{in}})$ from the adversary $\mathcal{S}$ if P_i is corrupted. Then, store $(\text{ready}, \text{sid}, i, j, a_{\text{in}})$, send $(\text{Alice-sample}, \text{sid}, i, j, a_{\text{in}})$ to P_i , and send $(\text{ready}, \text{sid}, i, j)$ to P_j .
 - Upon receiving $(\text{Bob-multiply}, \text{sid}, i, j, \mathbf{b}_{\text{in}})$ from party P_j , where $i, j \in [n]$, $\text{sid} = \text{sid}' || \text{pid}_i || \text{pid}_j$ for the respective party identifiers pid_i and pid_j of P_i and P_j , and $\mathbf{b}_{\text{in}} \in \mathbb{Z}_q^\ell$, if a tuple $(\text{ready}, \text{sid}, i, j, a_{\text{in}})$ has been stored and a tuple $(\text{completed}, \text{sid}, i, j)$ has not been stored, then:

- * If P_i and P_j are honest then sample $\mathbf{a}_{\text{out}} \in_R \mathbb{Z}_q^\ell$ and set $\mathbf{b}_{\text{out}} = a_{\text{in}} \cdot \mathbf{b}_{\text{in}} - \mathbf{a}_{\text{out}} \bmod q$.
- * If P_i is corrupted and P_j is honest then receive (Alice-share, sid, $i, j, \mathbf{a}_{\text{out}}$) from the adversary $\mathcal{S}$, where $\mathbf{a}_{\text{out}} \in \mathbb{Z}_q^\ell$, and set $\mathbf{b}_{\text{out}} = a_{\text{in}} \cdot \mathbf{b}_{\text{in}} - \mathbf{a}_{\text{out}} \bmod q$.
- * If P_i is honest and P_j is corrupted then receive (Bob-share, sid, $i, j, \mathbf{b}_{\text{out}}$) from the adversary $\mathcal{S}$, where $\mathbf{b}_{\text{out}} \in \mathbb{Z}_q^\ell$, and set $\mathbf{a}_{\text{out}} = a_{\text{in}} \cdot \mathbf{b}_{\text{in}} - \mathbf{b}_{\text{out}} \bmod q$.
- * Send (Alice-share, sid, $i, j, \mathbf{a}_{\text{out}}$) to P_i and (Bob-share, sid, $i, j, \mathbf{b}_{\text{out}}$) to P_j .
- * Store (completed, sid, i, j).

3 Overview of the DKLS23 Protocol and Our Refinement

In this section we first provide a high-level overview of the original DKLS23 protocol to establish the necessary context. We then detail our specific modifications, followed by the formal description of our refined, statistically-secure protocol.

Overview of the original protocol. At its core, the DKLS23 protocol leverages a clever rewriting of the ECDSA signing equation to securely compute the signature $s = k^{-1}(\mathbf{H}(m) + r \cdot x) \bmod q$ in three rounds using standard cryptographic functionalities (specifically, commitments, zero-sharing, and RVOLE). The original protocol flow proceeds as follows:

- **Round 1 (commitment & setup):** Each party P_i samples a local nonce k_i and commits to the corresponding group element $R_i = k_i \cdot G$. Simultaneously, the parties initiate the RVOLE and zero-sharing functionalities.
- **Round 2 (multiplication & decommitment):** The parties utilize the zero-sharing output to transform their secret key shares x_i to masked key shares $\hat{x}_i$. They then use the RVOLE functionality to compute multiplicative cross-terms between their local nonces and masked key shares $(k_i, \hat{x}_i)$ and the other parties' masked values ϕ_i obtained from the initiation of the RVOLE functionality. Finally, they decommit to R_i and broadcast their masked public key shares.
- **Round 3 (aggregation & share generation):** After validating the consistency of the public key shares and RVOLE outputs, each party aggregates the decommitted values to compute $R = \sum_{j \in [n]} R_j$ and its x -coordinate r . Utilizing the cross-terms generated in the previous round, each party locally computes additive signature shares u_i and w_i , which are then broadcast to all other parties.
- **Output (signature reconstruction):** The parties aggregate the broadcasted shares to compute $s = (\sum_{j \in [n]} w_j) \cdot (\sum_{j \in [n]} u_j)^{-1} \bmod q$, and conclude by verifying the validity of the signature (r, s) .

Our modifications. While the DKLS23 protocol is highly efficient, our analysis in Section 4 reveals vulnerabilities in its original specification. To ensure statistical UC security and improve the protocol's modularity, our refinement introduces the following specific modifications:

- **Nonce-consistency check.** As we will demonstrate in Section 4, the original protocol is vulnerable to a split-view attack since it lacks a mechanism to enforce a globally consistent view of the nonce $R = k \cdot G$ used to generate the signature. To mitigate this, we include a consistency check for verifying that all (honest) parties hold the same group element R . Specifically, in Step 3d of the refined protocol, each party additionally sends their locally-computed R to all other parties as part of its third-round message, and then verifies in Step 4a that they receive the identical R from all other parties (and aborts otherwise).
- **Relaxed zero sharing.** We replace the original protocol’s zero-sharing functionality $\mathcal{F}_{\text{Zero}[q,n]}$ (defined in Section 2) with a relaxed variant $\mathcal{F}_{\text{rZero}[q,n]}$ (defined below following the description of the protocol) that admits a highly efficient, setup-free realization that avoids the need for pre-shared seeds or long-term state.
- **Sign-consistency check.** ECDSA verification allows for signature malleability, where both (r, s) and $(r, -s)$ are valid signatures with respect to the same randomness R (as they correspond to the points R and $-R$, which share the same x -coordinate r). As we additionally demonstrate in Section 4, even with the nonce-consistency check in place, a computationally-unbounded adversary can still lead some honest parties to output (r, s) and others to output $(r, -s)$ (a scenario which cannot be simulated in the ideal model). To prevent this, we add a verification in Step 4c, hereafter referred to as the *sign-consistency check*, ensuring that the computed s strictly reconstructs the agreed-upon point R (specifically, each party verifies that $R = (s^{-1} \cdot H(m)) \cdot G + (s^{-1} \cdot r) \cdot Q$). We note that this is a consistency issue which can alternatively be resolved by enforcing the “low- s ” rule [Wui14] (i.e., setting $s \leftarrow \min\{s, q-s\}$) which is common in practice. We additionally note that, having added the sign-consistency check in Step 4c, the original protocol’s explicit signature verification in Step 4e is no longer needed (we nevertheless include the explicit signature verification for providing a unified exposition of both versions of the protocol).

Following the original protocol, our refined protocol (denoted Π_{ECDSA}) additionally relies on the ideal functionalities $\mathcal{F}_{\text{Com}}$ and $\mathcal{F}_{\text{RVOLE}[q,2]}$ described in Section 2. To provide a unified exposition, we present the full protocol flow of the refined version below, with our modifications to the original protocol colored red for convenience.

The Refined Protocol Π_{ECDSA}

- **Public parameters:**
 - Cyclic group $\mathbb{G}$ of prime order q that is generated by $G \in \mathbb{G}$
 - Number of parties $n \geq 2$
 - ECDSA hash function H
- **Ideal functionalities:** $\mathcal{F}_{\text{Com}}$, $\mathcal{F}_{\text{rZero}[q,n]}$, $\mathcal{F}_{\text{RVOLE}[q,2]}$.
- **Parties:** $P_1, \dots, P_n$ with distinct party identifiers $\text{pid}_1, \dots, \text{pid}_n$.
- **Inputs:**

– For every $i \in [n]$ party P_i holds (sid, m, Q, x_i) , where $Q = \sum_{j \in [n]} x_j \cdot G$.

• **Execution:**

1. **First round for every party P_i where $i \in [n]$:**

- (a) Sample $k_i, \phi_i \in_R \mathbb{Z}_q$
- (b) Compute $R_i = k_i \cdot G$
- (c) For every $j \in [n] \setminus \{i\}$:
 - i. Send $(\text{commit}, \text{sid} || \text{pid}_i || \text{pid}_j, i, j, R_i)$ to $\mathcal{F}_{\text{Com}}$
 - ii. Send $(\text{Alice-sample}, \text{sid} || \text{pid}_i || \text{pid}_j, i, j)$ to $\mathcal{F}_{\text{RVOLE}[q,2]}$
- (d) Send $(\text{sample}, \text{sid} || \text{pid}_1 || \dots || \text{pid}_n)$ to $\mathcal{F}_{\text{Zero}[q,n]}$

2. **Second round for every party P_i where $i \in [n]$:**

Upon receiving:

- $(\text{receipt}, \text{sid} || \text{pid}_j || \text{pid}_i, j, i)$ from $\mathcal{F}_{\text{Com}}$ for every $j \in [n] \setminus \{i\}$
- $(\text{ready}, \text{sid} || \text{pid}_j || \text{pid}_i, j, i)$ from $\mathcal{F}_{\text{RVOLE}[q,2]}$ for every $j \in [n] \setminus \{i\}$
- $(\text{Alice-sample}, \text{sid} || \text{pid}_i || \text{pid}_j, i, j, \chi_{i,j})$ from $\mathcal{F}_{\text{RVOLE}[q,2]}$ for every $j \in [n] \setminus \{i\}$
- $(\text{mask}, \text{sid} || \text{pid}_1 || \dots || \text{pid}_n, \zeta_i)$ from $\mathcal{F}_{\text{Zero}[q,n]}$

proceed as follows:

- (a) Compute

$$\hat{x}_i = x_i + \zeta_i \bmod q$$

$$\hat{Q}_i = \hat{x}_i \cdot G$$

- (b) For every $j \in [n] \setminus \{i\}$:

- i. Send $(\text{Bob-multiply}, \text{sid} || \text{pid}_j || \text{pid}_i, j, i, (k_i, \hat{x}_i))$ to $\mathcal{F}_{\text{RVOLE}[q,2]}$.
- ii. Receive $(\text{Bob-share}, \text{sid} || \text{pid}_j || \text{pid}_i, j, i, (c_{i,j}^u, c_{i,j}^v))$ from $\mathcal{F}_{\text{RVOLE}[q,2]}$.
- iii. Compute

$$C_{i,j}^u = c_{i,j}^u \cdot G$$

$$C_{i,j}^v = c_{i,j}^v \cdot G$$

$$\psi_{i,j} = \phi_i - \chi_{i,j} \bmod q$$

- iv. Send $(\text{decommit}, \text{sid} || \text{pid}_i || \text{pid}_j, i, j, R_i)$ to $\mathcal{F}_{\text{Com}}$.

- v. Send $(\text{sid} || \text{pid}_i || \text{pid}_j, C_{i,j}^u, C_{i,j}^v, \psi_{i,j}, \hat{Q}_i)$ to party P_j .

3. **Third round for every party P_i where $i \in [n]$:**

Upon receiving:

- $(\text{decommit}, \text{sid} || \text{pid}_j || \text{pid}_i, j, i, R_j)$ from $\mathcal{F}_{\text{Com}}$ for every $j \in [n] \setminus \{i\}$
- $(\text{Alice-share}, \text{sid} || \text{pid}_i || \text{pid}_j, i, j, (d_{i,j}^u, d_{i,j}^v))$ from $\mathcal{F}_{\text{RVOLE}[q,2]}$ for every $j \in [n] \setminus \{i\}$
- $(\text{sid} || \text{pid}_j || \text{pid}_i, C_{j,i}^u, C_{j,i}^v, \psi_{j,i}, \hat{Q}_j)$ from party P_j for every $j \in [n] \setminus \{i\}$

proceed as follows:

- (a) Verify that $\sum_{j \in [n]} \hat{Q}_j = Q$ and abort otherwise.
- (b) For every $j \in [n] \setminus \{i\}$ verify that

$$\chi_{i,j} \cdot R_j - C_{j,i}^u = d_{i,j}^u \cdot G$$

$$\chi_{i,j} \cdot \hat{Q}_j - C_{j,i}^v = d_{i,j}^v \cdot G$$

and abort otherwise.

(c) Compute

$$\begin{aligned}
 R &= \sum_{j \in [n]} R_j \\
 u_i &= k_i \cdot \left(\phi_i + \sum_{j \in [n] \setminus \{i\}} \psi_{j,i} \right) + \sum_{j \in [n] \setminus \{i\}} (c_{i,j}^u + d_{i,j}^u) \bmod q \\
 v_i &= \hat{x}_i \cdot \left(\phi_i + \sum_{j \in [n] \setminus \{i\}} \psi_{j,i} \right) + \sum_{j \in [n] \setminus \{i\}} (c_{i,j}^v + d_{i,j}^v) \bmod q \\
 w_i &= H(m) \cdot \phi_i + r \cdot v_i \bmod q
 \end{aligned}$$

where $r \in \mathbb{Z}_q$ is the x -coordinate (modulo q) of the elliptic-curve point $R \in \mathbb{G}$.

(d) For every $j \in [n] \setminus \{i\}$ send $(\text{sid} || \text{pid}_i || \text{pid}_j, R, w_i, u_i)$ to party P_j .

4. **Output for every party P_i where $i \in [n]$:**

Upon receiving $(\text{sid} || \text{pid}_j || \text{pid}_i, \tilde{R}_j, w_j, u_j)$ from party P_j for every $j \in [n] \setminus \{i\}$, proceed as follows:

- (a) **If $\tilde{R}_j \neq R$ for some $j \in [n] \setminus \{i\}$ then abort.**
(This is the nonce-consistency check)
- (b) Compute $s = \left(\sum_{j \in [n]} w_j \right) \cdot \left(\sum_{j \in [n]} u_j \right)^{-1} \bmod q$.
- (c) **If $R \neq (s^{-1} \cdot H(m)) \cdot G + (s^{-1} \cdot r) \cdot Q$ then abort.**
(This is the sign-consistency check which, as discussed above, can be replaced with the “low-s” rule)
- (d) Let $\sigma = (r, s)$.
- (e) If σ is not a valid ECDSA signature on the message m with respect to the public key Q and hash function H then abort.
(As discuss above, having added the sign-consistency check in Step 4c, this step is no longer needed and we keep it only for providing a unified exposition of both versions of the protocol.)
- (f) Output σ .

The relaxed zero-sharing functionality. Doerner et al. [DKLs23, DKLs24] note that the zero-sharing functionality $\mathcal{F}_{\text{Zero}[q,n]}$ on which their protocol relies can be instantiated using pre-shared seeds. First, during a setup phase, each pair of parties P_i and P_j (where $i < j$) agrees on a random secret seed $s_{i,j}$. Then, given a unique session identifier sid , each party P_i locally computes:

$$\zeta_i = \sum_{i < j} H(s_{i,j}, \text{sid}) - \sum_{j < i} H(s_{j,i}, \text{sid}) \bmod q,$$

where H is a hash function modeled as a random oracle (outputting at least $\log_2 q + \kappa_s$ bits). We note that a similar technique for generating zero-shares from pre-shared seeds for a pseudorandom function was previously utilized by Araki et al. [AFL⁺16] in the context of high-throughput semi-honest secure computation.

This instantiation, however, requires both a dedicated setup phase to establish the pre-shared seeds, and long-term secure pairwise storage for those seeds.

For avoiding both setup and long-term secure pairwise storage, our security proof in Section 5 shows that the following relaxed variant of the zero-sharing functionality, denoted $\mathcal{F}_{\text{rZero}[q,n]}$, in which the adversary may determine the sum of shares of the corrupted parties, suffices for the security of the protocol. For describing the relaxed functionality, we denote by $\mathcal{C}$ and $\mathcal{H}$ the subsets of corrupted and honest parties, respectively. Parameterized with a prime $q \geq 2$ and a number $n \geq 2$ of parties, the functionality $\mathcal{F}_{\text{rZero}[q,n]}$ is defined as follows:

- Upon receiving (sample, sid) from a party P_i , where $i \in [n]$ and $\text{sid} = \text{sid}' || \text{pid}_1 || \dots || \text{pid}_n$ for the respective party identifiers $\text{pid}_1, \dots, \text{pid}_n$ of $P_1, \dots, P_n$, proceed as follows:
 1. If a tuple $(\text{sid}, (\zeta_1, \dots, \zeta_n))$ has not been stored, then:
 - Send (choose, sid) to the adversary $\mathcal{S}$ to obtain $T \in \mathbb{Z}_q$.
(The adversary determines the sum T of shares of the corrupted parties)
 - Sample $\{\zeta_c\}_{c \in \mathcal{C}}$ uniformly and independently from $\mathbb{Z}_q$ subject to satisfying $\sum_{c \in \mathcal{C}} \zeta_c = T \bmod q$.
 - Sample $\{\zeta_h\}_{h \in \mathcal{H}}$ uniformly and independently from $\mathbb{Z}_q$ subject to satisfying $\sum_{h \in \mathcal{H}} \zeta_h = -T \bmod q$.
 - Send (mask, sid, ζ_i) to P_i .
 - Store $(\text{sid}, (\zeta_1, \dots, \zeta_n))$.
 2. If a tuple $(\text{sid}, (\zeta_1, \dots, \zeta_n))$ has already been stored then send (mask, sid, ζ_i) to P_i .

We prove that the following simple and highly efficient one-round protocol $\Pi_{\text{rZero}[q,n]}$ UC-realizes the functionality $\mathcal{F}_{\text{rZero}[q,n]}$:

- Each party P_i , upon receiving (sample, sid), uniformly samples $r_{i,j} \in_R \mathbb{Z}_q$ for every $j \in [n]$ such that $i < j$, and sends it to party P_j .
- Each party P_i , upon receiving $r_{j,i}$ for every $j \in [n]$ such that $j < i$, outputs $\zeta_i = \sum_{i < j} r_{i,j} - \sum_{j < i} r_{j,i} \bmod q$.

In every execution of the signing protocol, the parties can execute $\Pi_{\text{rZero}[q,n]}$ (i.e., send their message in the protocol $\Pi_{\text{rZero}[q,n]}$) as part of the first round since the zero shares $\zeta_1, \dots, \zeta_n$ are not needed until the second round. This avoids a dedicated setup, additional long-term secure storage, or additional rounds of communication. We prove the following theorem in Appendix A:

Theorem 3.1. *For any prime $q \geq 2$ and integer $n \geq 2$, protocol $\Pi_{\text{rZero}[q,n]}$ statistically UC-realizes the functionality $\mathcal{F}_{\text{rZero}[q,n]}$ against a malicious adversary that statically corrupts up to $n - 1$ parties.*

4 Statistical Insecurity of the Original Protocol

In this section we show that the original DKLS23 protocol does not achieve statistical UC security. Specifically, we construct a computationally-unbounded “split-view” adversary $\mathcal{A}$ that successfully attacks the original protocol by causing two or more honest parties to output different valid signatures on the same

message (specifically, signatures (r, s) and (r', s') with $r \neq r'$). Considering any ideal ECDSA signing functionality that outputs a single signature per session, such an attack cannot be simulated in the ideal model. Our attack relies solely on the adversary's ability to compute discrete logarithms (all other operations performed by the adversary are polynomial time).

We consider the case of $n \geq 3$ parties, where the adversary $\mathcal{A}$ statically corrupts party P_1 and there are at least two honest parties. The adversary is provided with P_1 's input (sid, m, Q, x_1) and is defined as follows:

- **Rounds 1 and 2:** The adversary $\mathcal{A}$ follows the protocol honestly, but instead of sampling a single $k_1 \in_R \mathbb{Z}_q$, for every $j \in \{2, \dots, n\}$ it uniformly samples $k_{1,j} \in_R \mathbb{Z}_q$ and commits to $R_{1,j} = k_{1,j} \cdot G$ to party P_j . As a result, each honest party P_j computes $\tilde{R}_j = R_{1,j} + \sum_{i=2}^n R_i$, and we let r_j denote the x -coordinate (modulo q) of $\tilde{R}_j$. Note that, since the $k_{1,j}$'s are sampled independently and uniformly, then $r_i \neq r_j$ for every $i \neq j \in \{2, \dots, n\}$ with all but a negligible probability.
- **Round 3:** The adversary $\mathcal{A}$ first receives the messages $\{(w_i, u_i)\}_{i \in \{2, \dots, n\}}$ from the honest parties. Then, $\mathcal{A}$ computes the discrete logarithms of Q , $R_2, \dots, R_n$, denoted x , and $k_2, \dots, k_n$, respectively. Finally, for every $j \in \{2, \dots, n\}$ the adversary $\mathcal{A}$ computes

$$u_{1,j} = k_{1,j} + \sum_{i=2}^n (k_i - u_i) \bmod q$$

$$w_{1,j} = (\mathbf{H}(m) + r_j \cdot x) - \sum_{i=2}^n w_i \bmod q ,$$

and sends $(w_{1,j}, u_{1,j})$ to party P_j .

- **Outcome:** Each honest party P_j , where $j \in \{2, \dots, n\}$, computes

$$s_j = \left(w_{1,j} + \sum_{i=2}^n w_i \right) \cdot \left(u_{1,j} + \sum_{i=2}^n u_i \right)^{-1} \bmod q$$

$$= (\mathbf{H}(m) + r_j \cdot x) \cdot \left(k_{1,j} + \sum_{i=2}^n k_i \right)^{-1} \bmod q$$

and lets $\sigma_j = (r_j, s_j)$. Noting that $(k_{1,j} + \sum_{i=2}^n k_i) \cdot G = \tilde{R}_j$, and recalling that r_j is the x -coordinate (modulo q) of $\tilde{R}_j$, it holds that σ_j is a valid signature on the message m and therefore party P_j outputs σ_j . Given that $r_i \neq r_j$ for every $i \neq j \in \{2, \dots, n\}$ with all but a negligible probability, the honest parties thus output different valid signatures with all but a negligible probability.

We note that the above attack is presented while decoupling the signing protocol from the key generation phase. However, we emphasize that the attack is not an artifact of our decoupling approach, but rather relies on a fundamental

property of the DKLs23 protocol even when deployed exactly as specified by Doerner et al. with their native key generation functionality $\mathcal{F}_{\text{RelaxedKeyGen}}$. In this context, an adversary would execute the above attack in two phases:

- The adversary participates honestly in the key-generation protocol implementing $\mathcal{F}_{\text{RelaxedKeyGen}}$, and obtains the global public key Q and their own secret key share x_1 .
- Equipped with Q and x_1 , the adversary executes the above-described attack.

Thus, the specific properties of the key-generation functionality $\mathcal{F}_{\text{RelaxedKeyGen}}$ do not prevent the attack, and their protocol does not provide statistical UC security in the $(\mathcal{F}_{\text{RelaxedKeyGen}}, \mathcal{F}_{\text{Com}}, \mathcal{F}_{\text{Zero}[q,n]}, \mathcal{F}_{\text{RVOLE}[q,2]})$ -hybrid model.

The necessity of the signature sign-consistency check. As discussed in Section 3, we additionally identify a second attack that exploits the known $\pm s$ malleability of ECDSA signatures. This attack applies even when the protocol *does* include the nonce-consistency check (Step 4a), but lacks the sign-consistency check (Step 4c). Specifically, we show that an adversary $\mathcal{A}$ (which can compute discrete logarithms) can cause two honest parties, P_j and $P_{j'}$, to output valid signatures of the form $\sigma_j = (r, s)$ and $\sigma_{j'} = (r, -s)$, respectively, violating statistical security. For simplicity, we again consider the case of $n \geq 3$ parties, where the adversary statically corrupts P_1 .

In the first and second round, the adversary follows the instructions of the protocol. Then, in the third round, the adversary receives $\{(u_i, w_i)\}_{i \in \{2, \dots, n\}}$ from all honest parties, and honestly computes (u_1, w_1) . For party P_j , the adversary sends (u_1, w_1) , which leads P_j to compute the canonical $s = k^{-1} \cdot (\text{H}(m) + r \cdot x) \bmod q$ in Step 4b of the protocol. However, for party $P_{j'}$, the adversary uses the discrete logarithms $x \in \mathbb{Z}_q$ and $k \in \mathbb{Z}_q$ of Q and R , respectively, to compute (u'_1, w'_1) that lead $P_{j'}$ to compute $-s$ in Step 4b of the protocol. Specifically, the adversary computes $\phi \in \mathbb{Z}_q$ satisfying $\sum_{i \in [n]} u_i = \phi \cdot k \bmod q$ and $\sum_{i \in [n]} w_i = \phi \cdot (\text{H}(m) + r \cdot x) \bmod q$, and defines $u'_1 = -\phi \cdot k - \sum_{i=2}^n u_i \bmod q$ and $w'_1 = w_1$. Party $P_{j'}$ then verifies the signature $(r, -s)$ using standard ECDSA verification. Since $(r, -s)$ corresponds to the point $-R$, and the x -coordinate of $-R$ is r , the standard verification passes. Without an explicit check that the reconstructed point equals R (or a “low- s ” enforcement), both parties once again output valid, yet different, signatures.

5 Statistical Security of the Refined Protocol

In this section we present an ECDSA signing functionality $\mathcal{F}_{\text{ECDSA}}$ that is decoupled from key generation, and prove that the refined protocol Π_{ECDSA} securely realizes it with statistical security. Our definition of the functionality $\mathcal{F}_{\text{ECDSA}}$ and our security analysis assume that the simulator is provided with the corrupted parties’ secret key shares at the onset of the signing phase. This abstraction cleanly decouples the signing protocol from the key generation phase, aligning with standard distributed key generation protocols that explicitly extract

these shares. However, the original DKLS23 protocol utilizes a “relaxed” key-generation functionality $\mathcal{F}_{\text{RelaxedKeyGen}}$ that avoids zero-knowledge proofs by not extracting the adversarial key shares during the setup phase. To demonstrate that our refined signing protocol remains fully secure and compatible with the original DKLS23 architecture, we additionally provide a supplementary security analysis in Appendix C. There, we prove the statistical UC security of the refined protocol when natively composed with the specific functionalities defined by Doerner et al. [DKLS23, DKLS24].

Together, these two analyses provide comprehensive coverage: Here we establish security under the modular framework where key generation explicitly extracts all parties’ secret shares (as is the case in most distributed key generation protocols), while in Appendix C we establish security under the specific relaxed framework of Doerner et al. where such extraction is intentionally avoided. In the latter setting, the individual public key shares $Q_1, \dots, Q_n$ are naturally removed from the parties’ inputs, as they are not produced by the relaxed key-generation functionality.

We now proceed to formally define our decoupled signing functionality denoted $\mathcal{F}_{\text{ECDSA}}$. The functionality, presented below, is highly standard: it securely computes an ECDSA signature upon receiving valid and consistent inputs (as defined by Lindell [Lin24] and discussed below).

The Functionality $\mathcal{F}_{\text{ECDSA}}$

- **Public parameters:**
 - Cyclic group $\mathbb{G}$ of prime order q that is generated by $G \in \mathbb{G}$
 - Number of parties $n \geq 2$
 - ECDSA hash function H
- **Parties:** $P_1, \dots, P_n$ with distinct party identifiers $\text{pid}_1, \dots, \text{pid}_n$.
- **Computation:**
 1. Upon receiving $(\text{sign}, \text{sid}, m, Q, Q_1, \dots, Q_n, x_i)$ from party P_i for every $i \in [n]$, if all parties sent the same $(\text{sid}, m, Q, Q_1, \dots, Q_n)$ and it holds that $Q = \sum_{i \in [n]} Q_i$ and $Q_i = x_i \cdot G$ for every $i \in [n]$ then continue to the next step and otherwise ignore.
 2. Sample $k \in_R \mathbb{Z}_q$, compute $R \leftarrow k \cdot G$, and let r denote the x -coordinate of R modulo q .
 3. Compute

$$\begin{aligned} x &\leftarrow \sum_{i \in [n]} x_i \bmod q \\ s &\leftarrow k^{-1} \cdot (H(m) + x \cdot r) \bmod q \\ \sigma &\leftarrow (r, s) \end{aligned}$$

4. Send (sid, σ) to all parties.

Our definition of the functionality $\mathcal{F}_{\text{ECDSA}}$ and our security analysis assume that the parties’ inputs include the individual public key shares $Q_1, \dots, Q_n$. This aligns with many distributed key generation protocols where parties publicly verify their shares. However, we emphasize that neither the refined pro-

protocol described in Section 3 nor its security analysis in this section fundamentally relies on each party P_i knowing the other parties' individual public shares $\{Q_j\}_{j \in [n] \setminus \{i\}}$ (indeed, in Appendix C, where we prove security under the relaxed key-generation functionality of Doerner et al., the individual public key shares are removed from the parties' inputs entirely). That is, our results hold even in settings where the underlying key generation provides each party P_i with only the global public key Q and their own secret share x_i . In such cases, the ideal functionality (and, similarly, Definition 5.1) would simply verify that $Q = \sum_{i \in [n]} x_i \cdot G$ without checking that $Q_i = x_i \cdot G$ for every $i \in [n]$.

We analyze the security of the protocol in the real-vs-ideal framework for static corruptions by malicious adversaries (without rewinding thus implying UC security as discussed below via [KLR10]). Following Lindell [Lin24], we analyze the security of the protocol assuming that the parties hold inputs that are consistent with a valid key generation phase. We formally capture this requirement via Lindell's notion of *valid and consistent inputs*. This definition ensures that the individual secret key shares x_i held by the parties correspond to their respective public key shares Q_i (i.e., $Q_i = x_i \cdot G$), and that all parties agree on the same global public key Q which is the sum of these individual shares. This approach allows us to prove the security of the signing protocol independently of the specific method used to generate the keys (e.g., whether by a trusted dealer or a distributed key generation protocol), provided that the key generation phase yields a valid output.

Definition 5.1 ([Lin24]). *We say that $P_1, \dots, P_n$ hold valid and consistent inputs if for every $i \in [n]$ party P_i holds $(sid, m, Q, Q_1, \dots, Q_n, x_i)$ such that:*

1. *All parties hold the same $(sid, m, Q, Q_1, \dots, Q_n)$.*
2. *$Q = \sum_{i \in [n]} Q_i$.*
3. *$Q_i = x_i \cdot G$ for every $i \in [n]$.*

We prove the following theorem:

Theorem 5.2. *Let $n \geq 2$ and assume that the parties hold valid and consistent inputs. Then, protocol Π_{ECDSA} statistically realizes the functionality $\mathcal{F}_{ECDSA}$ with abort in the $(\mathcal{F}_{Com}, \mathcal{F}_{Zero[q,n]}, \mathcal{F}_{RVOLE[q,2]})$ -hybrid model in the presence of a malicious adversary statically corrupting up to $n - 1$ parties.*

Proof overview. The formal proof of Theorem 5.2 is provided in Appendix B. Here we provide a high-level overview of the simulation strategy and the sequence of hybrid experiments. The proof proceeds by constructing an ideal-model simulator $\mathcal{S}$ that extracts the corrupted parties' effective inputs, queries the ideal functionality $\mathcal{F}_{ECDSA}$ to obtain a signature $\sigma = (r, s)$, and embeds this signature into the simulated protocol transcript. The indistinguishability of the real and ideal executions is established via a sequence of eleven hybrid experiments $(\mathcal{H}_1 \text{ to } \mathcal{H}_{11})$ implementing the following strategy:

- **Extracting effective inputs and replacing consistency checks.** The simulator extracts the adversary's effective nonce contributions and masked key shares from their inputs to the $\mathcal{F}_{\text{RVOLE}[q,2]}$ functionality. We show that the protocol's lightweight consistency checks can be replaced by direct checks on these extracted discrete logarithms. Since the RVOLE masks are uniformly random and information-theoretically hidden from the adversary, any attempt to pass the consistency checks with inconsistent inputs succeeds with probability at most $1/q$. We show that this step introduces a negligible statistical distance of at most $n^2/(2q)$.
- **Embedding the ideal signature.** The simulator designates an honest party P_{h^*} to absorb the adjustments required to force the protocol's output to match the ideal signature. Using the extracted adversarial inputs and the global target values R (derived from r, s , and m) and Q , the simulator back-computes P_{h^*} 's messages (e.g., R_{h^*} and w_{h^*}). We demonstrate that this back-computation perfectly preserves the marginal distributions of all simulated messages since the underlying values being replaced are uniformly distributed.
- **Simulating honest-honest interactions.** For the remaining honest parties $h \in \mathcal{H} \setminus \{h^*\}$, the simulator samples their signature shares u_h and v_h uniformly at random. We prove this is perfectly indistinguishable from the real protocol execution since the $\mathcal{F}_{\text{RVOLE}[q,2]}$ outputs corresponding to interactions between pairs of honest parties act as perfect one-time pads that perfectly mask the underlying values from the adversary.
- **Finalizing the output.** Finally, we show that the algebraic constraints of the protocol, combined with our introduced nonce-consistency check, ensure that whenever the honest parties do not abort, they all necessarily compute the exact same valid signature (r, s) provided by the ideal functionality. This is the crucial juncture where our analysis fundamentally deviates from the original DKLS23 proof. Technically, the simulator embeds the target signature into the view of a single honest party P_{h^*} , which guarantees that $\tilde{R}_{h^*} = R$. The nonce-consistency check explicitly forces all other honest parties to agree on this exact same curve point (i.e., verifying that $\tilde{R}_h = \tilde{R}_{h^*}$ for all $h \in \mathcal{H}$). This guarantees that every honest party uses the exact same x -coordinate $r_h = r$. Since ECDSA signatures are uniquely determined given the message, public key, and nonce (up to the $\pm s$ malleability, which is strictly handled by our introduced sign-consistency check), this agreement strictly binds all honest parties to the exact signature (r, s) provided by the functionality. This mechanism completely neutralizes the split-view attack presented in Section 4, as the adversary can no longer successfully supply inconsistent nonce shares to different honest parties without triggering an immediate abort.

UC security. Our proof of Theorem 5.2 provides an ideal-model adversary $\mathcal{S}$ that simulates the view of any adversary $\mathcal{A}$ in the $(\mathcal{F}_{\text{Com}}, \mathcal{F}_{\text{rZero}[q,n]}, \mathcal{F}_{\text{RVOLE}[q,2]})$ -hybrid model without rewinding it. Therefore, when the ideal functionalities $\mathcal{F}_{\text{Com}}$, $\mathcal{F}_{\text{rZero}[q,n]}$ and $\mathcal{F}_{\text{RVOLE}[q,2]}$ are instantiated via UC-secure implementations, and the inputs of all parties to each execution are fixed before the executions begins, then the protocol is UC-secure as proven by Kushilevitz, Lindell and

Rabin [KLR10]. Note that Theorem 5.2 assumes valid and consistent inputs, and therefore fixing the input to some party fixes the inputs to all other parties. Thus, in our setting, the inputs of all parties to each execution are indeed fixed before the execution begins.

6 Computational Security of the Original Protocol

The existence of the split-view attack described in Section 4 demonstrates that the original DKLS23 protocol is not statistically secure. In this section, we show that the original protocol is nevertheless computationally secure based on the assumption that the ECDSA signature scheme is *strongly unforgeable up to sign* (i.e., up to the trivial $(r, \pm s)$ -malleability of ECDSA signatures), as defined by Groth and Shoup [GS22b]. This assumption guarantees, in particular, that given a uniformly distributed ECDSA public key $Q = x \cdot G$ and oracle access to a signing oracle, it is computationally infeasible to produce an additional valid signature $\sigma' = (r', s')$ on a previously signed message m such that r' is distinct from the nonce of the signature provided by the oracle.

The shift to the monolithic setting. Our statistical analysis of the refined protocol in Section 5 utilized a decoupled approach, proving security for *any* arbitrary sequence of valid and consistent inputs. However, the strong unforgeability game for ECDSA requires the challenge public key Q to be uniformly distributed. Consequently, we cannot perform our computational reduction in the decoupled setting. Instead, we analyze the protocol in the monolithic setting of Doerner et al. [DKLS23, DKLS24] (as we consider in Appendix C), which explicitly guarantees that the global public key Q is uniformly distributed.

To formalize this, we rely on the monolithic refined protocol Π_{mECDSA} , defined in Appendix C, which explicitly combines the key-generation and signing phases into a single protocol that natively realizes the monolithic ECDSA functionality $\mathcal{F}_{\text{mECDSA}}$. We define Π_{mECDSA}^- to be the exact same protocol as Π_{mECDSA} , except that it omits the nonce-consistency check. Thus, Π_{mECDSA}^- faithfully represents the original DKLS23 protocol without our introduced refinement (but retaining our introduced sign-consistency check).

Intuition. To provide intuition for the reduction, we recall the mechanics of our decoupled proof in Section 5. Our proof of Theorem 5.2 relies on the nonce-consistency check solely to ensure that experiments $\mathcal{H}_9$ and $\mathcal{H}_{10}$ are identical. Specifically, whereas in both experiments it is guaranteed that the designated honest party P_{h^*} either outputs the target signature $\sigma = (r, s)$ or aborts, in experiment $\mathcal{H}_{10}$ this holds strictly for all other honest parties as well. In experiment $\mathcal{H}_9$, this global agreement is not guaranteed without the nonce-consistency check. In the absence of this check, the two experiments may be computationally distinguishable: an honest party P_h (where $h \neq h^*$) in $\mathcal{H}_9$ might output a valid signature $\sigma_h = (r_h, s_h)$ derived from their locally computed $\tilde{R}_h \neq R$. Since the adversary commits to the values $\{R_{c, h^*}\}_{c \in \mathcal{C}}$ and $\{R_{c, h}\}_{c \in \mathcal{C}}$ before the uniform and independent values R_{h^*} and R_h are determined, it holds that $\tilde{R}_h \neq \pm R$

with all but negligible probability. Thus, for the signature $\sigma_h = (r_h, s_h)$, it holds that $r_h \neq r$. An analogous divergence occurs in the monolithic hybrid sequence presented in Appendix C.

The reduction. We show that any probabilistic polynomial-time adversary $\mathcal{A}$ that successfully distinguishes the real execution from the ideal simulation can be transformed into a probabilistic polynomial-time adversary $\mathcal{B}$ that breaks the strong unforgeability up to sign of the ECDSA signature scheme. The adversary $\mathcal{B}$ receives a uniformly distributed ECDSA public key $Q = x \cdot G$ and oracle access to a signing oracle, and outputs a valid signature $\sigma' = (r', s')$ on a previously signed message m such that r' is distinct from the nonce of the signature provided by the oracle. To facilitate this reduction, $\mathcal{B}$ uses the monolithic setup simulation techniques established in Appendix C to embed the challenge public key Q as the global public key in the emulated key generation phase, obtaining the corrupted parties' public key shares $\{Q_c\}_{c \in \mathcal{C}}$. Crucially, because $\mathcal{B}$ is provided with chosen-message queries to the external ECDSA challenger, $\mathcal{B}$ never needs to extract the discrete logarithms of the corrupted parties' key shares to emulate the execution.

This enables us to prove the following theorem in Appendix D (see Appendix C for the formal description of the monolithic protocol Π_{mECDSA} from which the protocol Π_{mECDSA}^- is derived, and for the formal descriptions of the functionalities $\mathcal{F}_{\text{mECDSA}}$ and $\mathcal{F}_{\text{RelaxedKeyGen}}$):

Theorem 6.1. *For any cyclic group $\mathbb{G}$ of prime order q and for any integer $n \geq 2$, assuming that ECDSA is strongly unforgeable up to sign, protocol Π_{mECDSA}^- UC-realizes the functionality $\mathcal{F}_{\text{mECDSA}}$ in the $(\mathcal{F}_{\text{RelaxedKeyGen}[\mathcal{G}, n]}, \mathcal{F}_{\text{Com}}, \mathcal{F}_{\text{rZero}[q, n]}, \mathcal{F}_{\text{rVole}[q, 2]})$ -hybrid model against a malicious adversary that statically corrupts up to $n - 1$ parties.*

Scope of the computational guarantee. We remark that the computational security established by this reduction addresses only the absence of the nonce-consistency check, and not the absence of the sign-consistency check. In the absence of the sign-consistency check, a computationally-unbounded adversary can still cause some honest parties to output (r, s) and others to output $(r, -s)$, as demonstrated in Section 4. Since strong unforgeability up to sign explicitly permits $(r, -s)$ as a trivial variant of (r, s) , this discrepancy does not constitute a forgery and cannot be captured by the reduction. Consequently, the computational security guarantee for the original protocol holds with respect to a signing functionality that may output either (r, s) or $(r, -s)$ to different honest parties, but no other signature. In practice, this ambiguity is eliminated entirely by enforcing the standard “low- s ” rule (i.e., setting $s \leftarrow \min\{s, q - s\}$).

Acknowledgments

We thank Yehuda Lindell, Peihan Miao and Mike Rosulek for useful discussions throughout the various stages of this work.

References

- ABC⁺24. Michael Adjudj, Constantin Blokh, Geoffroy Couteau, Arik Galansky, Antoine Joux, and Nikolaos Makriyannis. Two-round 2PC ECDSA at the cost of 1 OLE. Cryptology ePrint Archive, Report 2024/1950, 2024. URL: <https://eprint.iacr.org/2024/1950>.
- AFL⁺16. Toshinori Araki, Jun Furukawa, Yehuda Lindell, Ariel Nof, and Kazuma Ohara. High-throughput semi-honest secure three-party computation with an honest majority. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016: 23rd Conference on Computer and Communications Security*, pages 805–817. ACM Press, October 2016. doi:10.1145/2976749.2978331.
- ANO⁺22. Damiano Abram, Ariel Nof, Claudio Orlandi, Peter Scholl, and Omer Shlomovits. Low-bandwidth threshold ECDSA via pseudorandom correlation generators. In *2022 IEEE Symposium on Security and Privacy*, pages 2554–2572. IEEE Computer Society Press, May 2022. doi:10.1109/SP46214.2022.9833559.
- BDLR25a. Renas Bacho, Sourav Das, Julian Loss, and Ling Ren. Adaptively secure three-round threshold Schnorr signatures from DDH. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025, Part VI*, volume 16005 of *Lecture Notes in Computer Science*, pages 390–422. Springer, Cham, August 2025. doi:10.1007/978-3-032-01887-8_13.
- BDLR25b. Renas Bacho, Sourav Das, Julian Loss, and Ling Ren. Glacius: Threshold schnorr signatures from DDH with full adaptive security. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EURO-CRYPT 2025, Part II*, volume 15602 of *Lecture Notes in Computer Science*, pages 304–334. Springer, Cham, May 2025. doi:10.1007/978-3-031-91124-8_11.
- BN06. Mihir Bellare and Gregory Neven. Multi-signatures in the plain public-key model and a general forking lemma. In Ari Juels, Rebecca N. Wright, and Sabrina De Capitani di Vimercati, editors, *ACM CCS 2006: 13th Conference on Computer and Communications Security*, pages 390–399. ACM Press, October / November 2006. doi:10.1145/1180405.1180453.
- Can00. Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. Cryptology ePrint Archive, Report 2000/067, 2000. URL: <https://eprint.iacr.org/2000/067>.
- CCL15. Ran Canetti, Asaf Cohen, and Yehuda Lindell. A simpler variant of universally composable security for standard multiparty computation. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *Advances in Cryptology – CRYPTO 2015, Part II*, volume 9216 of *Lecture Notes in Computer Science*, pages 3–22. Springer, Berlin, Heidelberg, August 2015. doi:10.1007/978-3-662-48000-7_1.
- CCL⁺20. Guilhem Castagnos, Dario Catalano, Fabien Laguillaumie, Federico Savasta, and Ida Tucker. Bandwidth-efficient threshold ECDSA. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020: 23rd International Conference on Theory and Practice of Public Key Cryptography, Part II*, volume 12111 of *Lecture Notes in Computer Science*, pages 266–296. Springer, Cham, May 2020. doi:10.1007/978-3-030-45388-6_10.

- CGG⁺20. Ran Canetti, Rosario Gennaro, Steven Goldfeder, Nikolaos Makriyannis, and Udi Peled. UC non-interactive, proactive, threshold ECDSA with identifiable aborts. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020: 27th Conference on Computer and Communications Security*, pages 1769–1787. ACM Press, November 2020. doi:10.1145/3372297.3423367.
- DKLs18. Jack Doerner, Yashvanth Kondi, Eysa Lee, and abhi shelat. Secure two-party threshold ECDSA from ECDSA assumptions. In *2018 IEEE Symposium on Security and Privacy*, pages 980–997. IEEE Computer Society Press, May 2018. doi:10.1109/SP.2018.00036.
- DKLs19. Jack Doerner, Yashvanth Kondi, Eysa Lee, and abhi shelat. Threshold ECDSA from ECDSA assumptions: The multiparty case. In *2019 IEEE Symposium on Security and Privacy*, pages 1051–1066. IEEE Computer Society Press, May 2019. doi:10.1109/SP.2019.00024.
- DKLs23. Jack Doerner, Yashvanth Kondi, Eysa Lee, and abhi shelat. Threshold ECDSA in three rounds. Cryptology ePrint Archive, Report 2023/765, 2023. URL: <https://eprint.iacr.org/2023/765>.
- DKLs24. Jack Doerner, Yashvanth Kondi, Eysa Lee, and abhi shelat. Threshold ECDSA in three rounds. In *2024 IEEE Symposium on Security and Privacy*, pages 3053–3071. IEEE Computer Society Press, May 2024. doi:10.1109/SP54263.2024.00178.
- GG18. Rosario Gennaro and Steven Goldfeder. Fast multiparty threshold ECDSA with fast trustless setup. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018: 25th Conference on Computer and Communications Security*, pages 1179–1194. ACM Press, October 2018. doi:10.1145/3243734.3243859.
- GJKR96. Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. Robust threshold DSS signatures. In Ueli M. Maurer, editor, *Advances in Cryptology – EUROCRYPT’96*, volume 1070 of *Lecture Notes in Computer Science*, pages 354–371. Springer, Berlin, Heidelberg, May 1996. doi:10.1007/3-540-68339-9_31.
- GS22a. Jens Groth and Victor Shoup. Design and analysis of a distributed ECDSA signing service. Cryptology ePrint Archive, Report 2022/506, 2022. URL: <https://eprint.iacr.org/2022/506>.
- GS22b. Jens Groth and Victor Shoup. On the security of ECDSA with additive key derivation and presignatures. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022, Part I*, volume 13275 of *Lecture Notes in Computer Science*, pages 365–396. Springer, Cham, May / June 2022. doi:10.1007/978-3-031-06944-4_13.
- KG20. Chelsea Komlo and Ian Goldberg. FROST: Flexible round-optimized Schnorr threshold signatures. In Orr Dunkelman, Michael J. Jacobson, Jr., and Colin O’Flynn, editors, *SAC 2020: 27th Annual International Workshop on Selected Areas in Cryptography*, volume 12804 of *Lecture Notes in Computer Science*, pages 34–65. Springer, Cham, October 2020. doi:10.1007/978-3-030-81652-0_2.
- KG24. Brian Koziel, S. Dov Gordon, and Craig Gentry. Fast two-party threshold ECDSA with proactive security. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024: 31st Conference on Computer and Communications Security*, pages 1567–1580. ACM Press, October 2024. doi:10.1145/3658644.3670387.

- KLR10. Eyal Kushilevitz, Yehuda Lindell, and Tal Rabin. Information-theoretically secure protocols and security under composition. *SIAM Journal on Computing*, 39(5):2090–2112, 2010. doi:[10.1137/090755886](https://doi.org/10.1137/090755886).
- Lan95. Susan K. Langford. Threshold dss signatures without a trusted party. In Don Coppersmith, editor, *Advances in Cryptology – CRYPTO’95*, volume 963 of *Lecture Notes in Computer Science*, pages 397–409. Springer, Berlin, Heidelberg, August 1995. doi:[10.1007/3-540-44750-4_32](https://doi.org/10.1007/3-540-44750-4_32).
- Lin17. Yehuda Lindell. Fast secure two-party ECDSA signing. In Jonathan Katz and Hovav Shacham, editors, *Advances in Cryptology – CRYPTO 2017, Part II*, volume 10402 of *Lecture Notes in Computer Science*, pages 613–644. Springer, Cham, August 2017. doi:[10.1007/978-3-319-63715-0_21](https://doi.org/10.1007/978-3-319-63715-0_21).
- Lin24. Yehuda Lindell. Simple three-round multiparty Schnorr signing with full simulatability. *IACR Communications in Cryptology (CiC)*, 1(1):25, 2024. doi:[10.62056/a36c015vt](https://doi.org/10.62056/a36c015vt).
- LN18. Yehuda Lindell and Ariel Nof. Fast secure multiparty ECDSA with practical distributed key generation and applications to cryptocurrency custody. In David Lie, Mohammad Mannan, Michael Backes, and Xiaofeng Wang, editors, *ACM CCS 2018: 25th Conference on Computer and Communications Security*, pages 1837–1854. ACM Press, October 2018. doi:[10.1145/3243734.3243788](https://doi.org/10.1145/3243734.3243788).
- LS26. Yehuda Lindell and Gil Segev. De-folklorizing Paillier-based VOLE: UC security and reusable keys, 2026.
- Mak22. Nikolaos Makriyannis. On the classic protocol for MPC Schnorr signatures. Cryptology ePrint Archive, Report 2022/1332, 2022. URL: <https://eprint.iacr.org/2022/1332>.
- MOR01. Silvio Micali, Kazuo Ohta, and Leonid Reyzin. Accountable-subgroup multisignatures: Extended abstract. In Michael K. Reiter and Pierangela Samarati, editors, *ACM CCS 2001: 8th Conference on Computer and Communications Security*, pages 245–254. ACM Press, November 2001. doi:[10.1145/501983.502017](https://doi.org/10.1145/501983.502017).
- NKDM03. Antonio Nicolosi, Maxwell N. Krohn, Yevgeniy Dodis, and David Mazières. Proactive two-party signatures for user authentication. In *ISOC Network and Distributed System Security Symposium – NDSS 2003*. The Internet Society, February 2003.
- Sch90. Claus-Peter Schnorr. Efficient identification and signatures for smart cards. In Gilles Brassard, editor, *Advances in Cryptology – CRYPTO’89*, volume 435 of *Lecture Notes in Computer Science*, pages 239–252. Springer, New York, August 1990. doi:[10.1007/0-387-34805-0_22](https://doi.org/10.1007/0-387-34805-0_22).
- Wui14. Pieter Wuille. BIP 0062: Dealing with malleability. <https://github.com/bitcoin/bips/blob/master/bip-0062.mediawiki>, 2014.

A Realizing the Relaxed Zero-Sharing Functionality

In this section we prove Theorem 3.1 stating that protocol $\Pi_{\text{rZero}[q,n]}$ UC-realizes the relaxed zero-sharing functionality $\mathcal{F}_{\text{rZero}[q,n]}$ (both defined in Section 3). Motivated by the Simplified UC (SUC) framework introduced by Canetti, Cohen, and Lindell [CCL15], and which formalizes common UC protocol settings, for clarity and simplicity we present our analysis assuming a fixed set of parties,

authenticated channels, and the UC framework's standard delayed output mechanism. As standard in the UC framework, in what follows for every black-box simulator $\mathcal{S}$, real-model adversary $\mathcal{A}$, security parameter $\kappa_c \in \mathbb{N}$, and input $z \in \{0, 1\}^*$ provided to the environment, we denote by $\text{REAL}_{\Pi_{\text{rZero}[q,n]}, \mathcal{A}, \mathcal{Z}}(\kappa_c, z)$ and $\text{IDEAL}_{\mathcal{F}_{\text{rZero}[q,n]}, \mathcal{S}^{\mathcal{A}}, \mathcal{Z}}(\kappa_c, z)$ the distributions corresponding to the output of the environment following an execution of the protocol $\Pi_{\text{rZero}[q,n]}$ in the real model and following an ideal execution with the functionality $\mathcal{F}_{\text{rZero}[q,n]}$, respectively.

Proof of Theorem 3.1. Fix a prime $q \geq 2$ and integer $n \geq 2$, and let $\mathcal{A}$ be a malicious real-model adversary that statically corrupts up to $n - 1$ parties. We present a probabilistic polynomial-time black-box simulator $\mathcal{S}$ such that for every probabilistic polynomial-time environment $\mathcal{Z}$, polynomial $p(\cdot)$, security parameter $\kappa_c \in \mathbb{N}$, and for every input $z \in \{0, 1\}^{p(\kappa_c)}$ provided to the environment $\mathcal{Z}$, it holds that

$$\text{REAL}_{\Pi_{\text{rZero}[q,n]}, \mathcal{A}, \mathcal{Z}}(\kappa_c, z) \equiv \text{IDEAL}_{\mathcal{F}_{\text{rZero}[q,n]}, \mathcal{S}^{\mathcal{A}}, \mathcal{Z}}(\kappa_c, z). \quad (\text{A.1})$$

The simulator $\mathcal{S}^{\mathcal{A}}(\kappa_c)$, defined below, emulates a real-model execution to the adversary $\mathcal{A}$ while forwarding all messages from the environment $\mathcal{Z}$ to $\mathcal{A}$ and vice versa. When the emulated execution begins, the real-model adversary $\mathcal{A}$ announces the identities of up to $n - 1$ corrupt parties, and we denote by $\mathcal{C} \subseteq [n]$ and $\mathcal{H} = [n] \setminus \mathcal{C}$ the sets of identities of the corrupted parties and honest parties, respectively. The simulator $\mathcal{S}$ interacts with the ideal functionality $\mathcal{F}_{\text{rZero}[q,n]}$ on behalf of every corrupted party, and emulates a real-model execution of the protocol $\Pi_{\text{rZero}[q,n]}$ to the real-model adversary $\mathcal{A}$ by interacting with $\mathcal{A}$ and the corrupted parties on behalf of all honest parties. The simulator $\mathcal{S}^{\mathcal{A}}$ is defined as follows upon receiving (choose, sid) from the ideal functionality $\mathcal{F}_{\text{rZero}[q,n]}$:

1. For every $h \in \mathcal{H}$ and $c \in \mathcal{C}$ such that $h < c$ uniformly sample $r_{h,c} \in_R \mathbb{Z}_q$ and send it to party P_c (i.e., to $\mathcal{A}$) on behalf of party P_h .
2. For every $c \in \mathcal{C}$ and $h \in \mathcal{H}$ such that $c < h$ obtain the value $r_{c,h}$ sent by $\mathcal{A}$ on behalf of party P_c to party P_h .
3. Let $T = \sum_{\substack{c \in \mathcal{C}, h \in \mathcal{H} \\ c < h}} r_{c,h} - \sum_{\substack{h \in \mathcal{H}, c \in \mathcal{C} \\ h < c}} r_{h,c} \bmod q$.
4. Send T to the ideal functionality $\mathcal{F}_{\text{rZero}[q,n]}$.

For establishing Eq. (A.1), we first prove that $\mathcal{A}$'s view is distributed identically in both models, and then prove that the outputs of the honest parties are distributed identically in both models when conditioned on any fixed view for $\mathcal{A}$. These imply that the entire view of the environment $\mathcal{Z}$ is identically distributed in both models.

First, considering $\mathcal{A}$'s view, in both models it consists of $\{r_{h,c}\}_{h \in \mathcal{H}, c \in \mathcal{C}, h < c}$ that are sampled uniformly and independently from $\mathbb{Z}_q$ for any fresh sid. In the real model this follows directly from the definition of the protocol $\Pi_{\text{rZero}[q,n]}$, and in the ideal model this follows directly from the definition of the simulator $\mathcal{S}$. Thus, $\mathcal{A}$'s view is distributed identically in both models.

Second, considering the outputs of the honest parties, fix any fresh sid and any view $\{r_{h,c}\}_{h \in \mathcal{H}, c \in \mathcal{C}, h < c}$ for $\mathcal{A}$, and denote by $\{r_{c,h}\}_{c \in \mathcal{C}, h \in \mathcal{H}, c < h}$ the response provided by $\mathcal{A}$ (note that since we have already established that $\mathcal{A}$'s view is distributed identically in both models, the same holds for $\mathcal{A}$'s response). In the ideal model, by the definition of the simulator $\mathcal{S}$, it sends the value T to the ideal functionality $\mathcal{F}_{\text{rZero}[q,n]}$, where T is defined as

$$T = \sum_{\substack{c \in \mathcal{C}, h \in \mathcal{H} \\ c < h}} r_{c,h} - \sum_{\substack{h \in \mathcal{H}, c \in \mathcal{C} \\ h < c}} r_{h,c} \pmod{q}.$$

By the definition of the ideal functionality $\mathcal{F}_{\text{rZero}[q,n]}$, for any such fresh sid the outputs $\{\zeta_h\}_{h \in \mathcal{H}}$ of the honest parties are sampled independently and uniformly from $\mathbb{Z}_q$ subject to $\sum_{h \in \mathcal{H}} \zeta_h = -T \pmod{q}$. For showing that this holds also in the real model, additionally denote by $r_{h,h'}$ the value sent from P_h to $P_{h'}$ for every two honest parties $h, h' \in \mathcal{H}$ such that $h < h'$. By the definition of the protocol, these values are sampled uniformly and independently from $\mathbb{Z}_q$ for any fresh sid , and the outputs $\{\zeta_h\}_{h \in \mathcal{H}}$ of the honest parties satisfy

$$\begin{aligned} \sum_{h \in \mathcal{H}} \zeta_h &= \sum_{h \in \mathcal{H}} \left(\sum_{h < j} r_{h,j} - \sum_{j < h} r_{j,h} \right) \pmod{q} \\ &= \sum_{h \in \mathcal{H}} \left(\sum_{\substack{h' \in \mathcal{H} \\ h < h'}} r_{h,h'} + \sum_{\substack{c \in \mathcal{C} \\ h < c}} r_{h,c} - \sum_{\substack{h' \in \mathcal{H} \\ h' < h}} r_{h',h} - \sum_{\substack{c \in \mathcal{C} \\ c < h}} r_{c,h} \right) \pmod{q} \\ &= \sum_{\substack{h, h' \in \mathcal{H} \\ h < h'}} r_{h,h'} + \sum_{\substack{h \in \mathcal{H}, c \in \mathcal{C} \\ h < c}} r_{h,c} - \sum_{\substack{h', h \in \mathcal{H} \\ h' < h}} r_{h',h} - \sum_{\substack{h \in \mathcal{H}, c \in \mathcal{C} \\ c < h}} r_{c,h} \pmod{q} \\ &= \sum_{\substack{h \in \mathcal{H}, c \in \mathcal{C} \\ h < c}} r_{h,c} - \sum_{\substack{c \in \mathcal{C}, h \in \mathcal{H} \\ c < h}} r_{c,h} \pmod{q} \\ &= -T \pmod{q}. \end{aligned}$$

Thus, the sum of the outputs of the honest parties is distributed exactly as in the ideal model. As for the joint distribution of the individual values $\{\zeta_h\}_{h \in \mathcal{H}}$ subject to their sum, if there is only one honest party then clearly the sum determines the output of the honest party. If there are at least two honest parties, then conditioned on any $\{r_{h,c}\}_{h \in \mathcal{H}, c \in \mathcal{C}, h < c}$ and $\{r_{c,h}\}_{c \in \mathcal{C}, h \in \mathcal{H}, c < h}$, consider the following set of equations that define the honest parties' shares:

$$\left\{ \zeta_h = \sum_{\substack{h' \in \mathcal{H} \\ h < h'}} r_{h,h'} - \sum_{\substack{h' \in \mathcal{H} \\ h' < h}} r_{h',h} + C_h \pmod{q} \right\}_{h \in \mathcal{H}},$$

where

$$C_h = \sum_{\substack{c \in \mathcal{C} \\ h < c}} r_{h,c} - \sum_{\substack{c \in \mathcal{C} \\ c < h}} r_{c,h} \pmod{q}$$

is a constant determined by $\mathcal{A}$'s fixed view and response. This set consists of $|\mathcal{H}|$ linear equations for the shares $\{\zeta_h\}_{h \in \mathcal{H}}$ with variables being the values $\{r_{h,h'}\}_{h,h' \in \mathcal{H}, h < h'}$ exchanged between pairs of honest parties. By the definition of the protocol, these $\binom{|\mathcal{H}|}{2}$ variables are sampled uniformly and independently from $\mathbb{Z}_q$ by the honest parties and are hidden from $\mathcal{A}$.

This system of equations defines a surjective affine transformation from the space of the uniformly distributed random variables $\{r_{h,h'}\}_{h,h' \in \mathcal{H}, h < h'}$ to the $(|\mathcal{H}| - 1)$ -dimensional affine subspace defined by the condition $\sum_{h \in \mathcal{H}} \zeta_h = -T \bmod q$. Since the inputs to this transformation (i.e., the $r_{h,h'}$ values) are uniformly and independently distributed, the output vector $\{\zeta_h\}_{h \in \mathcal{H}}$ is distributed uniformly over all possible vectors in $\mathbb{Z}_q^{|\mathcal{H}|}$ that sum to $-T$ modulo q . This is precisely the distribution of the honest parties' shares in the ideal model, as defined by the ideal functionality $\mathcal{F}_{\text{rZero}[q,n]}$. Therefore, the joint distribution of the outputs of the honest parties is identical in both models conditioned on any fixed view for $\mathcal{A}$, which completes the proof. ■

B Proof of Theorem 5.2

For the proof of Theorem 5.2 we rely on the following standard notation:²

- $\text{HYBRID}_{\Pi_{\text{ECDSA}}, \mathcal{A}(\text{aux}), \mathcal{C}}^{\mathcal{F}}(\alpha, \kappa)$ denotes the joint distribution of the random variables corresponding to the output of the hybrid-model adversary $\mathcal{A}$ (when provided with the auxiliary input aux) and the outputs of the honest parties in an execution of the protocol Π_{ECDSA} in the $\mathcal{F}$ -hybrid model, where $\mathcal{C}$ denotes the set of corrupted parties, $\alpha = (\alpha_1, \dots, \alpha_n)$ denotes the parties' inputs, and $\kappa \in \mathbb{N}$ denotes the security parameter.
- $\text{IDEAL}_{\mathcal{F}_{\text{ECDSA}}, \mathcal{S}(\text{aux}), \mathcal{C}}(\alpha, \kappa)$ the joint distribution of the random variables corresponding to the output of the ideal-model adversary $\mathcal{S}$ (when provided with the auxiliary input aux) and the outputs of the honest parties in an ideal-model computation of the functionality $\mathcal{F}_{\text{ECDSA}}$, where $\mathcal{C}$ denotes the set of corrupted parties, $\alpha = (\alpha_1, \dots, \alpha_n)$ denotes the parties' inputs, and $\kappa \in \mathbb{N}$ denotes the security parameter.

For simplicity, our proof is presented in a synchronous communication model. Following the proof, we explain how to extend the analysis to the asynchronous setting.

Proof of Theorem 5.2. Let $\mathcal{A}$ be a probabilistic polynomial-time adversary in the $\mathcal{F}$ -hybrid model, where $\mathcal{F} = (\mathcal{F}_{\text{Com}}, \mathcal{F}_{\text{rZero}[q,n]}, \mathcal{F}_{\text{rVOLE}[q,2]})$. Denote by $\mathcal{C} \subsetneq [n]$ the indices corresponding to the parties corrupted by $\mathcal{A}$, and let $\mathcal{H} = [n] \setminus \mathcal{C}$. For every $i \in [n]$ denote by $(\text{sid}, m, Q, Q_1, \dots, Q_n, x_i)$ the prescribed input held by party P_i (recall that the parties are assumed to hold valid and consistent inputs), and let $\alpha = \{(\text{sid}, m, Q, Q_1, \dots, Q_n, x_i)\}_{i \in [n]}$. We present a probabilistic polynomial-time ideal-model adversary $\mathcal{S}$ (i.e., a simulator) such that

² In what follows, since Theorem 5.2 establishes statistical security, then the security parameter κ may correspond to a statistical security parameter.

for every $\kappa \in \mathbb{N}$, for every α that consists of valid and consistent inputs, and for every auxiliary information $\text{aux} \in \{0,1\}^*$, it holds that the distributions $\text{HYBRID}_{\mathcal{H}_{\text{ECDSA}}, \mathcal{A}(\text{aux}), \mathcal{C}}^{\mathcal{F}}(\alpha, \kappa)$ and $\text{IDEAL}_{\mathcal{F}_{\text{ECDSA}}, \mathcal{S}(\text{aux}), \mathcal{C}}(\alpha, \kappa)$ are statistically close. The ideal-model adversary $\mathcal{S}$ is defined as follows:

1. On input $(\{(\text{sid}, m, Q, Q_1, \dots, Q_n, x_c)\}_{c \in \mathcal{C}}, \text{aux})$, for every $c \in \mathcal{C}$ send $(\text{sign}, \text{sid}, m, Q, Q_1, \dots, Q_n, x_c)$ to the functionality $\mathcal{F}_{\text{ECDSA}}$ on behalf of party P_c , and obtain (sid, σ) .
2. Parse $\sigma = (r, s)$, and compute $R = (s^{-1} \cdot H(m)) \cdot G + (s^{-1} \cdot r) \cdot Q$.
3. Invoke $\mathcal{A}$ on the input $(\{(\text{sid}, m, Q, Q_1, \dots, Q_n, x_c)\}_{c \in \mathcal{C}}, \text{aux})$.
4. For every $c \in \mathcal{C}$ and $h \in \mathcal{H}$ send $\mathcal{A}$ the following messages:
 - $(\text{receipt}, \text{sid} || \text{pid}_h || \text{pid}_c, h, c)$ on behalf of $\mathcal{F}_{\text{Com}}$.
 - $(\text{ready}, \text{sid} || \text{pid}_h || \text{pid}_c, h, c)$ on behalf of $\mathcal{F}_{\text{RVOLE}[q,2]}$.
5. Send the message $(\text{choose}, \text{sid})$ to $\mathcal{A}$ on behalf of $\mathcal{F}_{\text{rZero}[q,n]}$. When $\mathcal{A}$ replies with $t \in \mathbb{Z}_q$, sample $\{\zeta_c\}_{c \in \mathcal{C}}$ uniformly and independently from $\mathbb{Z}_q$ subject to $\sum_{c \in \mathcal{C}} \zeta_c = t \bmod q$, and for every $c \in \mathcal{C}$ send $(\text{mask}, \text{sid}, \zeta_c)$ to $\mathcal{A}$ on behalf of $\mathcal{F}_{\text{rZero}[q,n]}$.
6. For every $c \in \mathcal{C}$ and $h \in \mathcal{H}$ obtain the following messages from $\mathcal{A}$:
 - $(\text{commit}, \text{sid} || \text{pid}_c || \text{pid}_h, c, h, R_{c,h})$ sent by $\mathcal{A}$ on behalf of party P_c to $\mathcal{F}_{\text{Com}}$.
 - $(\text{Alice-sample}, \text{sid} || \text{pid}_c || \text{pid}_h, c, h, \chi_{c,h})$ and $(\text{Alice-share}, \text{sid} || \text{pid}_c || \text{pid}_h, c, h, (d_{c,h}^u, d_{c,h}^v))$ sent by $\mathcal{A}$ on behalf of party P_c to $\mathcal{F}_{\text{RVOLE}[q,2]}$.
7. Fix some $h^* \in \mathcal{H}$ (e.g., the smallest index $h \in [n]$ of an honest party P_h).
8. For every $h \in \mathcal{H} \setminus \{h^*\}$ sample $\hat{x}_h, k_h \in_R \mathbb{Z}_q$ and compute $\hat{Q}_h = \hat{x}_h \cdot G$ and $R_h = k_h \cdot G$ (note that the simulator does not generate the values $\hat{x}_{h^*}$ and k_{h^*}).
9. Compute

$$\begin{aligned} \hat{Q}_{h^*} &= Q - \sum_{h \in \mathcal{H} \setminus \{h^*\}} \hat{Q}_h - \sum_{c \in \mathcal{C}} (Q_c + \zeta_c \cdot G) \\ R_{h^*} &= R - \sum_{h \in \mathcal{H} \setminus \{h^*\}} R_h - \sum_{c \in \mathcal{C}} R_{c,h^*} \end{aligned}$$

10. For every $c \in \mathcal{C}$ and $h \in \mathcal{H}$ sample $\psi_{h,c} \in_R \mathbb{Z}_q$, compute

$$\begin{aligned} C_{h,c}^u &= \chi_{c,h} \cdot R_h - d_{c,h}^u \cdot G \\ C_{h,c}^v &= \chi_{c,h} \cdot \hat{Q}_h - d_{c,h}^v \cdot G, \end{aligned}$$

and send $\mathcal{A}$ the following messages:

- $(\text{decommit}, \text{sid} || \text{pid}_h || \text{pid}_c, h, c, R_h)$ on behalf of $\mathcal{F}_{\text{Com}}$.
 - $(\text{sid} || \text{pid}_h || \text{pid}_c, C_{h,c}^u, C_{h,c}^v, \psi_{h,c}, \hat{Q}_h)$ on behalf of party P_h .
11. For every $c \in \mathcal{C}$ and $h \in \mathcal{H}$ obtain the following messages from $\mathcal{A}$:
 - $(\text{Bob-multiply}, \text{sid} || \text{pid}_h || \text{pid}_c, h, c, (k_{c,h}, \hat{x}_{c,h}))$ sent by $\mathcal{A}$ on behalf of party P_c to $\mathcal{F}_{\text{RVOLE}[q,2]}$.
 - $(\text{Bob-share}, \text{sid} || \text{pid}_h || \text{pid}_c, h, c, (c_{c,h}^u, c_{c,h}^v))$ sent by $\mathcal{A}$ on behalf of party P_c to $\mathcal{F}_{\text{RVOLE}[q,2]}$.
 - $(\text{sid} || \text{pid}_c || \text{pid}_h, C_{c,h}^u, C_{c,h}^v, \psi_{c,h}, \hat{Q}_{c,h})$ sent by $\mathcal{A}$ on behalf of party P_c to party P_h .

12. For every $h \in \mathcal{H} \setminus \{h^*\}$:
 If $\sum_{c \in \mathcal{C}} \hat{Q}_{c,h} \neq \sum_{c \in \mathcal{C}} (Q_c + \zeta_c \cdot G)$, or if there exists $c \in \mathcal{C}$ such that $k_{c,h} \cdot G \neq R_{c,h}$ or $\hat{x}_{c,h} \cdot G \neq \hat{Q}_{c,h}$ or $c_{c,h}^u \cdot G \neq C_{c,h}^u$ or $c_{c,h}^v \cdot G \neq C_{c,h}^v$ then emulate party P_h aborting, instruct the trusted party computing the functionality $\mathcal{F}_{\text{ECDSA}}$ in the ideal model to send **abort** to all honest parties, output $\mathcal{A}$'s output, and halt. Otherwise, sample $u_h, v_h, \phi_h \in_R \mathbb{Z}_q$, compute

$$\begin{aligned} \tilde{R}_h &= \sum_{h' \in \mathcal{H}} R_{h'} + \sum_{c \in \mathcal{C}} R_{c,h} \\ w_h &= H(m) \cdot \phi_h + r_h \cdot v_h \bmod q, \end{aligned}$$

where $r_h \in \mathbb{Z}_q$ is the x -coordinate modulo q of $\tilde{R}_h$, and for every $c \in \mathcal{C}$ send the message $(\text{sid} || \text{pid}_h || \text{pid}_c, \tilde{R}_h, w_h, u_h)$ to $\mathcal{A}$ on behalf of party P_h .

13. If $\sum_{c \in \mathcal{C}} \hat{Q}_{c,h^*} \neq \sum_{c \in \mathcal{C}} (Q_c + \zeta_c \cdot G)$, or if there exists $c \in \mathcal{C}$ such that $k_{c,h^*} \cdot G \neq R_{c,h^*}$ or $\hat{x}_{c,h^*} \cdot G \neq \hat{Q}_{c,h^*}$ or $c_{c,h^*}^u \cdot G \neq C_{c,h^*}^u$ or $c_{c,h^*}^v \cdot G \neq C_{c,h^*}^v$ then emulate party P_{h^*} aborting, instruct the trusted party computing the functionality $\mathcal{F}_{\text{ECDSA}}$ in the ideal model to send **abort** to all honest parties, output $\mathcal{A}$'s output, and halt. Otherwise, proceed as follows:

- (a) Sample $u_{h^*} \in_R \mathbb{Z}_q$.
 (b) Compute:

$$\begin{aligned} \tilde{R}_{h^*} &= R \\ u_{\mathcal{H} \setminus \{h^*\}} &= \sum_{h \in \mathcal{H} \setminus \{h^*\}} u_h \bmod q \\ v_{\mathcal{H} \setminus \{h^*\}} &= \sum_{h \in \mathcal{H} \setminus \{h^*\}} v_h \bmod q \\ u_{\mathcal{C}} &= \sum_{c \in \mathcal{C}} \sum_{h \in \mathcal{H}} (k_{c,h^*} \cdot \psi_{h,c} + c_{c,h}^u + d_{c,h}^u) \\ &\quad + \left(\sum_{c \in \mathcal{C}} k_{c,h^*} \right) \cdot \left(\sum_{c \in \mathcal{C}} (\psi_{c,h^*} + \chi_{c,h^*}) \right) \bmod q \\ v_{\mathcal{C}} &= \sum_{c \in \mathcal{C}} \sum_{h \in \mathcal{H}} (\hat{x}_{c,h^*} \cdot \psi_{h,c} + c_{c,h}^v + d_{c,h}^v) \\ &\quad + \left(\sum_{c \in \mathcal{C}} \hat{x}_{c,h^*} \right) \cdot \left(\sum_{c \in \mathcal{C}} (\psi_{c,h^*} + \chi_{c,h^*}) \right) \bmod q \\ w_{\mathcal{H} \setminus \{h^*\}} &= H(m) \cdot \left(\sum_{h \in \mathcal{H} \setminus \{h^*\}} \phi_h \right) + r \cdot v_{\mathcal{H} \setminus \{h^*\}} \bmod q \\ w_{\mathcal{C}} &= H(m) \cdot \left(\sum_{c \in \mathcal{C}} (\psi_{c,h^*} + \chi_{c,h^*}) \right) + r \cdot v_{\mathcal{C}} \bmod q \\ w_{h^*} &= s \cdot (u_{h^*} + u_{\mathcal{H} \setminus \{h^*\}} + u_{\mathcal{C}}) - (w_{\mathcal{H} \setminus \{h^*\}} + w_{\mathcal{C}}) \bmod q \end{aligned}$$

- (c) For every $c \in \mathcal{C}$ send the message $(\text{sid}||\text{pid}_{h^*}||\text{pid}_c, R, w_{h^*}, u_{h^*})$ to $\mathcal{A}$ on behalf of party P_{h^*} .
- 14. For every $c \in \mathcal{C}$ and $h \in \mathcal{H}$ obtain the message $(\text{sid}||\text{pid}_c||\text{pid}_h, \tilde{R}_{c,h}, w_{c,h}, u_{c,h})$ sent from $\mathcal{A}$ to P_h on behalf of P_c .
- 15. If for some $h \neq h' \in \mathcal{H}$ it holds that $\tilde{R}_h \neq \tilde{R}_{h'}$ then instruct the trusted party computing the functionality $\mathcal{F}_{\text{ECDSA}}$ in the ideal model to send **abort** to all honest parties, output $\mathcal{A}$'s output, and halt.
- 16. For every $h \in \mathcal{H}$:
 - (a) If for some $c \in \mathcal{C}$ it holds that $\tilde{R}_{c,h} \neq \tilde{R}_h$ then emulate party P_h aborting, and instruct the trusted party computing the functionality $\mathcal{F}_{\text{ECDSA}}$ in the ideal model to send **abort** to party P_h .
 - (b) If

$$s = \left(\sum_{h' \in \mathcal{H}} w_{h'} + \sum_{c \in \mathcal{C}} w_{c,h} \right) \cdot \left(\sum_{h' \in \mathcal{H}} u_{h'} + \sum_{c \in \mathcal{C}} u_{c,h} \right)^{-1} \bmod q$$

then instruct the trusted party computing the functionality $\mathcal{F}_{\text{ECDSA}}$ in the ideal model to send party P_h its output, and otherwise instruct it to send **abort** to party P_h .

(Note that (r, s) is a valid signature since the simulator obtained it from the functionality $\mathcal{F}_{\text{ECDSA}}$, and that (r, s) passes the verification in Step 4c of the protocol since the simulator explicitly defined $R = (s^{-1} \cdot H(m)) \cdot G + (s^{-1} \cdot r) \cdot Q$.)

- 17. Output $\mathcal{A}$'s output.

For completing the formal description of the ideal-model adversary $\mathcal{S}$, we note that throughout its execution it additionally simulates all communication between honest parties, as well as their interactions with the ideal functionalities $\mathcal{F}_{\text{Com}}$, $\mathcal{F}_{\text{rZero}[q,n]}$ and $\mathcal{F}_{\text{rVole}[q,2]}$. Since these parties are not corrupted, $\mathcal{S}$ simply generates dummy messages of the exact required lengths and simulates the standard delayed-output notifications (e.g., passing “ready” and “receipt” signals to $\mathcal{A}$ and waiting for “continue” signals) to perfectly replicate the network metadata and scheduling profile of a real execution.

Provided with the above description of the simulator $\mathcal{S}$, we consider a sequence of experiments $\mathcal{H}_1, \dots, \mathcal{H}_{11}$, and prove that the distributions of $\mathcal{A}$'s view and the output of the honest parties in each pair of consecutive experiments $\mathcal{H}_i$ and $\mathcal{H}_{i+1}$ are statistically indistinguishable. We abuse notation and additionally denote by $\mathcal{H}_i = \mathcal{H}_i(\alpha, \text{aux}, \kappa)$ the latter joint distribution in the i th experiment where $\alpha = (\alpha_1, \dots, \alpha_n)$ denotes the parties' inputs, κ denotes the security parameter, and the adversary is additionally provided the auxiliary input aux . The sequence of experiments begins with the execution of the protocol Π_{ECDSA} in the $\mathcal{F}$ -hybrid model. Then, by gradually replacing the code of the honest parties with that of the simulator $\mathcal{S}$, the sequence ends with the ideal-model computation of the functionality $\mathcal{F}_{\text{ECDSA}}$ with the simulator $\mathcal{S}$.

- **Experiment $\mathcal{H}_1(\alpha, \text{aux}, \kappa)$.** This experiment is obtained from the execution of the protocol Π_{ECDSA} in the $\mathcal{F}$ -hybrid model by replacing the honest

parties and the functionality $\mathcal{F}$ with a single emulator algorithm that emulates them and interacts with the adversary $\mathcal{A}$ on their behalf.

Experiment $\mathcal{H}_1$ does not modify the view of the adversary $\mathcal{A}$ or the outputs of the honest parties compared to an execution of the protocol Π_{ECDSA} in the $\mathcal{F}$ -hybrid model, and thus

$$\{\mathcal{H}_1(\alpha, \text{aux}, \kappa)\}_{\alpha, \text{aux} \in \{0,1\}^*, \kappa \in \mathbb{N}} \equiv \{\text{HYBRID}_{\Pi_{\text{ECDSA}}, \mathcal{A}(\text{aux}), \mathcal{C}}^{\mathcal{F}}(\alpha, \kappa)\}_{\alpha, \text{aux} \in \{0,1\}^*, \kappa \in \mathbb{N}}.$$

- **Experiment $\mathcal{H}_2(\alpha, \text{aux}, \kappa)$.** This experiment is obtained from experiment $\mathcal{H}_1$ by modifying the consistency check performed by the emulator on behalf of the honest parties in Step 3b of the protocol. For each honest party P_h and corrupted party P_c , recall that in Step 3b of the protocol (and thus in experiment $\mathcal{H}_1$) party P_h verifies that

$$\chi_{h,c} \cdot R_{c,h} - C_{c,h}^u = d_{h,c}^u \cdot G \quad (\text{B.1})$$

$$\chi_{h,c} \cdot \hat{Q}_{c,h} - C_{c,h}^v = d_{h,c}^v \cdot G \quad (\text{B.2})$$

and otherwise abort. Now, in experiment $\mathcal{H}_2$, the emulator instead verifies that

$$k_{c,h} \cdot G = R_{c,h} \quad (\text{B.3})$$

$$\hat{x}_{c,h} \cdot G = \hat{Q}_{c,h} \quad (\text{B.4})$$

$$c_{c,h}^u \cdot G = C_{c,h}^u \quad (\text{B.5})$$

$$c_{c,h}^v \cdot G = C_{c,h}^v \quad (\text{B.6})$$

and otherwise emulates an abort on behalf of P_h .

We show that the statistical distance between the distributions $\mathcal{H}_1(\alpha, \text{aux}, \kappa)$ and $\mathcal{H}_2(\alpha, \text{aux}, \kappa)$ is at most $n^2/(2q)$. By the definition of the functionality $\mathcal{F}_{\text{RVOLE}}$ it holds that

$$\begin{aligned} c_{c,h}^u + d_{h,c}^u &= \chi_{h,c} \cdot k_{c,h} \bmod q \\ c_{c,h}^v + d_{h,c}^v &= \chi_{h,c} \cdot \hat{x}_{c,h} \bmod q \end{aligned}$$

and therefore Eq. (B.1)–(B.2) are equivalent to

$$\chi_{h,c} \cdot (R_{c,h} - k_{c,h} \cdot G) = C_{c,h}^u - c_{c,h}^u \cdot G \quad (\text{B.7})$$

$$\chi_{h,c} \cdot (\hat{Q}_{c,h} - \hat{x}_{c,h} \cdot G) = C_{c,h}^v - c_{c,h}^v \cdot G. \quad (\text{B.8})$$

On the one hand, this shows that if Eq. (B.3)–(B.6) are satisfied then Eq. (B.7)–(B.8) are satisfied as well. On the other hand, we observe the following:

- If $k_{c,h} \cdot G \neq R_{c,h}$ or $c_{c,h}^u \cdot G \neq C_{c,h}^u$, then for Eq. (B.7) to be satisfied it must hold that $k_{c,h} \cdot G \neq R_{c,h}$. In this case, Eq. (B.7) is satisfied only for

$$\chi_{h,c} = (\text{dlog}_G(R_{c,h}) - k_{c,h})^{-1} \cdot (\text{dlog}_G(C_{c,h}^u) - c_{c,h}^u) \bmod q,$$

where $\text{dlog}_G(R_{c,h})$ and $\text{dlog}_G(C_{c,h}^u)$ denote the discrete logarithms to the base G of $R_{c,h}$ and $C_{c,h}^u$, respectively. However, given that $\chi_{h,c}$ is sampled by the functionality $\mathcal{F}_{\text{RVOLE}}$ uniformly at random from $\mathbb{Z}_q$, and is independent of the adversary's view, this occurs with probability $1/q$.

- Similarly, if $\hat{x}_{c,h} \cdot G \neq \hat{Q}_{c,h}$ or $c_{c,h}^v \cdot G \neq C_{c,h}^v$, then Eq. (B.8) is satisfied with probability at most $1/q$ over the uniformly random choice of $\chi_{h,c}$ by the functionality $\mathcal{F}_{\text{RVOLE}}$.

Overall, for any honest party P_h and corrupted party P_c , the probability that Eq. (B.7)–(B.8) are satisfied but Eq. (B.3)–(B.6) are not satisfied is at most $2/q$. A union bound over all $(n - |\mathcal{C}|) \cdot |\mathcal{C}| \leq (n/2)^2$ pairs of an honest party and a corrupted party guarantees that the statistical distance between the two experiments is at most $(n/2)^2 \cdot (2/q) = n^2/(2q)$. Thus, $\{\mathcal{H}_2(\alpha, \text{aux}, \kappa)\}_{\alpha, \text{aux} \in \{0,1\}^*, \kappa \in \mathbb{N}} \approx^s \{\mathcal{H}_1(\alpha, \text{aux}, \kappa)\}_{\alpha, \text{aux} \in \{0,1\}^*, \kappa \in \mathbb{N}}$.

- **Experiment $\mathcal{H}_3(\alpha, \text{aux}, \kappa)$.** This experiment is obtained from experiment $\mathcal{H}_2$ by modifying the computation of the values $\{\hat{x}_h\}_{h \in \mathcal{H}}$ by the honest parties in Step 2a of the protocol. Recall that, by the definition of the functionality $\mathcal{F}_{\text{rZero}[q,n]}$, in Step 2a of the protocol (and thus in experiment $\mathcal{H}_2$), each honest party P_h computes $\hat{x}_h = x_h + \zeta_h \bmod q$, where the values $\{\zeta_h\}_{h \in \mathcal{H}}$ are sampled by the functionality $\mathcal{F}_{\text{rZero}[q,n]}$ uniformly and independently subject to $\sum_{h \in \mathcal{H}} \zeta_h = -t \bmod q$ for the value $t \in \mathbb{Z}_q$ determined by the adversary $\mathcal{A}$. Now, in experiment $\mathcal{H}_3$, for every $h \in \mathcal{H} \setminus \{h^*\}$ the emulator uniformly samples $\hat{x}_h \in_R \mathbb{Z}_q$, and then sets $\hat{x}_{h^*} = x_{h^*} - \sum_{h \in \mathcal{H} \setminus \{h^*\}} (\hat{x}_h - x_h) - t \bmod q$.

Defining $\zeta_h = \hat{x}_h - x_h \bmod q$ for every $h \in \mathcal{H}$ in experiment $\mathcal{H}_3$, then in both experiments the values $\{\zeta_h\}_{h \in \mathcal{H}}$ are uniformly and independently distributed subject to $\sum_{h \in \mathcal{H}} \zeta_h = -t \bmod q$ for the value $t \in \mathbb{Z}_q$ determined by the adversary $\mathcal{A}$. Thus, $\{\mathcal{H}_3(\alpha, \text{aux}, \kappa)\}_{\alpha, \text{aux} \in \{0,1\}^*, \kappa \in \mathbb{N}} \equiv \{\mathcal{H}_2(\alpha, \text{aux}, \kappa)\}_{\alpha, \text{aux} \in \{0,1\}^*, \kappa \in \mathbb{N}}$.

- **Experiment $\mathcal{H}_4(\alpha, \text{aux}, \kappa)$.** This experiment is obtained from experiment $\mathcal{H}_3$ by modifying the computation of the value $\hat{Q}_{h^*}$ by the honest party P_{h^*} in Step 2a of the protocol. Recall that, in Step 2a of the protocol (and thus in experiment $\mathcal{H}_3$), the honest party P_{h^*} computes $\hat{Q}_{h^*} = \hat{x}_{h^*} \cdot G$. Now, in experiment $\mathcal{H}_4$, the emulator sets $\hat{Q}_{h^*} = Q - \sum_{h \in \mathcal{H} \setminus \{h^*\}} \hat{Q}_h - \sum_{c \in \mathcal{C}} (Q_c + \zeta_c \cdot G)$.

The assumption that the parties hold valid and consistent inputs guarantees that $Q = \sum_{i \in [n]} Q_i$ where $Q_i = x_i \cdot G$ for every $i \in [n]$. In addition, in both experiment $\mathcal{H}_3$ and experiment $\mathcal{H}_4$ it holds that $\sum_{c \in \mathcal{C}} \zeta_c = t \bmod q$ (where $t \in \mathbb{Z}_q$ is the value sent by the adversary $\mathcal{A}$ to the functionality $\mathcal{F}_{\text{rZero}[q,n]}$), $\hat{x}_{h^*} = x_{h^*} - \sum_{h \in \mathcal{H} \setminus \{h^*\}} (\hat{x}_h - x_h) - t \bmod q$, and $\hat{Q}_h = \hat{x}_h \cdot G$ for every $h \in \mathcal{H} \setminus \{h^*\}$. Therefore, in experiment $\mathcal{H}_4$ it holds that

$$\hat{Q}_{h^*} = Q - \sum_{h \in \mathcal{H} \setminus \{h^*\}} \hat{Q}_h - \sum_{c \in \mathcal{C}} (Q_c + \zeta_c \cdot G)$$

$$\begin{aligned}
&= \sum_{i \in [n]} x_i \cdot G - \sum_{h \in \mathcal{H} \setminus \{h^*\}} \hat{x}_h \cdot G - \sum_{c \in \mathcal{C}} x_c \cdot G - t \cdot G \\
&= x_{h^*} \cdot G - \sum_{h \in \mathcal{H} \setminus \{h^*\}} (\hat{x}_h - x_h) \cdot G - t \cdot G \\
&= \hat{x}_{h^*} \cdot G
\end{aligned}$$

as in experiment $\mathcal{H}_3$. Thus,

$$\{\mathcal{H}_4(\alpha, \text{aux}, \kappa)\}_{\alpha, \text{aux} \in \{0,1\}^*, \kappa \in \mathbb{N}} \equiv \{\mathcal{H}_3(\alpha, \text{aux}, \kappa)\}_{\alpha, \text{aux} \in \{0,1\}^*, \kappa \in \mathbb{N}}.$$

- **Experiment $\mathcal{H}_5(\alpha, \text{aux}, \kappa)$.** This experiment is obtained from experiment $\mathcal{H}_4$ by modifying the computation of the values $(C_{h,c}^u, C_{h,c}^v, \psi_{h,c})$ in Step 2(b)iii for every honest party P_h and corrupted party. Recall that, in Step 2(b)iii of the protocol (and thus in experiment $\mathcal{H}_4$), every honest party P_h computes

$$\begin{aligned}
C_{h,c}^u &= c_{h,c}^u \cdot G \\
C_{h,c}^v &= c_{h,c}^v \cdot G \\
\psi_{h,c} &= \phi_h - \chi_{h,c} \bmod q
\end{aligned}$$

for every $c \in \mathcal{C}$, where ϕ_h is uniformly sampled by P_h , and $\chi_{h,c}$ is uniformly sampled by the functionality $\mathcal{F}_{\text{RVOLE}}$ and sent to P_h . Now, in experiment $\mathcal{H}_5$, the emulator samples $\psi_{h,c} \in_R \mathbb{Z}_q$, and computes

$$\begin{aligned}
C_{h,c}^u &= \chi_{c,h} \cdot R_h - d_{c,h}^u \cdot G \\
C_{h,c}^v &= \chi_{c,h} \cdot \hat{Q}_h - d_{c,h}^v \cdot G,
\end{aligned}$$

where $\chi_{c,h}$, $d_{c,h}^u$ and $d_{c,h}^v$ are sent to the functionality $\mathcal{F}_{\text{RVOLE}}$ by the adversary $\mathcal{A}$.

By the definition of the functionality $\mathcal{F}_{\text{RVOLE}}$, it holds that

$$\begin{aligned}
c_{h,c}^u + d_{c,h}^u &= \chi_{c,h} \cdot k_h \bmod q \\
c_{h,c}^v + d_{c,h}^v &= \chi_{c,h} \cdot \hat{x}_h \bmod q.
\end{aligned}$$

Defining $\chi_{h,c} = \phi_h - \psi_{h,c}$ for every $h \in \mathcal{H}$ and $c \in \mathcal{C}$ in experiment $\mathcal{H}_5$, then in both experiments the values $\chi_{h,c}$ are sampled independently and uniformly. Furthermore, in both experiments it holds that

$$\begin{aligned}
c_{h,c}^u \cdot G &= (\chi_{c,h} \cdot k_h - d_{c,h}^u) \cdot G = \chi_{c,h} \cdot R_h - d_{c,h}^u \cdot G \\
c_{h,c}^v \cdot G &= (\chi_{c,h} \cdot \hat{x}_h - d_{c,h}^v) \cdot G = \chi_{c,h} \cdot \hat{Q}_h - d_{c,h}^v \cdot G.
\end{aligned}$$

Thus, $\{\mathcal{H}_5(\alpha, \text{aux}, \kappa)\}_{\alpha, \text{aux} \in \{0,1\}^*, \kappa \in \mathbb{N}} \equiv \{\mathcal{H}_4(\alpha, \text{aux}, \kappa)\}_{\alpha, \text{aux} \in \{0,1\}^*, \kappa \in \mathbb{N}}.$

- **Experiment $\mathcal{H}_6(\alpha, \text{aux}, \kappa)$.** This experiment is obtained from experiment $\mathcal{H}_5$ by modifying the distribution of k_{h^*} and the computation of R_{h^*} in Steps

1a–1b. Recall that, in Steps 1a–1b of the protocol (and thus in experiment $\mathcal{H}_5$), the honest party P_{h^*} samples $k_{h^*} \in_R \mathbb{Z}_q$ and computes $R_{h^*} = k_{h^*} \cdot G$. Now, in experiment $\mathcal{H}_6$, the emulator additionally emulates the execution of the functionality $\mathcal{F}_{\text{ECDSA}}$ on the input $\{(\text{sign}, \text{sid}, m, Q, Q_1, \dots, Q_n, x_i)\}_{i \in [n]}$. Let (sid, σ) denote the output of this execution, where $\sigma = (r, s)$, and $R = k \cdot G = (s^{-1} \cdot H(m)) \cdot G + (s^{-1} \cdot r) \cdot Q$ for the value $k \in_R \mathbb{Z}_q$ sampled by the emulated functionality $\mathcal{F}_{\text{ECDSA}}$. The emulator now sets

$$R_{h^*} = R - \sum_{h \in \mathcal{H} \setminus \{h^*\}} R_h - \sum_{c \in \mathcal{C}} R_{c, h^*}$$

when responding to the adversary on behalf of the functionality $\mathcal{F}_{\text{Com}}$. Note that, for emulating the second-message sent by P_{h^*} the value k_{h^*} (which has not yet been defined) is no longer needed given the modified manner in which the group elements $C_{h^*, c}^u$ and $C_{h^*, c}^v$ are computed starting in experiment $\mathcal{H}_5$. Then, given the values $\{k_{c, h^*}\}_{c \in \mathcal{C}}$ sent by $\mathcal{A}$ on behalf of the corrupted parties to $\mathcal{F}_{\text{RVOLE}[q, 2]}$, the emulator sets

$$k_{h^*} = k - \sum_{h \in \mathcal{H} \setminus \{h^*\}} k_h - \sum_{c \in \mathcal{C}} k_{c, h^*} \bmod q$$

and uses k_{h^*} for the remainder of the protocol's execution (i.e., for computing u_{h^*} and w_{h^*} as instructed by the protocol).

Both in experiment $\mathcal{H}_5$ and experiment $\mathcal{H}_6$ it holds that $R_{h^*} = k_{h^*} \cdot G$. In experiment $\mathcal{H}_5$ this follows since P_{h^*} explicitly sets $R_{h^*} = k_{h^*} \cdot G$. In experiment $\mathcal{H}_6$, recall that for every honest party P_h where $h \neq h^*$ it holds that $R_h = k_h \cdot G$ (as in the protocol since this was not modified), and for every corrupted party we are guaranteed that $k_{c, h^*} \cdot G = R_{c, h^*}$ starting in experiment $\mathcal{H}_2$ (and otherwise P_{h^*} aborts in both experiments). In addition, the value R computed by the emulated functionality $\mathcal{F}_{\text{ECDSA}}$ satisfies $R = k \cdot G$, and therefore in experiment $\mathcal{H}_6$ it holds that

$$\begin{aligned} R_{h^*} &= R - \sum_{h \in \mathcal{H} \setminus \{h^*\}} R_h - \sum_{c \in \mathcal{C}} R_{c, h^*} \\ &= \left(k - \sum_{h \in \mathcal{H} \setminus \{h^*\}} k_h - \sum_{c \in \mathcal{C}} k_{c, h^*} \right) \cdot G \\ &= k_{h^*} \cdot G \end{aligned}$$

Furthermore, in both experiments the value k_{h^*} is uniformly distributed, where in experiment $\mathcal{H}_5$ this follows from the definition of the protocol, and in experiment $\mathcal{H}_6$ this follows from the fact that k is uniformly distributed and independent of the values $\{k_h\}_{h \in \mathcal{H} \setminus \{h^*\}}$ and $\{k_{c, h^*}\}_{c \in \mathcal{C}}$ (note that, starting in experiment $\mathcal{H}_2$, the values $\{k_{c, h^*}\}_{c \in \mathcal{C}}$ are determined via the values $\{R_{c, h^*}\}_{c \in \mathcal{C}}$ since otherwise P_{h^*} aborts).

- **Experiment $\mathcal{H}_7(\alpha, \mathbf{aux}, \kappa)$.** This experiment is obtained from experiment $\mathcal{H}_6$ by modifying the computation of w_{h^*} by party P_{h^*} in Step 3c. Recall that, in Step 3c of the protocol (and thus in experiment $\mathcal{H}_6$), the honest party P_{h^*} sets

$$w_{h^*} = H(m) \cdot \phi_{h^*} + r \cdot v_{h^*} \bmod q.$$

Now, in experiment $\mathcal{H}_7$, given the values $\{(u_h, v_h, w_h)\}_{h \in \mathcal{H} \setminus \{h^*\}}$ computed by all other honest parties, the emulator first computes u_{h^*} as instructed in Step 3c of the protocol. Then, the emulator computes:

$$\begin{aligned} u_{\mathcal{H} \setminus \{h^*\}} &= \sum_{h \in \mathcal{H} \setminus \{h^*\}} u_h \bmod q \\ v_{\mathcal{H} \setminus \{h^*\}} &= \sum_{h \in \mathcal{H} \setminus \{h^*\}} v_h \bmod q \\ u_{\mathcal{C}} &= \sum_{c \in \mathcal{C}} \sum_{h \in \mathcal{H}} (k_{c,h^*} \cdot \psi_{h,c} + c_{c,h}^u + d_{c,h}^u) \\ &\quad + \left(\sum_{c \in \mathcal{C}} k_{c,h^*} \right) \cdot \left(\sum_{c \in \mathcal{C}} (\psi_{c,h^*} + \chi_{c,h^*}) \right) \bmod q \\ v_{\mathcal{C}} &= \sum_{c \in \mathcal{C}} \sum_{h \in \mathcal{H}} (\hat{x}_{c,h^*} \cdot \psi_{h,c} + c_{c,h}^v + d_{c,h}^v) \\ &\quad + \left(\sum_{c \in \mathcal{C}} \hat{x}_{c,h^*} \right) \cdot \left(\sum_{c \in \mathcal{C}} (\psi_{c,h^*} + \chi_{c,h^*}) \right) \bmod q \\ w_{\mathcal{H} \setminus \{h^*\}} &= H(m) \cdot \left(\sum_{h \in \mathcal{H} \setminus \{h^*\}} \phi_h \right) + r \cdot v_{\mathcal{H} \setminus \{h^*\}} \bmod q \\ w_{\mathcal{C}} &= H(m) \cdot \left(\sum_{c \in \mathcal{C}} (\psi_{c,h^*} + \chi_{c,h^*}) \right) + r \cdot v_{\mathcal{C}} \bmod q \\ w_{h^*} &= s \cdot (u_{h^*} + u_{\mathcal{H} \setminus \{h^*\}} + u_{\mathcal{C}}) - (w_{\mathcal{H} \setminus \{h^*\}} + w_{\mathcal{C}}) \bmod q \end{aligned}$$

We show that experiment $\mathcal{H}_7$ is identical to experiment $\mathcal{H}_6$. Recall that, in both experiments, it holds that

$$\begin{aligned} k_{h^*} &= k - \sum_{h \in \mathcal{H} \setminus \{h^*\}} k_h - \sum_{c \in \mathcal{C}} k_{c,h^*} \bmod q \\ R_h &= k_h \cdot G \text{ for every } h \in \mathcal{H} \setminus \{h^*\} \\ R_{c,h^*} &= k_{c,h^*} \cdot G \text{ for every } c \in \mathcal{C}, \end{aligned}$$

where $k \in \mathbb{Z}_q$ is the value sampled by the emulated functionality $\mathcal{F}_{\text{ECDSA}}$. In addition, in both executions, if P_{h^*} does not abort then for every honest party P_h , where $h \in \mathcal{H}$, it holds that $\sum_{c \in \mathcal{C}} \hat{Q}_{c,h} = \sum_{c \in \mathcal{C}} (Q_c + \zeta_c \cdot G)$ (recall that this verification is performed in Step 3a of protocol and in Steps 12 and 13 of the simulator). Thus, the adversary's inputs to the VOLE functionality with respect

to each individual honest party are consistent with Q , and since party P_{h^*} locally computes $R = R_{h^*} + \sum_{h \in \mathcal{H} \setminus \{h^*\}} R_h - \sum_{c \in \mathcal{C}} R_{c,h^*}$ (by the definition of R_{h^*} in experiment $\mathcal{H}_6$) whose x -coordinate is r , then in both experiments party P_{h^*} may either abort or output the signature $\sigma = (r, s)$ computed relative to Q using R by the emulated functionality $\mathcal{F}_{\text{ECDSA}}$, but may not output any other signature $\sigma' \neq \sigma$.³ Furthermore, given the values $\{(u_h, w_h)\}_{h \in \mathcal{H} \setminus \{h^*\}}$ computed by all other honest parties, and given the value u_{h^*} computed by P_{h^*} , if all corrupted parties followed the protocol (i.e., behaved consistently) and provided their expected $\{(u_c, w_c)\}_{c \in \mathcal{C}}$ values, then w_{h^*} is uniquely determined via the constraint

$$s = \left(\sum_{j \in [n]} w_j \right) \cdot \left(\sum_{j \in [n]} u_j \right)^{-1} \bmod q .$$

That is, w_{h^*} is uniquely determined as

$$\begin{aligned} w_{h^*} &= s \cdot \left(\sum_{j \in [n]} u_j \right) - \sum_{j \in [n] \setminus \{h^*\}} w_j \bmod q \\ &= s \cdot \left(u_{h^*} + \sum_{h \in \mathcal{H} \setminus \{h^*\}} u_h + \sum_{c \in \mathcal{C}} u_c \right) \\ &\quad - \left(\sum_{h \in \mathcal{H} \setminus \{h^*\}} w_h + \sum_{c \in \mathcal{C}} w_c \right) \bmod q \end{aligned} \tag{B.9}$$

In experiment $\mathcal{H}_7$, given the values $\{(u_h, v_h, w_h)\}_{h \in \mathcal{H} \setminus \{h^*\}}$, the emulator computes their corresponding sums:

$$\begin{aligned} u_{\mathcal{H} \setminus \{h^*\}} &= \sum_{h \in \mathcal{H} \setminus \{h^*\}} u_h \bmod q \\ w_{\mathcal{H} \setminus \{h^*\}} &= \mathbf{H}(m) \cdot \left(\sum_{h \in \mathcal{H} \setminus \{h^*\}} \phi_h \right) + r \cdot v_{\mathcal{H} \setminus \{h^*\}} = \sum_{h \in \mathcal{H} \setminus \{h^*\}} w_h \bmod q . \end{aligned}$$

Thus, Eq. (B.9) is equivalent to

$$w_{h^*} = s \cdot \left(u_{h^*} + u_{\mathcal{H} \setminus \{h^*\}} + \sum_{c \in \mathcal{C}} u_c \right) - \left(w_{\mathcal{H} \setminus \{h^*\}} + \sum_{c \in \mathcal{C}} w_c \right) \bmod q .$$

³ Specifically, note that since k uniquely determines r , the only two signatures that pass the ECDSA verification in Step 4e of the protocol are (r, s) and $(r, -s)$. Furthermore, the verification $R = (s^{-1} \cdot \mathbf{H}(m)) \cdot G + (s^{-1} \cdot r) \cdot Q$ in Step 4c of the protocol rules out $(r, -s)$ as long as $R \neq -R$ (which occurs with all but a statistically negligible probability) since $((-s)^{-1} \cdot \mathbf{H}(m)) \cdot G + ((-s)^{-1} \cdot r) \cdot Q = -R$.

Therefore, for showing that experiments $\mathcal{H}_6$ and $\mathcal{H}_7$ are identical it suffices to show that the values $u_{\mathcal{C}}$ and $w_{\mathcal{C}}$, as computed by the emulator in experiment $\mathcal{H}_7$, satisfy

$$u_{\mathcal{C}} = \sum_{c \in \mathcal{C}} u_c \bmod q \quad (\text{B.10})$$

$$w_{\mathcal{C}} = \sum_{c \in \mathcal{C}} w_c \bmod q \quad (\text{B.11})$$

where $\{(u_c, w_c)\}_{c \in \mathcal{C}}$ are the expected values provided by the corrupted parties to P_{h^*} assuming that they followed the protocol. In what follows we first establish Eq. (B.10) and then establish Eq. (B.11).

For establishing Eq. (B.10), note that from the perspective of P_{h^*} , if a corrupted party P_c would behave honestly they would compute u_c as defined by the protocol. That is,

$$u_c = k_c \cdot \left(\phi_c + \sum_{j \in [n] \setminus \{c\}} \psi_{j,c} \right) + \sum_{j \in [n] \setminus \{c\}} (c_{c,j}^u + d_{c,j}^u) \bmod q .$$

Thus,

$$\begin{aligned} \sum_{c \in \mathcal{C}} u_c &= \sum_{c \in \mathcal{C}} k_c \cdot \left(\phi_c + \sum_{j \in [n] \setminus \{c\}} \psi_{j,c} \right) + \sum_{c \in \mathcal{C}} \sum_{j \in [n] \setminus \{c\}} (c_{c,j}^u + d_{c,j}^u) \bmod q \\ &= \sum_{c \in \mathcal{C}} k_c \cdot \phi_c + \sum_{c \in \mathcal{C}} \sum_{c' \in \mathcal{C} \setminus \{c\}} (k_c \cdot \psi_{c',c} + c_{c,c'}^u + d_{c,c'}^u) \\ &\quad + \sum_{c \in \mathcal{C}} \sum_{h \in \mathcal{H}} (k_c \cdot \psi_{h,c} + c_{c,h}^u + d_{c,h}^u) \bmod q . \end{aligned}$$

By the definition of the functionality $\mathcal{F}_{\text{RVOLE}}$, for every $c, c' \in \mathcal{C}$ it holds that

$$c_{c,c'}^u = \chi_{c',c} \cdot k_c - d_{c',c}^u \bmod q ,$$

where $\psi_{c',c} + \chi_{c',c} = \phi_{c'}$ mod q , and therefore

$$\begin{aligned} \sum_{c \in \mathcal{C}} u_c &= \sum_{c \in \mathcal{C}} k_c \cdot \phi_c + \sum_{c \in \mathcal{C}} \sum_{c' \in \mathcal{C} \setminus \{c\}} (k_c \cdot \phi_{c'} - k_c \cdot \chi_{c',c} + \chi_{c',c} \cdot k_c - d_{c',c}^u + d_{c,c'}^u) \\ &\quad + \sum_{c \in \mathcal{C}} \sum_{h \in \mathcal{H}} (k_c \cdot \psi_{h,c} + c_{c,h}^u + d_{c,h}^u) \bmod q \\ &= \left(\sum_{c \in \mathcal{C}} k_c \right) \cdot \left(\sum_{c \in \mathcal{C}} \phi_c \right) + \sum_{c \in \mathcal{C}} \sum_{c' \in \mathcal{C} \setminus \{c\}} (d_{c,c'}^u - d_{c',c}^u) \\ &\quad + \sum_{c \in \mathcal{C}} \sum_{h \in \mathcal{H}} (k_c \cdot \psi_{h,c} + c_{c,h}^u + d_{c,h}^u) \bmod q \end{aligned}$$

$$= \left(\sum_{c \in \mathcal{C}} k_c \right) \cdot \left(\sum_{c \in \mathcal{C}} \phi_c \right) + \sum_{c \in \mathcal{C}} \sum_{h \in \mathcal{H}} (k_c \cdot \psi_{h,c} + c_{c,h}^u + d_{c,h}^u) \bmod q, \quad (\text{B.12})$$

where Eq. (B.12) follows from the fact that $\sum_{c \in \mathcal{C}} \sum_{c' \in \mathcal{C} \setminus \{c\}} (d_{c,c'}^u - d_{c',c}^u) = 0$. Setting $k_c \leftarrow k_{c,h^*}$ and $\phi_c \leftarrow \psi_{c,h^*} + \chi_{c,h^*} \bmod q$ for every $c \in \mathcal{C}$, as defined via the interaction between P_{h^*} and P_c , we obtain

$$\begin{aligned} \sum_{c \in \mathcal{C}} u_c &= \left(\sum_{c \in \mathcal{C}} k_{c,h^*} \right) \cdot \left(\sum_{c \in \mathcal{C}} (\psi_{c,h^*} + \chi_{c,h^*}) \right) \\ &\quad + \sum_{c \in \mathcal{C}} \sum_{h \in \mathcal{H}} (k_{c,h^*} \cdot \psi_{h,c} + c_{c,h}^u + d_{c,h}^u) \bmod q \\ &= u_{\mathcal{C}} \bmod q \end{aligned}$$

and thus Eq. (B.10) holds.

Following an identical algebraic sequence as above, substituting k_c with $\hat{x}_c$ and the respective RVOLE outputs, we now establish Eq. (B.11). From the perspective of P_{h^*} , if a corrupted party P_c would behave honestly they would compute v_c as defined by the protocol. That is,

$$v_c = \hat{x}_c \cdot \left(\phi_c + \sum_{j \in [n] \setminus \{c\}} \psi_{j,c} \right) + \sum_{j \in [n] \setminus \{c\}} (c_{c,j}^v + d_{c,j}^v) \bmod q.$$

Thus,

$$\begin{aligned} \sum_{c \in \mathcal{C}} v_c &= \sum_{c \in \mathcal{C}} \hat{x}_c \cdot \left(\phi_c + \sum_{j \in [n] \setminus \{c\}} \psi_{j,c} \right) + \sum_{c \in \mathcal{C}} \sum_{j \in [n] \setminus \{c\}} (c_{c,j}^v + d_{c,j}^v) \bmod q \\ &= \sum_{c \in \mathcal{C}} \hat{x}_c \cdot \phi_c + \sum_{c \in \mathcal{C}} \sum_{c' \in \mathcal{C} \setminus \{c\}} (\hat{x}_c \cdot \psi_{c',c} + c_{c,c'}^v + d_{c,c'}^v) \\ &\quad + \sum_{c \in \mathcal{C}} \sum_{h \in \mathcal{H}} (\hat{x}_c \cdot \psi_{h,c} + c_{c,h}^v + d_{c,h}^v) \bmod q \end{aligned}$$

By the definition of the functionality $\mathcal{F}_{\text{RVOLE}}$, for every $c, c' \in \mathcal{C}$ it holds that

$$c_{c,c'}^v = \chi_{c',c} \cdot \hat{x}_c - d_{c',c}^v \bmod q,$$

where $\psi_{c',c} + \chi_{c',c} = \phi_{c'} \bmod q$, and therefore

$$\begin{aligned} \sum_{c \in \mathcal{C}} v_c &= \sum_{c \in \mathcal{C}} \hat{x}_c \cdot \phi_c + \sum_{c \in \mathcal{C}} \sum_{c' \in \mathcal{C} \setminus \{c\}} (\hat{x}_c \cdot \phi_{c'} - \hat{x}_c \cdot \chi_{c',c} + \chi_{c',c} \cdot \hat{x}_c - d_{c',c}^v + d_{c,c'}^v) \\ &\quad + \sum_{c \in \mathcal{C}} \sum_{h \in \mathcal{H}} (\hat{x}_c \cdot \psi_{h,c} + c_{c,h}^v + d_{c,h}^v) \bmod q \\ &= \left(\sum_{c \in \mathcal{C}} \hat{x}_c \right) \cdot \left(\sum_{c \in \mathcal{C}} \phi_c \right) + \sum_{c \in \mathcal{C}} \sum_{c' \in \mathcal{C} \setminus \{c\}} (d_{c,c'}^v - d_{c',c}^v) \end{aligned}$$

$$\begin{aligned}
& + \sum_{c \in \mathcal{C}} \sum_{h \in \mathcal{H}} (\hat{x}_c \cdot \psi_{h,c} + c_{c,h}^v + d_{c,h}^v) \bmod q \\
& = \left(\sum_{c \in \mathcal{C}} \hat{x}_c \right) \cdot \left(\sum_{c \in \mathcal{C}} \phi_c \right) + \sum_{c \in \mathcal{C}} \sum_{h \in \mathcal{H}} (\hat{x}_c \cdot \psi_{h,c} + c_{c,h}^v + d_{c,h}^v) \bmod q, \quad (\text{B.13})
\end{aligned}$$

where Eq. (B.13) follows from the fact that $\sum_{c \in \mathcal{C}} \sum_{c' \in \mathcal{C} \setminus \{c\}} (d_{c,c'}^v - d_{c',c}^v) = 0$. Setting $\hat{x}_c \leftarrow \hat{x}_{c,h^*}$ and $\phi_c \leftarrow \psi_{c,h^*} + \chi_{c,h^*} \bmod q$ for every $c \in \mathcal{C}$, as defined via the interaction between P_{h^*} and P_c , we obtain

$$\begin{aligned}
\sum_{c \in \mathcal{C}} v_c & = \left(\sum_{c \in \mathcal{C}} \hat{x}_{c,h^*} \right) \cdot \left(\sum_{c \in \mathcal{C}} (\psi_{c,h^*} + \chi_{c,h^*}) \right) \\
& \quad + \sum_{c \in \mathcal{C}} \sum_{h \in \mathcal{H}} (\hat{x}_{c,h^*} \cdot \psi_{h,c} + c_{c,h}^v + d_{c,h}^v) \bmod q \\
& = v_{\mathcal{C}} \bmod q.
\end{aligned}$$

Therefore,

$$\begin{aligned}
\sum_{c \in \mathcal{C}} w_c & = \sum_{c \in \mathcal{C}} (H(m) \cdot \phi_c + r \cdot v_c) \bmod q \\
& = H(m) \cdot \left(\sum_{c \in \mathcal{C}} (\psi_{c,h^*} + \chi_{c,h^*}) \right) + r \cdot v_{\mathcal{C}} \bmod q \\
& = w_{\mathcal{C}} \bmod q
\end{aligned}$$

and thus Eq. (B.11) holds.

- **Experiment $\mathcal{H}_8(\alpha, \text{aux}, \kappa)$.** This experiment is obtained from experiment $\mathcal{H}_7$ by modifying the generation of the value u_{h^*} on behalf of the honest party P_{h^*} . Instead of generating this value by following Step 3c of the protocol, in experiment $\mathcal{H}_8$ the emulator samples it uniformly from $\mathbb{Z}_q$.

We show that experiments $\mathcal{H}_8$ and $\mathcal{H}_7$ are statistically indistinguishable. In experiment $\mathcal{H}_7$, the honest party P_{h^*} computes

$$u_{h^*} = k_{h^*} \cdot \left(\phi_{h^*} + \sum_{j \in [n] \setminus \{h^*\}} \psi_{j,h^*} \right) + \sum_{j \in [n] \setminus \{h^*\}} (c_{h^*,j}^u + d_{h^*,j}^u) \bmod q,$$

where k_{h^*} is information-theoretically determined via R_{h^*} , but ϕ_{h^*} is uniformly distributed in $\mathbb{Z}_q$ and independent of the adversary's view. Specifically, although the adversary's view includes the values $\psi_{h^*,c}$ for every corrupted party $c \in \mathcal{C}$, it holds that $\psi_{h^*,c} = \phi_{h^*} - \chi_{h^*,c} \bmod q$, where the value $\chi_{h^*,c}$ is sampled uniformly from $\mathbb{Z}_q$ by the functionality $\mathcal{F}_{\text{RVOLE}}$. Therefore, as long as $k_{h^*} \neq 0 \bmod q$, then u_{h^*} is uniformly distributed and independent of all other values included in the adversary's view in experiment $\mathcal{H}_7$. Recalling that k_{h^*} is uniformly distributed, since

$$k_{h^*} = k - \sum_{h \in \mathcal{H} \setminus \{h^*\}} k_h - \sum_{c \in \mathcal{C}} k_{c,h^*} \bmod q,$$

where $k \in_R \mathbb{Z}_q$ is sampled by the emulated functionality $\mathcal{F}_{\text{ECDSA}}$, it holds that $k_{h^*} \neq 0 \bmod q$ with probability $1 - 1/q$. Thus,

$$\{\mathcal{H}_7(\alpha, \mathbf{aux}, \kappa)\}_{\alpha, \mathbf{aux} \in \{0,1\}^*, \kappa \in \mathbb{N}} \approx^s \{\mathcal{H}_8(\alpha, \mathbf{aux}, \kappa)\}_{\alpha, \mathbf{aux} \in \{0,1\}^*, \kappa \in \mathbb{N}}.$$

- **Experiment $\mathcal{H}_9(\alpha, \mathbf{aux}, \kappa)$.** This experiment is obtained from experiment $\mathcal{H}_8$ by modifying the generation of the values $\{(u_h, v_h)\}_{h \in \mathcal{H} \setminus \{h^*\}}$ on behalf of the honest parties excluding P_{h^*} . Instead of generating these values by following Step 3c of the protocol, in experiment $\mathcal{H}_9$ the emulator samples them uniformly and independently from $\mathbb{Z}_q$.

We show that experiments $\mathcal{H}_9$ and $\mathcal{H}_8$ are identically distributed. Consider any honest party P_h where $h \in \mathcal{H} \setminus \{h^*\}$, then in experiment $\mathcal{H}_8$ the party P_h computes

$$\begin{aligned} u_h &= k_h \cdot \left(\phi_h + \sum_{j \in [n] \setminus \{h\}} \psi_{j,h} \right) + \sum_{j \in [n] \setminus \{h\}} (c_{h,j}^u + d_{h,j}^u) \bmod q \\ v_h &= \hat{x}_h \cdot \left(\phi_h + \sum_{j \in [n] \setminus \{h\}} \psi_{j,h} \right) + \sum_{j \in [n] \setminus \{h\}} (c_{h,j}^v + d_{h,j}^v) \bmod q. \end{aligned}$$

Since $h^* \in \mathcal{H}$ and $h \neq h^*$, the sums include the following terms corresponding to the interactions with the honest party P_{h^*} :

$$\begin{aligned} \Delta_{h,h^*}^u &= c_{h,h^*}^u + d_{h,h^*}^u \\ \Delta_{h,h^*}^v &= c_{h,h^*}^v + d_{h,h^*}^v. \end{aligned}$$

The values $(d_{h,h^*}^u, d_{h,h^*}^v)$ are the outputs received by P_h (acting as Alice) in the RVOLE execution where P_{h^*} acts as Bob. By the definition of $\mathcal{F}_{\text{RVOLE}[q,2]}$, this output vector $\mathbf{a}_{\text{out}} \in \mathbb{Z}_q^2$ is sampled uniformly at random by the functionality. Similarly, the values $(c_{h,h^*}^u, c_{h,h^*}^v)$ are the outputs received by P_h (acting as Bob) in the separate RVOLE execution where P_{h^*} acts as Alice. By the definition of $\mathcal{F}_{\text{RVOLE}[q,2]}$, this output vector $\mathbf{b}_{\text{out}} \in \mathbb{Z}_q^2$ is also sampled uniformly at random.

Since the adversary does not corrupt P_h or P_{h^*} , these internal random masks are information-theoretically hidden from the adversary's view. Furthermore, because the functionality samples the masks from $\mathbb{Z}_q^2$, the mask applied to the first component (u) is independent of the mask applied to the second component (v). Consequently, the pair $(\Delta_{h,h^*}^u, \Delta_{h,h^*}^v)$ is uniformly distributed in $\mathbb{Z}_q^2$ and independent of the adversary's view. These terms act as a one-time pad for the sums u_h and v_h , rendering the pair (u_h, v_h) uniformly distributed in $\mathbb{Z}_q^2$.

- **Experiment $\mathcal{H}_{10}(\alpha, \mathbf{aux}, \kappa)$.** This experiment is obtained from experiment $\mathcal{H}_9$ by modifying the manner in which the honest parties compute their output. In the protocol (and thus in experiment $\mathcal{H}_9$), recall that each honest party P_h produces their output as follows:

- Upon receiving $\{(w_{h'}, u_{h'})\}_{h' \in \mathcal{H} \setminus \{h\}}$ and $\{(w_{c,h}, u_{c,h})\}_{c \in \mathcal{C}}$, compute

$$s_h = \left(\sum_{h' \in \mathcal{H}} w_{h'} + \sum_{c \in \mathcal{C}} w_{c,h} \right) \cdot \left(\sum_{h' \in \mathcal{H}} u_{h'} + \sum_{c \in \mathcal{C}} u_{c,h} \right)^{-1} \bmod q .$$

- Let $\sigma_h = (r_h, s_h)$, where $r_h \in \mathbb{Z}_q$ is the x -coordinate modulo q of $\tilde{R}_h = \sum_{h' \in \mathcal{H}} R_{h'} + \sum_{c \in \mathcal{C}} R_{c,h}$.
 - If σ_h is a valid ECDSA signature on the message m with respect to the public key Q and hash function H then output σ_h and otherwise abort.
- Now, in experiment $\mathcal{H}_{10}$, the emulator produces an output on behalf of every honest party P_h as follows:
- Denote by $\sigma = (r, s)$ the signature computed by the emulated ideal functionality $\mathcal{F}_{\text{ECDSA}}$.
 - Upon receiving $\{(w_{h'}, u_{h'})\}_{h' \in \mathcal{H} \setminus \{h\}}$ and $\{(w_{c,h}, u_{c,h})\}_{c \in \mathcal{C}}$, if

$$s = \left(\sum_{h' \in \mathcal{H}} w_{h'} + \sum_{c \in \mathcal{C}} w_{c,h} \right) \cdot \left(\sum_{h' \in \mathcal{H}} u_{h'} + \sum_{c \in \mathcal{C}} u_{c,h} \right)^{-1} \bmod q \quad (\text{B.14})$$

then output σ and otherwise abort.

We show that experiments $\mathcal{H}_9$ and $\mathcal{H}_{10}$ are identical. Note that, in both experiments, the honest parties verify that they have all computed the same value of $\tilde{R}_h$. Recall that, starting in experiment $\mathcal{H}_6$, the emulator sets

$$R_{h^*} = R - \sum_{h \in \mathcal{H} \setminus \{h^*\}} R_h - \sum_{c \in \mathcal{C}} R_{c,h^*} ,$$

and therefore $\tilde{R}_{h^*} = R$. Thus, if some honest party P_h computes $\tilde{R}_h \neq R$ then all honest parties abort in both experiments. Therefore, from now on we consider the case in which all honest parties computed the same $\tilde{R}_h = R$ (and therefore the same $r_h = r \in \mathbb{Z}_q$).

Having fixed R and $r \in \mathbb{Z}_q$, recall that the value s provided by the emulated ideal functionality $\mathcal{F}_{\text{ECDSA}}$ satisfies $s = k^{-1} \cdot (H(m) + x \cdot r) \bmod q$. By the uniqueness of ECDSA signatures up to the sign of s , the only two signatures that pass the ECDSA verification (performed in both experiments) are (r, s) and $(r, -s)$. However, if $s_h = -s$, then as long as $R \neq -R$ the signature $(r, -s)$ is ruled out in experiment $\mathcal{H}_9$ since $((-s)^{-1} \cdot H(m)) \cdot G + ((-s)^{-1} \cdot r) \cdot Q = -R$. Thus, for every honest party P_h , if the value s_h computed in experiment $\mathcal{H}_9$ satisfies $s_h = s$ then P_h outputs $\sigma = (r, s)$ and aborts otherwise, and this is the exact same verification performed by the emulator in experiment $\mathcal{H}_{10}$.

- **Experiment $\mathcal{H}_{11}(\alpha, \text{aux}, \kappa)$.** This experiment is an ideal-model computation of the functionality $\mathcal{F}_{\text{ECDSA}}$ with the simulator $\mathcal{S}$.

Note that in experiment $\mathcal{H}_{10}$ the emulator implements all steps of the simulator $\mathcal{S}$, and that the honest parties are dummy parties that send their provided inputs

to the functionality $\mathcal{F}_{\text{ECDSA}}$ and receive their outputs from the functionality $\mathcal{F}_{\text{ECDSA}}$. Thus,

$$\{\mathcal{H}_{10}(\alpha, \text{aux}, \kappa)\}_{\alpha, \text{aux} \in \{0,1\}^*, \kappa \in \mathbb{N}} \equiv \{\text{IDEAL}_{\mathcal{F}_{\text{ECDSA}}, S(\text{aux}), \mathcal{C}}(\alpha, \kappa)\}_{\alpha, \text{aux} \in \{0,1\}^*, \kappa \in \mathbb{N}}.$$

■

Adapting the proof to the asynchronous setting. As noted above, our proof of Theorem 5.2 is presented in a synchronous communication model for simplicity. However, the protocol and its security proof extend straightforwardly to an asynchronous network where the adversary schedules the delivery of messages, as considered by Doerner et al. [DKLs23, DKLs24]. To adapt our simulator $\mathcal{S}$ for an asynchronous setting, the following modifications are required:

1. The simulator maintains the state of each honest party independently.
2. When the adversary delivers all necessary incoming messages for round i to an honest party P_h , the simulator responds with a message on behalf of P_h for round $i + 1$.
3. The special honest party P_{h^*} (used to embed the ideal signature) is not fixed in advance. Instead, it is defined dynamically as the *last* honest party for whom the adversary completes the second round.

Since the simulator does not rely on any specific timing assumptions beyond the dependencies inherent in the protocol's flow, all of the above indistinguishability arguments hold in the asynchronous setting as well.

C Statistical Security of the Refined Protocol with Relaxed Key Generation

In this section we establish the statistical UC security of the refined protocol under the specific framework of Doerner et al. [DKLs23, DKLs24] that avoids extracting the adversarial key shares during the setup phase. In contrast to the decoupled analysis in Section 5, which assumes valid and consistent inputs including individual public key shares $Q_i = x_i \cdot G$, the analysis in this section does not require individual public key shares as inputs. This aligns with the relaxed key-generation functionality of Doerner et al. which provides each honest party P_h only with the global public key Q and their secret share x_h , without producing or distributing individual public key shares.

C.1 The Relaxed Key Generation Functionality

The relaxed key-generation functionality $\mathcal{F}_{\text{RelaxedKeyGen}}$ models a setup process where the ideal functionality does not explicitly extract the secret key shares of the corrupted parties. As originally defined by Doerner et al. [DKLs23, DKLs24], the functionality is parameterized for the general t -out-of- n threshold setting and therefore relies on Shamir secret sharing. Since we present our analysis in the full-threshold (n -out-of- n) setting, we naturally simplify their functionality to an equivalent additive variant, as detailed below following the formal description.

The Functionality $\mathcal{F}_{\text{RelaxedKeyGen}[\mathcal{G}, n]}$

- **Public parameters:**
 - Description $\mathcal{G}$ of a cyclic group $\mathbb{G}$ of prime order q that is generated by $G \in \mathbb{G}$
 - Number of parties $n \geq 2$
- **Parties:** $P_1, \dots, P_n$ with distinct party identifiers $\text{pid}_1, \dots, \text{pid}_n$. Let $\mathcal{C} \subseteq [n]$ denote the subset of corrupted parties and $\mathcal{H} = [n] \setminus \mathcal{C}$ denote the subset of honest parties.
- **Computation:**
 1. Upon receiving $(\text{keygen}, \text{sid})$ from some party P_i , where sid is fresh and $i \in [n]$, send $(\text{keygen-req}, \text{sid}, i)$ to the adversary $\mathcal{S}$. Upon receiving $(\text{keygen}, \text{sid})$ from all parties, proceed as follows:
 - (a) Receive $(\text{adv-shares}, \text{sid}, \{\tilde{Q}_c\}_{c \in \mathcal{C}})$ from $\mathcal{S}$.
 - (b) For every $h \in \mathcal{H}$, sample $x_h \in_R \mathbb{Z}_q$ and compute $Q_h \leftarrow x_h \cdot G$.
 - (c) For every $c \in \mathcal{C}$, sample $\hat{x}_c \in_R \mathbb{Z}_q$ and compute $Q_c \leftarrow \tilde{Q}_c + \hat{x}_c \cdot G$.
 - (d) Compute $Q \leftarrow \sum_{i \in [n]} Q_i$.
 - (e) Send $(\text{honest-shares}, \text{sid}, Q, \{Q_h\}_{h \in \mathcal{H}}, \{\hat{x}_c\}_{c \in \mathcal{C}})$ to $\mathcal{S}$.
 2. Upon receiving $(\text{release}, \text{sid}, h)$ from $\mathcal{S}$ for some $h \in \mathcal{H}$, send $(\text{key-pair}, \text{sid}, Q, x_h)$ to party P_h . If $(\text{abort}, \text{sid}, h)$ is received instead, send $(\text{abort}, \text{sid})$ to party P_h .

Our presentation of $\mathcal{F}_{\text{RelaxedKeyGen}}$ reflects two natural simplifications that arise when specializing the original DKLS23 functionality from the general t -out-of- n threshold setting to the full-threshold (n -out-of- n) setting. First, in the original functionality, the adversary supplies scalar values $\{\tilde{p}(h)\}_{h \in \mathcal{H}}$ representing the corrupted parties' additive contributions to the honest parties' Shamir shares. These values are needed in the general setting to verify that the adversary's contributions form a polynomial of the correct degree $t - 1$ over $\mathbb{G}$. In the full-threshold setting, however, any set of n values trivially determines a polynomial of degree $n - 1$, and therefore this degree check is vacuously satisfied. Consequently, the adversary's scalar contributions to the honest parties' shares serve no purpose: the functionality adds independent uniform shifts to all shares, making any adversarial contribution to the honest parties' shares information-theoretically irrelevant. We therefore omit these values from the above functionality. Second, Shamir shares reduce to additive shares when $t = n$, and accordingly we replace the polynomial notation with simplified additive notation throughout.

C.2 The Monolithic ECDSA Functionality

To compose our refined protocol natively with the relaxed key-generation functionality defined above, we must also adopt the specific ideal signing functionality utilized by Doerner et al. [DKLS23, DKLS24]. Unlike the decoupled functionality presented in Section 5, which takes the secret key shares x_i as explicit inputs, the native DKLS23 formulation is monolithic. It consists of a setup phase that internally samples the global key pair (x, Q) and a signing phase that uses the

internally stored secret key x to generate signatures. Consequently, the functionality never requests or receives secret key shares from the parties. Below, we present this monolithic functionality, adapted from its original t -out-of- n specification to our full-threshold (n -out-of- n) setting.

The Monolithic ECDSA Functionality $\mathcal{F}_{\text{mECDSA}}$

- **Public parameters:**
 - Description $\mathcal{G}$ of a cyclic group $\mathbb{G}$ of prime order q that is generated by $G \in \mathbb{G}$
 - Number of parties $n \geq 2$
 - ECDSA hash function H
- **Parties:** $P_1, \dots, P_n$ with distinct party identifiers $\text{pid}_1, \dots, \text{pid}_n$. Let $\mathcal{C} \subseteq [n]$ denote the subset of corrupted parties and $\mathcal{H} = [n] \setminus \mathcal{C}$ denote the subset of honest parties.
- **Setup Phase:**
 1. Upon receiving $(\text{init}, \text{sid})$ from some party P_i , where sid is fresh and $i \in [n]$, send $(\text{init-req}, \text{sid}, i)$ to the adversary $\mathcal{S}$. Upon receiving $(\text{init}, \text{sid})$ from all parties, proceed as follows:
 - (a) Sample the global secret key $x \in_R \mathbb{Z}_q$.
 - (b) Compute the global public key $Q \leftarrow x \cdot G$.
 - (c) Store $(\text{secret-key}, \text{sid}, x)$ and send $(\text{public-key}, \text{sid}, Q)$ to the adversary $\mathcal{S}$.
 2. Upon receiving $(\text{release}, \text{sid}, i)$ from $\mathcal{S}$ for some $i \in [n]$, send $(\text{public-key}, \text{sid}, Q)$ to party P_i . If $(\text{abort}, \text{sid}, i)$ is received instead, send $(\text{abort}, \text{sid})$ to P_i .
- **Signing Phase:**
 1. Upon receiving $(\text{sign}, \text{sid}, \text{sigid}, m_i)$ from some party P_i , if sigid is fresh and $(\text{public-key}, \text{sid}, Q)$ was previously delivered to P_i , send $(\text{sig-req}, \text{sid}, \text{sigid}, i, m_i)$ to the adversary $\mathcal{S}$.
 2. Upon receiving $(\text{sign}, \text{sid}, \text{sigid}, m_i)$ from all parties $P_1, \dots, P_n$, proceed as follows:
 - (a) If there exist $i, j \in [n]$ such that $H(m_i) \neq H(m_j)$, send $(\text{failure}, \text{sid}, \text{sigid})$ to all parties and ignore further messages for sigid .
 - (b) Sample $k \in_R \mathbb{Z}_q$, compute $R \leftarrow k \cdot G$, and let r denote the x -coordinate of R modulo q .
 - (c) Compute $s \leftarrow k^{-1} \cdot (H(m_1) + x \cdot r) \bmod q$.
 - (d) Let $\sigma \leftarrow (r, s)$, and store $(\text{signature}, \text{sid}, \text{sigid}, \sigma)$.
 3. Upon receiving $(\text{fail}, \text{sid}, \text{sigid}, i)$ from $\mathcal{S}$ for some $i \in [n]$, if a signature is stored for sigid , send $(\text{failure}, \text{sid}, \text{sigid})$ to party P_i and ignore any future $(\text{fail}, \text{sid}, \text{sigid}, i)$ or $(\text{proceed}, \text{sid}, \text{sigid}, i)$ messages.
 4. Upon receiving $(\text{proceed}, \text{sid}, \text{sigid}, i)$ from $\mathcal{S}$ for some $i \in [n]$, if a signature σ is stored for sigid , send $(\text{signature}, \text{sid}, \text{sigid}, \sigma)$ to party P_i and ignore any future $(\text{fail}, \text{sid}, \text{sigid}, i)$ or $(\text{proceed}, \text{sid}, \text{sigid}, i)$ messages.
 5. Once all parties have received an output for sigid , ignore all future messages with this sigid .

As with $\mathcal{F}_{\text{RelaxedKeyGen}}$, our presentation of $\mathcal{F}_{\text{mECDSA}}$ specializes the original DKLs23 functionality to the full-threshold setting. The only structural difference is that the signing phase requires all n parties to participate (rather than any subset of size t). The interface with the adversary, including the ability to instruct

per-party proceed or fail when corrupt parties are present, is preserved exactly as in the original.

C.3 The Monolithic Refined Protocol

To securely realize the monolithic $\mathcal{F}_{\text{mECDSA}}$ functionality in the $\mathcal{F}_{\text{RelaxedKeyGen}}$ -hybrid model, we must formally combine the key-generation and signing phases into a single monolithic protocol, Π_{mECDSA} . Compared to the decoupled refined protocol Π_{ECDSA} presented in Section 3 (which assumes the keys have already been distributed), its monolithic variant Π_{mECDSA} includes an explicit setup phase where the parties invoke $\mathcal{F}_{\text{RelaxedKeyGen}}$ to obtain the global public key Q and their secret key shares x_i . The signing phase then proceeds nearly identically to Π_{ECDSA} , with two necessary adjustments to match the $\mathcal{F}_{\text{mECDSA}}$ interface: First, session identifiers for all sub-protocols and messages are prepended with a unique signature identifier **sigid** to support multiple concurrent signing sessions under the same key. Second, explicit protocol aborts during the signing phase are handled by outputting a *failure* message to the environment for that specific **sigid**.

The Monolithic Refined Protocol Π_{mECDSA}

- **Public parameters:**
 - Description $\mathcal{G}$ of a cyclic group $\mathbb{G}$ of prime order q that is generated by $G \in \mathbb{G}$
 - Number of parties $n \geq 2$
 - ECDSA hash function H
- **Ideal functionalities:** $\mathcal{F}_{\text{RelaxedKeyGen}[\mathcal{G}, n]}$, $\mathcal{F}_{\text{Com}}$, $\mathcal{F}_{\text{Zero}[q, n]}$, $\mathcal{F}_{\text{RVOLE}[q, 2]}$.
- **Parties:** $P_1, \dots, P_n$ with distinct party identifiers $\text{pid}_1, \dots, \text{pid}_n$.
- **Setup Phase:**
 1. Upon receiving $(\text{init}, \text{sid})$ from the environment $\mathcal{Z}$, each party P_i where $i \in [n]$ checks whether they have already locally stored a record of the form $(\text{key-pair}, \text{sid}, Q, x_i)$. If not, then P_i sends $(\text{keygen}, \text{sid})$ to $\mathcal{F}_{\text{RelaxedKeyGen}[\mathcal{G}, n]}$.
 2. Upon receiving $(\text{abort}, \text{sid})$ from $\mathcal{F}_{\text{RelaxedKeyGen}[\mathcal{G}, n]}$, each party P_i sends $(\text{abort}, \text{sid})$ to the environment $\mathcal{Z}$ and ignores any further messages for **sid**.
 3. Upon receiving $(\text{key-pair}, \text{sid}, Q, x_i)$ from $\mathcal{F}_{\text{RelaxedKeyGen}[\mathcal{G}, n]}$, each party P_i locally stores it, and sends $(\text{public-key}, \text{sid}, Q)$ to the environment $\mathcal{Z}$.
- **Signing Phase:**
 4. **First round for every party P_i where $i \in [n]$:**
 Upon receiving $(\text{sign}, \text{sid}, \text{sigid}, m)$ from the environment $\mathcal{Z}$, if a record $(\text{key-pair}, \text{sid}, Q, x_i)$ has not been stored or **sigid** is not fresh, then ignore and otherwise proceed as follows:
 - (a) Sample $k_i, \phi_i \in_R \mathbb{Z}_q$
 - (b) Compute $R_i = k_i \cdot G$
 - (c) For every $j \in [n] \setminus \{i\}$:
 - i. Send $(\text{commit}, \text{sid} || \text{sigid} || \text{pid}_i || \text{pid}_j, i, j, R_i)$ to $\mathcal{F}_{\text{Com}}$
 - ii. Send $(\text{Alice-sample}, \text{sid} || \text{sigid} || \text{pid}_i || \text{pid}_j, i, j)$ to $\mathcal{F}_{\text{RVOLE}[q, 2]}$
 - (d) Send $(\text{sample}, \text{sid} || \text{sigid} || \text{pid}_1 || \dots || \text{pid}_n)$ to $\mathcal{F}_{\text{Zero}[q, n]}$
 5. **Second round for every party P_i where $i \in [n]$:**
 Upon receiving:

- (receipt, sid||sigid||pid_j||pid_i, j, i) from $\mathcal{F}_{\text{Com}}$ for every $j \in [n] \setminus \{i\}$
- (ready, sid||sigid||pid_j||pid_i, j, i) from $\mathcal{F}_{\text{RVOLE}[q,2]}$ for every $j \in [n] \setminus \{i\}$
- (Alice-sample, sid||sigid||pid_i||pid_j, i, j, $\chi_{i,j}$) from $\mathcal{F}_{\text{RVOLE}[q,2]}$ for every $j \in [n] \setminus \{i\}$
- (mask, sid||sigid||pid₁||...||pid_n, ζ_i) from $\mathcal{F}_{\text{rZero}[q,n]}$

proceed as follows:

- (a) Compute

$$\hat{x}_i = x_i + \zeta_i \bmod q$$

$$\hat{Q}_i = \hat{x}_i \cdot G$$

- (b) For every $j \in [n] \setminus \{i\}$:

- i. Send (Bob-multiply, sid||sigid||pid_j||pid_i, j, i, (k_i , $\hat{x}_i$)) to $\mathcal{F}_{\text{RVOLE}[q,2]}$.
- ii. Receive (Bob-share, sid||sigid||pid_j||pid_i, j, i, ($c_{i,j}^u$, $c_{i,j}^v$)) from $\mathcal{F}_{\text{RVOLE}[q,2]}$.
- iii. Compute

$$C_{i,j}^u = c_{i,j}^u \cdot G$$

$$C_{i,j}^v = c_{i,j}^v \cdot G$$

$$\psi_{i,j} = \phi_i - \chi_{i,j} \bmod q$$

- iv. Send (decommit, sid||sigid||pid_i||pid_j, i, j, R_i) to $\mathcal{F}_{\text{Com}}$.
- v. Send (sid||sigid||pid_i||pid_j, $C_{i,j}^u$, $C_{i,j}^v$, $\psi_{i,j}$, $\hat{Q}_i$) to party P_j .

6. **Third round for every party P_i where $i \in [n]$:**

Upon receiving:

- (decommit, sid||sigid||pid_j||pid_i, j, i, R_j) from $\mathcal{F}_{\text{Com}}$ for every $j \in [n] \setminus \{i\}$
- (Alice-share, sid||sigid||pid_i||pid_j, i, j, ($d_{i,j}^u$, $d_{i,j}^v$)) from $\mathcal{F}_{\text{RVOLE}[q,2]}$ for every $j \in [n] \setminus \{i\}$
- (sid||sigid||pid_j||pid_i, $C_{j,i}^u$, $C_{j,i}^v$, $\psi_{j,i}$, $\hat{Q}_j$) from party P_j for every $j \in [n] \setminus \{i\}$

proceed as follows:

- (a) Verify that $\sum_{j \in [n]} \hat{Q}_j = Q$. If not, output (failure, sid, sigid) to the environment $\mathcal{Z}$ and halt the execution for sigid.
- (b) For every $j \in [n] \setminus \{i\}$ verify that

$$\chi_{i,j} \cdot R_j - C_{j,i}^u = d_{i,j}^u \cdot G$$

$$\chi_{i,j} \cdot \hat{Q}_j - C_{j,i}^v = d_{i,j}^v \cdot G$$

If any verification fails, output (failure, sid, sigid) to the environment $\mathcal{Z}$ and halt the execution for sigid.

- (c) Compute

$$R = \sum_{j \in [n]} R_j$$

$$u_i = k_i \cdot \left(\phi_i + \sum_{j \in [n] \setminus \{i\}} \psi_{j,i} \right) + \sum_{j \in [n] \setminus \{i\}} (c_{i,j}^u + d_{i,j}^u) \bmod q$$

$$v_i = \hat{x}_i \cdot \left(\phi_i + \sum_{j \in [n] \setminus \{i\}} \psi_{j,i} \right) + \sum_{j \in [n] \setminus \{i\}} (c_{i,j}^\vee + d_{i,j}^\vee) \bmod q$$

$$w_i = H(m) \cdot \phi_i + r \cdot v_i \bmod q$$

where $r \in \mathbb{Z}_q$ is the x -coordinate (modulo q) of the elliptic-curve point $R \in \mathbb{G}$.

(d) For every $j \in [n] \setminus \{i\}$ send $(\text{sid}||\text{sigid}||\text{pid}_i||\text{pid}_j, R, w_i, u_i)$ to party P_j .

7. Output for every party P_i where $i \in [n]$:

Upon receiving $(\text{sid}||\text{sigid}||\text{pid}_j||\text{pid}_i, \tilde{R}_j, w_j, u_j)$ from party P_j for every $j \in [n] \setminus \{i\}$, proceed as follows:

- (a) If $\tilde{R}_j \neq R$ for some $j \in [n] \setminus \{i\}$, output $(\text{failure}, \text{sid}, \text{sigid})$ to the environment $\mathcal{Z}$ and halt the execution for **sigid**.
- (b) Compute $s = \left(\sum_{j \in [n]} w_j \right) \cdot \left(\sum_{j \in [n]} u_j \right)^{-1} \bmod q$.
- (c) If $R \neq (s^{-1} \cdot H(m)) \cdot G + (s^{-1} \cdot r) \cdot Q$, output $(\text{failure}, \text{sid}, \text{sigid})$ to the environment $\mathcal{Z}$ and halt the execution for **sigid**.
- (d) Send $(\text{signature}, \text{sid}, \text{sigid}, \sigma)$ to the environment $\mathcal{Z}$, where $\sigma = (r, s)$.

C.4 Proof of Statistical Security

We prove the following theorem:

Theorem C.1. *For any cyclic group $\mathbb{G}$ of prime order q and for any integer $n \geq 2$, protocol Π_{mECDSA} statistically UC-realizes the functionality $\mathcal{F}_{\text{mECDSA}}$ in the $(\mathcal{F}_{\text{RelaxedKeyGen}}[G, n], \mathcal{F}_{\text{Com}}, \mathcal{F}_{\text{rZero}}[q, n], \mathcal{F}_{\text{rVole}}[q, 2])$ -hybrid model against a malicious adversary that statically corrupts up to $n - 1$ parties.*

As in Appendix A, for clarity and simplicity we present our analysis assuming a fixed set of parties, authenticated channels, and the UC framework's standard delayed output mechanism. As standard in the UC framework, in what follows for every black-box simulator $\mathcal{S}$, hybrid-model adversary $\mathcal{A}$, security parameter $\kappa_c \in \mathbb{N}$, and input $z \in \{0, 1\}^*$ provided to the environment, we denote by $\text{HYBRID}_{\Pi_{\text{mECDSA}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{F}}(\kappa_c, z)$ and $\text{IDEAL}_{\mathcal{F}_{\text{mECDSA}}, \mathcal{S}^{\mathcal{A}}, \mathcal{Z}}(\kappa_c, z)$ the distributions corresponding to the output of the environment following an execution of the protocol Π_{mECDSA} in the $\mathcal{F}$ -hybrid model and following an ideal execution with the functionality $\mathcal{F}_{\text{mECDSA}}$, respectively. In addition, as in our proof of Theorem 5.2, for clarity and simplicity we present the proof of Theorem C.1 in a synchronous communication model, and note that our analysis extends to the asynchronous setting as discussed in Section 5 following the proof of Theorem 5.2.

The proof of Theorem C.1 closely follows the structure of the proof of Theorem 5.2, and relies on its core technical content. This include, in particular, the consistency check replacement argument (experiments $\mathcal{H}_1$ – $\mathcal{H}_2$ of Theorem 5.2), the algebraic verification that the simulator's back-computed values u_C , v_C , and w_C correctly represent the corrupted parties' contributions (experiment $\mathcal{H}_7$ of Theorem 5.2), and the arguments establishing the uniformity of the simulated honest parties' values (experiments $\mathcal{H}_8$ – $\mathcal{H}_9$ of Theorem 5.2). The key differences introduced by the monolithic setting are as follows. First, the simulator must

handle the setup phase by interacting with both $\mathcal{F}_{\text{mECDSA}}$ and $\mathcal{F}_{\text{RelaxedKeyGen}[\mathcal{G},n]}$, embedding the public key Q received from $\mathcal{F}_{\text{mECDSA}}$ into the emulated key generation without knowing its discrete logarithm; this is addressed in experiment $\mathcal{H}_2$ below. Second, since the monolithic functionality $\mathcal{F}_{\text{mECDSA}}$ internally checks hash consistency across all parties' messages (whereas the decoupled functionality $\mathcal{F}_{\text{ECDSA}}$ assumes all parties receive the same message), the proof must explicitly handle the case where honest parties receive messages with inconsistent hash images from the environment; this is addressed in experiment $\mathcal{H}_5$ below. Third, the signature obtained from $\mathcal{F}_{\text{mECDSA}}$ must be embedded into the protocol transcript for multiple concurrent signing sessions under the same key, requiring the simulator to manage per-session state and blacklisting of corrupted parties across sessions; these mechanisms are incorporated throughout the simulator's description and are reflected in the hybrid argument for experiment $\mathcal{H}_6$.

Proof of Theorem C.1. Fix a cyclic group $\mathbb{G}$ of prime order q and an integer $n \geq 2$, and let $\mathcal{A}$ be a malicious adversary in the $\mathcal{F}$ -hybrid model that statically corrupts up to $n - 1$ parties, where $\mathcal{F} = (\mathcal{F}_{\text{RelaxedKeyGen}[\mathcal{G},n]}, \mathcal{F}_{\text{Com}}, \mathcal{F}_{\text{rZero}[q,n]}, \mathcal{F}_{\text{RVOLE}[q,2]})$. We present a probabilistic polynomial-time black-box simulator $\mathcal{S}$ such that for every probabilistic polynomial-time environment $\mathcal{Z}$, polynomial $p(\cdot)$, security parameter $\kappa_c \in \mathbb{N}$, and for every input $z \in \{0,1\}^{p(\kappa_c)}$ provided to the environment $\mathcal{Z}$, it holds that the statistical distance between the distributions $\text{HYBRID}_{\Pi_{\text{mECDSA}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{F}}(\kappa_c, z)$ and $\text{IDEAL}_{\mathcal{F}_{\text{mECDSA}}, \mathcal{S}^{\mathcal{A}}, \mathcal{Z}}(\kappa_c, z)$ is a negligible function of the security parameter $\kappa_c \in \mathbb{N}$.

The simulator $\mathcal{S}^{\mathcal{A}}(\kappa_c)$, defined below, emulates a hybrid-model execution to the adversary $\mathcal{A}$ while forwarding all messages from the environment $\mathcal{Z}$ to $\mathcal{A}$ and vice versa. When the emulated execution begins, the hybrid-model adversary $\mathcal{A}$ announces the identities of up to $n - 1$ corrupt parties, and we denote by $\mathcal{C} \subseteq [n]$ and $\mathcal{H} = [n] \setminus \mathcal{C}$ the sets of identities of the corrupted parties and honest parties, respectively. The simulator $\mathcal{S}$ interacts with the ideal functionality $\mathcal{F}_{\text{mECDSA}}$ on behalf of every corrupted party, and emulates a hybrid-model execution of the protocol Π_{mECDSA} to the hybrid-model adversary $\mathcal{A}$ by interacting with $\mathcal{A}$ and the corrupted parties on behalf of all honest parties and the ideal functionalities $\mathcal{F}_{\text{RelaxedKeyGen}[\mathcal{G},n]}$, $\mathcal{F}_{\text{Com}}$, $\mathcal{F}_{\text{rZero}[q,n]}$ and $\mathcal{F}_{\text{RVOLE}[q,2]}$. The simulator $\mathcal{S}^{\mathcal{A}}$ is defined as follows:

• **Setup Phase:**

1. On receiving $(\text{init-req}, \text{sid}, h)$ from $\mathcal{F}_{\text{mECDSA}}$ for some $h \in \mathcal{H}$, send $(\text{keygen-req}, \text{sid}, h)$ to $\mathcal{A}$ on behalf of the emulated functionality $\mathcal{F}_{\text{RelaxedKeyGen}[\mathbb{G},n]}$.
2. On receiving $(\text{keygen}, \text{sid})$ sent by $\mathcal{A}$ on behalf of some corrupted party P_c , where $c \in \mathcal{C}$, to the emulated functionality $\mathcal{F}_{\text{RelaxedKeyGen}[\mathbb{G},n]}$, send $(\text{init}, \text{sid})$ to $\mathcal{F}_{\text{mECDSA}}$ on behalf of party P_c and send $(\text{keygen-req}, \text{sid}, c)$ to $\mathcal{A}$ on behalf of the emulated functionality $\mathcal{F}_{\text{RelaxedKeyGen}[\mathbb{G},n]}$.
3. On receiving $(\text{public-key}, \text{sid}, Q)$ from the functionality $\mathcal{F}_{\text{mECDSA}}$ (indicating all parties initiated), obtain $(\text{adv-shares}, \text{sid}, \{\tilde{Q}_c\}_{c \in \mathcal{C}})$ sent by $\mathcal{A}$ to the emulated functionality $\mathcal{F}_{\text{RelaxedKeyGen}[\mathbb{G},n]}$.
4. Sample $\hat{x}_c \in_R \mathbb{Z}_q$ for every $c \in \mathcal{C}$ and compute $Q_c \leftarrow \tilde{Q}_c + \hat{x}_c \cdot G$.

5. Fix some $h^* \in \mathcal{H}$ (e.g., the smallest index $h \in [n]$ of an honest party P_h).
6. For every $h \in \mathcal{H} \setminus \{h^*\}$, sample $x_h \in_R \mathbb{Z}_q$ and compute $Q_h \leftarrow x_h \cdot G$.
7. Compute $Q_{h^*} \leftarrow Q - \sum_{h \in \mathcal{H} \setminus \{h^*\}} Q_h - \sum_{c \in \mathcal{C}} Q_c$.
8. Send (honest-shares, sid, $Q, \{Q_h\}_{h \in \mathcal{H}}, \{\hat{x}_c\}_{c \in \mathcal{C}}$) to $\mathcal{A}$ on behalf of the emulated functionality $\mathcal{F}_{\text{RelaxedKeyGen}[G, n]}$.
9. On receiving (release, sid, h) or (abort, sid, h) from $\mathcal{A}$ sent to the emulated functionality $\mathcal{F}_{\text{RelaxedKeyGen}[G, n]}$ for some $h \in \mathcal{H}$, forward it directly to the functionality $\mathcal{F}_{\text{mECDSA}}$.
10. Initialize a ‘blacklist’ for sid as empty.
- **Handling Aborts:**
11. If at any point during the signing phase $\mathcal{A}$ sends an explicit (abort, ...) message on behalf of $\mathcal{F}_{\text{RVOLE}[q, 2]}$ or $\mathcal{F}_{\text{Com}}$ for an interaction between $c \in \mathcal{C}$ and $h \in \mathcal{H}$, emulate P_h outputting (failure, sid, sigid) to the environment. Send (fail, sid, sigid', h) directly to $\mathcal{F}_{\text{mECDSA}}$ for sigid and for all concurrent sessions sigid' involving c . Append (h, c) to the ‘blacklist’ for sid, and ignore all future instructions for these sessions.
- **Signing Phase Round 1:**
12. On receiving (sig-req, sid, sigid, h, m_h) from $\mathcal{F}_{\text{mECDSA}}$ for some honest party P_h where $h \in \mathcal{H}$ (indicating that the environment instructed P_h to sign), proceed as follows: if for any $c \in \mathcal{C}$ it holds that (h, c) is in the ‘blacklist’ for sid, emulate P_h ignoring the session for this pair. Otherwise, emulate the first-round messages of P_h by sending $\mathcal{A}$ the following messages:
 - (receipt, sid||sigid||pid _{h} ||pid _{c} , h, c) on behalf of the emulated functionality $\mathcal{F}_{\text{Com}}$.
 - (ready, sid||sigid||pid _{h} ||pid _{c} , h, c) on behalf of the emulated functionality $\mathcal{F}_{\text{RVOLE}[q, 2]}$.
13. Send the message (choose, sid||sigid||pid₁||...||pid _{n}) to $\mathcal{A}$ on behalf of the emulated functionality $\mathcal{F}_{\text{rZero}[q, n]}$. When $\mathcal{A}$ replies with $t \in \mathbb{Z}_q$, sample $\{\zeta_c\}_{c \in \mathcal{C}}$ uniformly and independently from $\mathbb{Z}_q$ subject to $\sum_{c \in \mathcal{C}} \zeta_c = t \bmod q$, and for every $c \in \mathcal{C}$ send (mask, sid||sigid||pid₁||...||pid _{n} , ζ_c) to $\mathcal{A}$ on behalf of the emulated functionality $\mathcal{F}_{\text{rZero}[q, n]}$.
- **Signing Phase Round 2:**
14. For every $c \in \mathcal{C}$ and $h \in \mathcal{H}$ obtain the following messages from $\mathcal{A}$:
 - (commit, sid||sigid||pid _{c} ||pid _{h} , $c, h, R_{c, h}$) sent to the emulated functionality $\mathcal{F}_{\text{Com}}$.
 - (Alice-sample, sid||sigid||pid _{c} ||pid _{h} , $c, h, \chi_{c, h}$) sent to the emulated functionality $\mathcal{F}_{\text{RVOLE}[q, 2]}$.
 - (Alice-share, sid||sigid||pid _{c} ||pid _{h} , $c, h, (d_{c, h}^u, d_{c, h}^v)$) sent to the emulated functionality $\mathcal{F}_{\text{RVOLE}[q, 2]}$.
15. Once the messages in Step 13 have been collected for all relevant pairs, send (sign, sid, sigid, m_{h^*}) to $\mathcal{F}_{\text{mECDSA}}$ on behalf of every $c \in \mathcal{C}$, and then send (proceed, sid, sigid, c) to $\mathcal{F}_{\text{mECDSA}}$.
 - (a) If (signature, sid, sigid, σ) is received from $\mathcal{F}_{\text{mECDSA}}$, parse $\sigma = (r, s)$, and compute $R = (s^{-1} \cdot H(m_{h^*})) \cdot G + (s^{-1} \cdot r) \cdot Q$.
 - (b) If (failure, sid, sigid) is received instead, sample $R \in_R \mathbb{G}$ and $s \in_R \mathbb{Z}_q$ uniformly, and let r be the x -coordinate modulo q of R .

16. For every $h \in \mathcal{H} \setminus \{h^*\}$ sample $\hat{x}_h, k_h \in_R \mathbb{Z}_q$ and compute $\hat{Q}_h = \hat{x}_h \cdot G$ and $R_h = k_h \cdot G$.
17. Compute

$$\begin{aligned}\hat{Q}_{h^*} &= Q - \sum_{h \in \mathcal{H} \setminus \{h^*\}} \hat{Q}_h - \sum_{c \in \mathcal{C}} (Q_c + \zeta_c \cdot G) \\ R_{h^*} &= R - \sum_{h \in \mathcal{H} \setminus \{h^*\}} R_h - \sum_{c \in \mathcal{C}} R_{c,h^*}\end{aligned}$$

18. For every $c \in \mathcal{C}$ and $h \in \mathcal{H}$ sample $\psi_{h,c} \in_R \mathbb{Z}_q$, compute

$$\begin{aligned}C_{h,c}^u &= \chi_{c,h} \cdot R_h - d_{c,h}^u \cdot G \\ C_{h,c}^v &= \chi_{c,h} \cdot \hat{Q}_h - d_{c,h}^v \cdot G ,\end{aligned}$$

and send $\mathcal{A}$ the following messages:

- (decommit, sid||sigid||pid_h||pid_c, h, c, R_h) on behalf of the emulated functionality $\mathcal{F}_{\text{Com}}$.
- (sid||sigid||pid_h||pid_c, $C_{h,c}^u, C_{h,c}^v, \psi_{h,c}, \hat{Q}_h$) on behalf of party P_h .

• **Signing Phase Round 3:**

19. For every $c \in \mathcal{C}$ and $h \in \mathcal{H}$ obtain the following messages from $\mathcal{A}$:
 - (Bob-multiply, sid||sigid||pid_h||pid_c, $h, c, (k_{c,h}, \hat{x}_{c,h})$) sent to $\mathcal{F}_{\text{RVOLE}[q,2]}$.
 - (Bob-share, sid||sigid||pid_h||pid_c, $h, c, (c_{c,h}^u, c_{c,h}^v)$) sent to $\mathcal{F}_{\text{RVOLE}[q,2]}$.
 - (sid||sigid||pid_c||pid_h, $C_{c,h}^u, C_{c,h}^v, \psi_{c,h}, \hat{Q}_{c,h}$) sent to party P_h .
20. For every $h \in \mathcal{H} \setminus \{h^*\}$:

If $\sum_{c \in \mathcal{C}} \hat{Q}_{c,h} \neq \sum_{c \in \mathcal{C}} (Q_c + \zeta_c \cdot G)$, or if there exists $c \in \mathcal{C}$ such that $k_{c,h} \cdot G \neq R_{c,h}$ or $\hat{x}_{c,h} \cdot G \neq \hat{Q}_{c,h}$ or $c_{c,h}^u \cdot G \neq C_{c,h}^u$ or $c_{c,h}^v \cdot G \neq C_{c,h}^v$, then emulate party P_h outputting (failure, sid, sigid) to the environment. Send (fail, sid, sigid', h) directly to $\mathcal{F}_{\text{mECDSA}}$ for sigid and all concurrent sessions sigid' involving c , append (h, c) to the 'blacklist' for sid, and ignore all future instructions pertaining to these sessions. Otherwise, sample $u_h, v_h, \phi_h \in_R \mathbb{Z}_q$, compute

$$\begin{aligned}\tilde{R}_h &= \sum_{h' \in \mathcal{H}} R_{h'} + \sum_{c \in \mathcal{C}} R_{c,h} \\ w_h &= H(m_{h^*}) \cdot \phi_h + r_h \cdot v_h \bmod q ,\end{aligned}$$

where $r_h \in \mathbb{Z}_q$ is the x -coordinate modulo q of $\tilde{R}_h$, and for every $c \in \mathcal{C}$ send the message (sid||sigid||pid_h||pid_c, $\tilde{R}_h, w_h, u_h$) to $\mathcal{A}$ on behalf of party P_h .

21. If $\sum_{c \in \mathcal{C}} \hat{Q}_{c,h^*} \neq \sum_{c \in \mathcal{C}} (Q_c + \zeta_c \cdot G)$, or if there exists $c \in \mathcal{C}$ such that $k_{c,h^*} \cdot G \neq R_{c,h^*}$ or $\hat{x}_{c,h^*} \cdot G \neq \hat{Q}_{c,h^*}$ or $c_{c,h^*}^u \cdot G \neq C_{c,h^*}^u$ or $c_{c,h^*}^v \cdot G \neq C_{c,h^*}^v$, then emulate party P_{h^*} outputting (failure, sid, sigid) to the environment. Send (fail, sid, sigid', h^*) directly to $\mathcal{F}_{\text{mECDSA}}$ for sigid and all concurrent sessions sigid' involving c , append (h^*, c) to the 'blacklist' for sid, and ignore all future instructions pertaining to these sessions. Otherwise, proceed as follows:
 - (a) Sample $u_{h^*} \in_R \mathbb{Z}_q$.

(b) Compute:

$$\begin{aligned}
\tilde{R}_{h^*} &= R \\
u_{\mathcal{H} \setminus \{h^*\}} &= \sum_{h \in \mathcal{H} \setminus \{h^*\}} u_h \bmod q \\
v_{\mathcal{H} \setminus \{h^*\}} &= \sum_{h \in \mathcal{H} \setminus \{h^*\}} v_h \bmod q \\
u_{\mathcal{C}} &= \sum_{c \in \mathcal{C}} \sum_{h \in \mathcal{H}} (k_{c,h^*} \cdot \psi_{h,c} + c_{c,h}^u + d_{c,h}^u) \\
&\quad + \left(\sum_{c \in \mathcal{C}} k_{c,h^*} \right) \cdot \left(\sum_{c \in \mathcal{C}} (\psi_{c,h^*} + \chi_{c,h^*}) \right) \bmod q \\
v_{\mathcal{C}} &= \sum_{c \in \mathcal{C}} \sum_{h \in \mathcal{H}} (\hat{x}_{c,h^*} \cdot \psi_{h,c} + c_{c,h}^v + d_{c,h}^v) \\
&\quad + \left(\sum_{c \in \mathcal{C}} \hat{x}_{c,h^*} \right) \cdot \left(\sum_{c \in \mathcal{C}} (\psi_{c,h^*} + \chi_{c,h^*}) \right) \bmod q \\
w_{\mathcal{H} \setminus \{h^*\}} &= H(m_{h^*}) \cdot \left(\sum_{h \in \mathcal{H} \setminus \{h^*\}} \phi_h \right) + r \cdot v_{\mathcal{H} \setminus \{h^*\}} \bmod q \\
w_{\mathcal{C}} &= H(m_{h^*}) \cdot \left(\sum_{c \in \mathcal{C}} (\psi_{c,h^*} + \chi_{c,h^*}) \right) + r \cdot v_{\mathcal{C}} \bmod q \\
w_{h^*} &= s \cdot (u_{h^*} + u_{\mathcal{H} \setminus \{h^*\}} + u_{\mathcal{C}}) - (w_{\mathcal{H} \setminus \{h^*\}} + w_{\mathcal{C}}) \bmod q
\end{aligned}$$

(c) For every $c \in \mathcal{C}$ send the message $(\text{sid} || \text{sigid} || \text{pid}_{h^*} || \text{pid}_c, R, w_{h^*}, u_{h^*})$ to $\mathcal{A}$ on behalf of party P_{h^*} .

• **Output Phase:**

22. For every $c \in \mathcal{C}$ and $h \in \mathcal{H}$ obtain the message $(\text{sid} || \text{sigid} || \text{pid}_c || \text{pid}_h, \tilde{R}_{c,h}, w_{c,h}, u_{c,h})$ sent from $\mathcal{A}$ to P_h on behalf of P_c .
23. If for some $h \neq h' \in \mathcal{H}$ it holds that $\tilde{R}_h \neq \tilde{R}_{h'}$ then send $(\text{fail}, \text{sid}, \text{sigid}, h'')$ to $\mathcal{F}_{\text{mECDSA}}$ for all honest parties $h'' \in \mathcal{H}$, and ignore all future instructions for sigid .
24. For every $h \in \mathcal{H}$:
 - (a) If for some $c \in \mathcal{C}$ it holds that $\tilde{R}_{c,h} \neq \tilde{R}_h$ then emulate party P_h outputting $(\text{failure}, \text{sid}, \text{sigid})$ to the environment, send $(\text{fail}, \text{sid}, \text{sigid}', h)$ directly to $\mathcal{F}_{\text{mECDSA}}$ for sigid and all concurrent sessions sigid' , append (h, c) to the ‘blacklist’, and ignore future instructions.
 - (b) If

$$s = \left(\sum_{h' \in \mathcal{H}} w_{h'} + \sum_{c \in \mathcal{C}} w_{c,h} \right) \cdot \left(\sum_{h' \in \mathcal{H}} u_{h'} + \sum_{c \in \mathcal{C}} u_{c,h} \right)^{-1} \bmod q$$

then emulate party P_h outputting $(\text{signature}, \text{sid}, \text{sigid}, \sigma)$ to the environment and send $(\text{proceed}, \text{sid}, \text{sigid}, h)$ directly to $\mathcal{F}_{\text{mECDSA}}$; otherwise, send $(\text{fail}, \text{sid}, \text{sigid}, h)$ to $\mathcal{F}_{\text{mECDSA}}$.

Provided with the above description of $\mathcal{S}$, we consider a sequence of experiments $\mathcal{H}_1, \dots, \mathcal{H}_7$, and prove that the distributions of the environment's output in each pair of consecutive experiments are statistically close. We abuse notation and additionally denote by $\mathcal{H}_i = \mathcal{H}_i(\kappa_c, z)$ the distribution of the environment's output in the i th experiment when provided with κ_c and z as inputs. The sequence of experiments begins with the execution of the protocol Π_{mECDSA} in the $\mathcal{F}$ -hybrid model. Then, by gradually replacing the code of the honest parties and the ideal functionalities $\mathcal{F}_{\text{RelaxedKeyGen}[\mathcal{G}, n]}$, $\mathcal{F}_{\text{Com}}$, $\mathcal{F}_{\text{rZero}[q, n]}$ and $\mathcal{F}_{\text{rVole}[q, 2]}$ with that of the simulator $\mathcal{S}$, the sequence ends with the ideal-model execution of the functionality $\mathcal{F}_{\text{mECDSA}}$.

- **Experiment $\mathcal{H}_1(\kappa_c, z)$.** This experiment is obtained from the execution of the protocol Π_{mECDSA} in the $\mathcal{F}$ -hybrid model by replacing the honest parties and the ideal functionalities $\mathcal{F}_{\text{RelaxedKeyGen}[\mathcal{G}, n]}$, $\mathcal{F}_{\text{Com}}$, $\mathcal{F}_{\text{rZero}[q, n]}$ and $\mathcal{F}_{\text{rVole}[q, 2]}$ with a single emulator algorithm that emulates them and interacts with the adversary, the environment, and the corrupted parties on their behalf.

Experiment $\mathcal{H}_1$ does not modify the distribution of the environment's view compared to an execution of the protocol Π_{mECDSA} in the $\mathcal{F}$ -hybrid model, and thus

$$\{\mathcal{H}_1(\kappa_c, z)\}_{\kappa_c \in \mathbb{N}, z \in \{0,1\}^{p(\kappa_c)}} \equiv \{\text{HYBRID}_{\Pi_{\text{mECDSA}}, \mathcal{A}, \mathcal{Z}}^{\mathcal{F}}(\kappa_c, z)\}_{\kappa_c \in \mathbb{N}, z \in \{0,1\}^{p(\kappa_c)}}.$$

- **Experiment $\mathcal{H}_2(\kappa_c, z)$.** This experiment is obtained from experiment $\mathcal{H}_1$ by modifying the emulation of the setup phase. Specifically, instead of internally generating the global key pair (x, Q) and the honest parties' shares x_h , the emulator now proceeds exactly as defined in Steps 2–7 of the simulator $\mathcal{S}$. It receives the global public key Q from $\mathcal{F}_{\text{mECDSA}}$. Upon receiving $\{\check{Q}_c\}_{c \in \mathcal{C}}$ from $\mathcal{A}$, the emulator samples $\hat{x}_c \in_R \mathbb{Z}_q$ and computes $Q_c \leftarrow \check{Q}_c + \hat{x}_c \cdot G$ for every $c \in \mathcal{C}$. It then samples $x_h \in_R \mathbb{Z}_q$ and computes $Q_h = x_h \cdot G$ for every $h \in \mathcal{H} \setminus \{h^*\}$, and computes $Q_{h^*} \leftarrow Q - \sum_{h \in \mathcal{H} \setminus \{h^*\}} Q_h - \sum_{c \in \mathcal{C}} Q_c$. The emulator sends (honest-shares, sid, Q , $\{Q_h\}_{h \in \mathcal{H}}$, $\{\hat{x}_c\}_{c \in \mathcal{C}}$) to $\mathcal{A}$.

We show that experiments $\mathcal{H}_2$ and $\mathcal{H}_1$ are identically distributed. In experiment $\mathcal{H}_2$, the global public key Q is uniformly distributed over $\mathbb{G}$ (as it is generated via $x \cdot G$ for a uniformly sampled $x \in_R \mathbb{Z}_q$ by $\mathcal{F}_{\text{mECDSA}}$). Since x_h is sampled uniformly and independently for every $h \in \mathcal{H} \setminus \{h^*\}$, the resulting $\{Q_h\}_{h \in \mathcal{H} \setminus \{h^*\}}$ are uniformly distributed, exactly matching $\mathcal{H}_1$. Furthermore, Since Q is uniform and independent of the adversary's inputs, the back-computed value $Q_{h^*} = Q - \sum_{h \in \mathcal{H} \setminus \{h^*\}} Q_h - \sum_{c \in \mathcal{C}} Q_c$ is also uniformly distributed, exactly matching the marginal distribution of Q_{h^*} in $\mathcal{H}_1$. Thus, the view of the adversary in the setup phase is identically distributed in both experiments, and therefore

$$\{\mathcal{H}_2(\kappa_c, z)\}_{\kappa_c \in \mathbb{N}, z \in \{0,1\}^{p(\kappa_c)}} \equiv \{\mathcal{H}_1(\kappa_c, z)\}_{\kappa_c \in \mathbb{N}, z \in \{0,1\}^{p(\kappa_c)}}.$$

- **Experiment $\mathcal{H}_3(\kappa_c, z)$.** This experiment is obtained from experiment $\mathcal{H}_2$ by modifying the consistency checks performed by the emulator on behalf of the honest parties during the signing phase (Steps 19 and 20 of $\mathcal{S}$). Instead of

verifying that $\chi_{h,c} \cdot R_{c,h} - C_{c,h}^u = d_{h,c}^u \cdot G$ and $\chi_{h,c} \cdot \hat{Q}_{c,h} - C_{c,h}^v = d_{h,c}^v \cdot G$, the emulator now directly verifies the explicit discrete logarithms extracted from $\mathcal{F}_{\text{RVOLE}[q,2]}$: it checks that $k_{c,h} \cdot G = R_{c,h}$, $\hat{x}_{c,h} \cdot G = \hat{Q}_{c,h}$, $c_{c,h}^u \cdot G = C_{c,h}^u$, and $c_{c,h}^v \cdot G = C_{c,h}^v$. If any check fails, it follows the abort procedures outlined in the simulator.

By the exact same argument establishing the indistinguishability of experiments $\mathcal{H}_1$ and $\mathcal{H}_2$ in the proof of Theorem 5.2, the statistical distance between experiments $\mathcal{H}_3$ and $\mathcal{H}_2$ is bounded by $n^2/(2q)$ per signing session. Thus,

$$\{\mathcal{H}_3(\kappa_c, z)\}_{\kappa_c \in \mathbb{N}, z \in \{0,1\}^{p(\kappa_c)}} \approx^s \{\mathcal{H}_2(\kappa_c, z)\}_{\kappa_c \in \mathbb{N}, z \in \{0,1\}^{p(\kappa_c)}}.$$

- **Experiment $\mathcal{H}_4(\kappa_c, z)$.** This experiment behaves identically to $\mathcal{H}_3$, except in the case where the adversary corrupts no parties ($\mathcal{C} = \emptyset$). If the environment triggers a signing instance where all n parties are honest and all messages m_i have consistent images under H , the emulator no longer executes the internal protocol code. Instead, it locally reconstructs $x = \sum_{i \in [n]} x_i \bmod q$, computes $\sigma \leftarrow (r, s)$ via the standard ECDSA signing algorithm using x and the consistent message m , and outputs σ to the environment on behalf of all parties.

When all parties are honest, the consistency checks trivially pass. By inspection of the protocol's output phase, the aggregation of the honest values u_i and w_i perfectly reconstructs the operations of the standard ECDSA signing algorithm, yielding a signature with a distribution identical to one produced by a local ECDSA computation. Thus, the output distributions are identical, and no other internal protocol values are observable by the adversary when $\mathcal{C} = \emptyset$. Therefore, $\{\mathcal{H}_4(\kappa_c, z)\} \equiv \{\mathcal{H}_3(\kappa_c, z)\}$.

- **Experiment $\mathcal{H}_5(\kappa_c, z)$.** This experiment behaves identically to $\mathcal{H}_4$, except when the environment triggers a signing instance containing two or more honest parties but provides them with messages that have inconsistent images under H . In $\mathcal{H}_5$, the emulator replaces m_h with m_{h^*} in the calculations of every honest party P_h for $h \in \mathcal{H} \setminus \{h^*\}$. If there exists some $h \in \mathcal{H} \setminus \{h^*\}$ such that $H(m_h) \neq H(m_{h^*})$, the emulator samples $w_{h^*} \in_R \mathbb{Z}_q$ uniformly at random instead of calculating it via the instructions in the protocol. It then outputs (failure, sid, sigid) to the environment on behalf of all honest parties, and ignores all future messages for sigid.

We show that experiments $\mathcal{H}_5$ and $\mathcal{H}_4$ are statistically indistinguishable. In $\mathcal{H}_4$, honest parties output a failure only if the reconstructed signature is invalid. When computing their signature fragments, the honest parties calculate $w_{h^*} = H(m_{h^*}) \cdot \phi_{h^*} + r \cdot v_{h^*} \bmod q$ and $w_h = H(m_h) \cdot \phi_h + r \cdot v_h \bmod q$. We observe that the leftmost terms of these equations are the only constituent parts of the final aggregated signature that depend upon m_{h^*} or m_h . Furthermore, the variables u_h and u_{h^*} commit the emulator to the sum of ϕ_h and ϕ_{h^*} (due to their linear dependence on the shared $\mathcal{F}_{\text{RVOLE}[q,2]}$ outputs $c_{h,h^*}^u + d_{h,h^*}^u$ and $c_{h^*,h}^u + d_{h^*,h}^u$),

but leave a degree of freedom in choosing the individual values. By summing w_{h^*} and w_h and rewriting, we obtain:

$$w_{h^*} + w_h = H(m_{h^*}) \cdot (\phi_{h^*} + \phi_h) + r \cdot (v_{h^*} + v_h) + (H(m_h) - H(m_{h^*})) \cdot \phi_h \bmod q .$$

If $H(m_{h^*}) = H(m_h)$, the rightmost term is zero and the signature is valid provided no parties deviate from the protocol. However, if $H(m_{h^*}) \neq H(m_h)$, the term $(H(m_h) - H(m_{h^*})) \cdot \phi_h$ acts as a uniform one-time pad because ϕ_h is sampled uniformly and is information-theoretically hidden from the adversary. This implies that w_{h^*} and w_h are not constrained by their sum, but are rather distributed uniformly over $\mathbb{Z}_q$. Once the message, public key, and nonce are fixed, there is exactly one valid ECDSA signature. Because w_{h^*} and w_h are uniform without constraint, the reconstructed signature s is identically uniform over $\mathbb{Z}_q$. The probability that this unconstrained, uniformly random s successfully satisfies the ECDSA verification equation is at most $1/q$ per signing session. Since n is polynomial in the security parameter κ_c , and the number of signing sessions is polynomially bounded, the probability of the real protocol producing a valid signature under inconsistent hashes is negligible. Thus, the forced failure in $\mathcal{H}_5$ is statistically indistinguishable from the honest parties failing the verification check in $\mathcal{H}_4$. Therefore,

$$\{\mathcal{H}_5(\kappa_c, z)\}_{\kappa_c \in \mathbb{N}, z \in \{0,1\}^{p(\kappa_c)}} \approx^s \{\mathcal{H}_4(\kappa_c, z)\}_{\kappa_c \in \mathbb{N}, z \in \{0,1\}^{p(\kappa_c)}} .$$

- **Experiment $\mathcal{H}_6(\kappa_c, z)$.** This experiment is obtained from experiment $\mathcal{H}_5$ by modifying the computation of the honest parties' signing values to match the sequence of modifications from experiments $\mathcal{H}_3$ through $\mathcal{H}_{10}$ in the proof of Theorem 5.2. Specifically, the emulator applies the exact operations described in Steps 14–23 of the simulator $\mathcal{S}$:

1. It queries $\mathcal{F}_{\text{mECDSA}}$ to obtain the signature $\sigma = (r, s)$ (or a failure notification), and computes R (Step 14).
2. It samples $\hat{x}_h, k_h \in_R \mathbb{Z}_q$ for every $h \in \mathcal{H} \setminus \{h^*\}$ and computes $\hat{Q}_h = \hat{x}_h \cdot G$ and $R_h = k_h \cdot G$. It then back-computes $\hat{Q}_{h^*}$ and R_{h^*} via subtraction (Steps 15–16).
3. It locally computes the responses $(C_{h,c}^u, C_{h,c}^v, \psi_{h,c})$ using the explicitly extracted $\chi_{c,h}, d_{c,h}^u, d_{c,h}^v$ values from the $\mathcal{F}_{\text{RVOLE}[q,2]}$ interaction (Step 17).
4. For every $h \in \mathcal{H} \setminus \{h^*\}$, it samples $u_h, v_h, \phi_h \in_R \mathbb{Z}_q$ independently and computes $w_h = H(m_{h^*}) \cdot \phi_h + r_h \cdot v_h \bmod q$, where r_h is the x -coordinate modulo q of $\hat{R}_h = \sum_{h' \in \mathcal{H}} R_{h'} + \sum_{c \in \mathcal{C}} R_{c,h}$ (Step 19).
5. It computes the aggregate components $u_c, v_c, w_c, u_{\mathcal{H} \setminus \{h^*\}},$ and $w_{\mathcal{H} \setminus \{h^*\}}$ using the extracted values k_{c,h^*} and $\hat{x}_{c,h^*}$, samples $u_{h^*} \in_R \mathbb{Z}_q$, and defines $w_{h^*} = s \cdot (u_{h^*} + u_{\mathcal{H} \setminus \{h^*\}} + u_c) - (w_{\mathcal{H} \setminus \{h^*\}} + w_c) \bmod q$ to embed the signature (Step 20).
6. It evaluates the final output by checking whether the signature algebraically reconstructs to s , issuing **proceed** or **fail** to $\mathcal{F}_{\text{mECDSA}}$ accordingly (Steps 21–23).

We show that experiments $\mathcal{H}_6$ and $\mathcal{H}_5$ are statistically indistinguishable. The modifications in experiment $\mathcal{H}_6$ mirror the cumulative sequence of changes established across experiments $\mathcal{H}_3$ through $\mathcal{H}_{10}$ in the proof of Theorem 5.2. We verify that each of the properties required by that argument is preserved in the monolithic setting:

1. *Distribution of R .* In both $\mathcal{H}_5$ and $\mathcal{H}_6$, the nonce point R is uniformly distributed over $\mathbb{G}$. In $\mathcal{H}_5$ this follows from the uniform sampling of the honest parties' nonce shares k_h combined with the commit-and-release mechanism; in $\mathcal{H}_6$ it follows from the uniform sampling performed internally by $\mathcal{F}_{\text{mECDSA}}$ (or by the simulator's uniform sampling in the failure case). Since the back-computation $R_{h^*} = R - \sum_{h \in \mathcal{H} \setminus \{h^*\}} R_h - \sum_{c \in \mathcal{C}} R_{c,h^*}$ is a deterministic function of uniformly and independently sampled components, the marginal distribution of each R_h is identical in both experiments.
2. *Distribution of masked key shares $\hat{Q}_h$.* For every $h \in \mathcal{H} \setminus \{h^*\}$, the values $\hat{x}_h$ are sampled uniformly and independently, and $\hat{Q}_h = \hat{x}_h \cdot G$ is therefore uniformly distributed over $\mathbb{G}$ in both experiments. The back-computed value $\hat{Q}_{h^*} = Q - \sum_{h \in \mathcal{H} \setminus \{h^*\}} \hat{Q}_h - \sum_{c \in \mathcal{C}} (Q_c + \zeta_c \cdot G)$ is uniquely determined by the public key Q , the other honest parties' shares, and the adversary's contributions; its marginal distribution is therefore uniform and consistent with the constraint $\sum_{i \in [n]} \hat{Q}_i = Q$.
3. *Extraction of corrupt inputs via $\mathcal{F}_{\text{RVOLE}[q,2]}$.* The simulator extracts the adversary's effective nonce contributions k_{c,h^*} and masked key shares $\hat{x}_{c,h^*}$ from the Bob-multiply messages sent to $\mathcal{F}_{\text{RVOLE}[q,2]}$. Conditioned on the consistency checks in Steps 19–20 passing (i.e., $k_{c,h} \cdot G = R_{c,h}$ and $\hat{x}_{c,h} \cdot G = \hat{Q}_{c,h}$ for all $c \in \mathcal{C}$ and $h \in \mathcal{H}$), these extracted values are the unique discrete logarithms of the committed curve points. This extraction mechanism is identical to the one used in the decoupled proof.
4. *Simulated values for honest parties $h \neq h^*$.* For every $h \in \mathcal{H} \setminus \{h^*\}$, the simulator samples $u_h, v_h, \phi_h \in_R \mathbb{Z}_q$ independently and computes $w_h = H(m_{h^*}) \cdot \phi_h + r_h \cdot v_h$. In the real protocol, u_h depends linearly on k_h, ϕ_h , and the RVOLE outputs $c_{h,j}^u + d_{h,j}^u$ for $j \neq h$. The honest-honest RVOLE outputs are sampled uniformly subject to constraints on their pairwise sums, providing exactly the degrees of freedom needed to ensure that (u_h, v_h, ϕ_h) are jointly uniform from the adversary's perspective. The adversary observes $\psi_{h,c}, R_h, \hat{Q}_h, C_{h,c}^u$, and $C_{h,c}^v$ for each corrupt c , but these values are computed from $\phi_h, k_h, \hat{x}_h, \chi_{c,h}$, and $d_{c,h}^u, d_{c,h}^v$ – none of which reveal information about the honest-honest cross-terms. Thus, the independent uniform sampling in the simulator produces an identical marginal distribution.
5. *Back-computation of w_{h^*} .* The simulator computes $w_{h^*} = s \cdot (u_{h^*} + u_{\mathcal{H} \setminus \{h^*\}} + u_{\mathcal{C}}) - (w_{\mathcal{H} \setminus \{h^*\}} + w_{\mathcal{C}})$, embedding the signature value s from $\mathcal{F}_{\text{mECDSA}}$ into the protocol transcript. In $\mathcal{H}_5$, the honest parties compute values such that $s = (\sum_{i \in [n]} w_i) / (\sum_{i \in [n]} u_i)$ holds exactly when no party deviates (and the honest parties have messages with identical images under H). The simulator's formula preserves this algebraic relationship by construction: $\sum_{i \in [n]} w_i = s \cdot \sum_{i \in [n]} u_i$. If the corrupt parties induce additive offsets into the output,

these offsets satisfy the same algebraic relationship with the embedded value of s in $\mathcal{H}_6$ as they do with the hypothetical value of s that would arise under honest execution in $\mathcal{H}_5$.

6. *Distribution of s .* In $\mathcal{H}_5$, the value s is determined by the honest protocol computation and is distributed identically to a signature produced by the standard ECDSA signing algorithm, conditioned on all honest parties having consistent messages. In $\mathcal{H}_6$, s is generated internally by the functionality $\mathcal{F}_{\text{mECDSA}}$. Since $\mathcal{F}_{\text{mECDSA}}$ computes signatures using the exact same standard ECDSA signing operations, both distributions are identical.

Since each of the above properties is preserved, the cumulative algebraic argument from the proof of Theorem 5.2 applies directly, and the total statistical distance introduced by this hybrid is bounded by the same negligible quantity established therein. Therefore,

$$\{\mathcal{H}_6(\kappa_c, z)\}_{\kappa_c \in \mathbb{N}, z \in \{0,1\}^{p(\kappa_c)}} \approx^s \{\mathcal{H}_5(\kappa_c, z)\}_{\kappa_c \in \mathbb{N}, z \in \{0,1\}^{p(\kappa_c)}}.$$

- **Experiment $\mathcal{H}_7(\kappa_c, z)$.** This experiment is an ideal-model computation of the monolithic functionality $\mathcal{F}_{\text{mECDSA}}$ with the simulator $\mathcal{S}$.

Note that in experiment $\mathcal{H}_6$ the emulator implements all steps of the simulator $\mathcal{S}$, and that the honest parties are dummy parties that forward inputs to and outputs from $\mathcal{F}_{\text{mECDSA}}$. Thus,

$$\{\mathcal{H}_7(\kappa_c, z)\}_{\kappa_c \in \mathbb{N}, z \in \{0,1\}^{p(\kappa_c)}} \equiv \{\text{IDEAL}_{\mathcal{F}_{\text{mECDSA}}, \mathcal{S}^A, z}(\kappa_c, z)\}_{\kappa_c \in \mathbb{N}, z \in \{0,1\}^{p(\kappa_c)}}.$$

■

D Proof of Theorem 6.1

Let $\mathcal{A}$ be a probabilistic polynomial-time adversary in the $\mathcal{F}$ -hybrid model, where $\mathcal{F} = (\mathcal{F}_{\text{RelaxedKeyGen}}[\mathbb{G}, n], \mathcal{F}_{\text{Com}}, \mathcal{F}_{\text{rZero}}[q, n], \mathcal{F}_{\text{RVOLE}}[q, 2])$. We define an ideal-model simulator $\mathcal{S}^-$ exactly as the simulator $\mathcal{S}$ presented in the proof of Theorem C.1, with the sole exception that $\mathcal{S}^-$ does not perform the explicit nonce-consistency checks on behalf of the honest parties. Specifically, in the output phase, $\mathcal{S}^-$ does not verify whether $\tilde{R}_h = \tilde{R}_{h'}$ or $\tilde{R}_{c,h} = \tilde{R}_h$. Instead, it allows each honest party P_h to proceed directly to the signature computation and local verification.

We define a sequence of hybrid experiments $\mathcal{H}_1^-, \dots, \mathcal{H}_7^-$ analogously to those in the proof of Theorem C.1. In $\mathcal{H}_1^-$, the environment interacts with the real execution of Π_{mECDSA}^- . We transition through the hybrid experiments identically to Appendix C, arriving at $\mathcal{H}_6^-$. As established in the proof of Theorem C.1, the statistical distance between each pair of consecutive hybrids up to $\mathcal{H}_6^-$ relies purely on the identical distributions of the simulated components, the extraction of the adversarial inputs via $\mathcal{F}_{\text{RVOLE}}[q, 2]$, and the substitution of the unobserved honest-honest cross terms with uniform one-time pads. These steps are entirely independent of the final output verification phase. Thus, the statistical indistinguishability of the sequence holds unconditionally up to $\mathcal{H}_6^-$.

The only divergence occurs between $\mathcal{H}_6^-$ and the final ideal execution $\mathcal{H}_7^-$. In $\mathcal{H}_7^-$ (the ideal model), the honest dummy parties simply output the exact target signature $\sigma = (r, s)$ provided by $\mathcal{F}_{\text{mECDSA}}$. In $\mathcal{H}_6^-$, an honest party P_h (where $h \neq h^*$) locally computes $s_h = (\sum_{j \in [n]} w_j) \cdot (\sum_{j \in [n]} u_j)^{-1} \bmod q$. Because the adversary is not constrained by the nonce-consistency check, it may provide inputs such that $\tilde{R}_h \neq R$, resulting in an x -coordinate $r_h \neq r$. If the resulting signature $\sigma_h = (r_h, s_h)$ successfully passes the local ECDSA verification equation, P_h will output σ_h . Since $r_h \neq r$, this output diverges from the ideal execution, allowing the environment $\mathcal{Z}$ to distinguish the two experiments.

Assume for the sake of contradiction that there exists a PPT environment $\mathcal{Z}$ that distinguishes $\mathcal{H}_6^-$ from $\mathcal{H}_7^-$ with non-negligible advantage $\epsilon(\kappa)$. This implies that in $\mathcal{H}_6^-$, with probability at least $\epsilon(\kappa)$, some honest party P_h (where $h \neq h^*$) outputs a valid signature $\sigma_h = (r_h, s_h)$ where $r_h \neq r$.

We construct a PPT adversary $\mathcal{B}$ that breaks the strong unforgeability up to sign of the ECDSA signature scheme. The adversary $\mathcal{B}$ interacts with an external ECDSA challenger, receiving a uniformly distributed challenge public key $Q \in \mathbb{G}$ and oracle access to a signing oracle $\mathcal{O}_{\text{sign}}(Q, \cdot)$. $\mathcal{B}$ emulates experiment $\mathcal{H}_6^-$ for the environment $\mathcal{Z}$ and the adversary $\mathcal{A}$ as follows:

- **Setup phase:** $\mathcal{B}$ emulates the functionality $\mathcal{F}_{\text{RelaxedKeyGen}}[\mathbb{G}, n]$ and the honest parties exactly as defined in $\mathcal{H}_6$. Specifically, it embeds the challenge public key Q into the key generation transcript (computing $Q_{h^*} \leftarrow Q - \sum_{h \neq h^*} Q_h - \sum_{c \in \mathcal{C}} Q_c$) and obtains the corrupted parties' public key shares $\{Q_c\}_{c \in \mathcal{C}}$, entirely without extracting their discrete logarithms.
- **Signing phase:** Whenever $\mathcal{Z}$ instructs the honest parties to sign a message m in a session sigid , $\mathcal{B}$ queries its signing oracle $\mathcal{O}_{\text{sign}}(Q, m)$ to obtain a valid signature $\sigma = (r, s)$. $\mathcal{B}$ then proceeds to emulate the signing phase exactly as defined in $\mathcal{H}_6$, using σ to back-compute R_{h^*} and w_{h^*} to embed the signature into the protocol transcript.
- **Output extraction:** At the end of each session sigid , $\mathcal{B}$ observes the values computed by every honest party P_h . Specifically, it evaluates $\tilde{R}_h$ (yielding x -coordinate r_h) and s_h . If $\sigma_h = (r_h, s_h)$ is a valid ECDSA signature on m under Q and $r_h \neq r$, $\mathcal{B}$ halts and outputs (m, σ_h) to the ECDSA challenger as its forgery.

Given that $\mathcal{B}$ explicitly utilizes the target signatures obtained from the signing oracle to perform the embedding, it perfectly simulates the view of $\mathcal{Z}$ and $\mathcal{A}$ in $\mathcal{H}_6^-$. Therefore, the probability that the distinguishing event occurs during the emulation is exactly $\epsilon(\kappa)$. When this event occurs, some honest party computes a valid signature σ_h with $r_h \neq r$. Since $r_h \neq r$, the signature σ_h is fundamentally distinct from σ and is not a trivial derivation via the $(r, \pm s)$ malleability of ECDSA.

Thus, $\mathcal{B}$ operates in polynomial time and outputs a valid forgery with probability $\epsilon(\kappa)$. Since $\epsilon(\kappa)$ is non-negligible, this contradicts the assumption that ECDSA is strongly unforgeable up to sign. We conclude that no such distinguishing environment $\mathcal{Z}$ can exist, and therefore Π_{mECDSA}^- computationally UC-realizes $\mathcal{F}_{\text{mECDSA}}$. ■